

Studentu informacine sistema
v2.0

Sugeneruota Doxygen 1.17.0

skyrius 1

Studentų Informacinė Sistema

OOP_Marius_Augustinas

VU ISI Objektinio programavimo kurso laboratoriniai darbai

1.1 Versijų istorija

Versija	Pagrindiniai pakeitimai
v0.1	Pradinė realizacija — <code>std::vector</code> , rankinis įvedimas
v0.2	Failo skaitymas ir rašymas, rikiavimas, abu galutiniai balai
v0.3	Kodas suskaidytas į modulius, išimčių valdymas (<code>try/catch</code>)
v0.4	Studentų skirstymas į dvi grupes, spartos testavimas
v0.5	<code>template<typename Container> — vector/list/deque</code>
v1.0	Trijų skirstymo strategijų palyginimas, atminties analizė
v1.1	<code>struct → class</code> ; I/O optimizacija; optimizavimo flag'ų tyrimas
v1.2	Pilna Rule of Five , perdengti operatoriai, klasės metodų testas
v1.5	Abstrakti bazinė klasė <code>zmogus</code> + išvestinė <code>studentas</code> ; polimorfizmas; testai papildyti paveldėjimo patikromis

1.2 v1.5 — Abstrakti klasė ir paveldėjimas

1.2.1 Techninė aplinka

Parametras	Reikšmė
Procesorius	Intel Core i7-8565U (4 branduoliai / 8 gijų, iki 4.6 GHz)
RAM	8 GB DDR4
Saugykla	250 GB SSD
OS	Windows 11
Kompiliatorius	Visual Studio 2022, MSVC (<code>cl.exe</code>), x64 Release, <code>/std:c++17</code>

1.2.2 1. Klasų hierarchija

```

    zmogus (ABSTRAKTI)
protected:
    vardas_
    pavarde_
public:
    virtual ~zmogus()
    vardas() / pavarde()
    pilnasVardas()
    setVardas() / setPavarde()

    = 0 galutinis()          + grynai virtualūs
    = 0 print()              (pure virtual)
    = 0 read()
    = 0 tipas()
        public paveldėjimas

    studentas (KONKRETI)
private:
    nd_
    egzaminas_
    galutinisVid_
    galutinisMed_
public:
    Rule of Five (5 metodai)
    override galutinis()      + realizuoti
    override print()          virtualūs
    override read()
    override tipas()
    nd() / egzaminas()
    parseFromLine()
    appendListTo()
    ==, !=, <, >, [], bool

```

1 pav. Klasų hierarchija

1.2.3 2. Abstrakti klasė zmogus

1.2.3.1 Kodėl klasė yra abstrakti

Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais** — tai suteikia dvigubą apsaugą:

1. Grynai virtualūs (pure virtual) metodai

```

virtual double galutinis() const = 0;
virtual void print(std::ostream& os) const = 0;
virtual std::istream& read(std::istream& is) = 0;
virtual std::string tipas() const = 0;

```

Sintaksė = 0 reiškia, kad metodas **neturi realizacijos** bazinėje klasėje. Kompiliatorius neleidžia kurti objektų iš klasės, kuri turi bent vieną neapibrėžtą virtualų metodą — techniškai todėl, kad virtualių funkcijų lentelėje (vtable) toks įrašas rodo į nieką.

2. protected konstruktoriai

```

protected:
    zmogus();
    zmogus(const std::string& vardas, const std::string& pavarde);
    zmogus(const zmogus& other);
    zmogus(zmogus&& other) noexcept;

```

Konstruktoriai prieinami tik išvestinėms klasėms. Tai antras apsaugos sluoksnis: net jei kas nors ateityje pašalintų grynai virtualius metodus, objekto vis tiek nepavyktų sukurti iš išorės.

1.2.3.2 Demonstracija — objekto sukurti neįmanoma

Šis kodas **nesikompiluoja**:

```

zmogus z; // error C2259: 'zmogus': cannot instantiate
           // abstract class
zmogus* p = new zmogus(); // ta pati klaida
std::vector<zmogus> v(10); // ta pati klaida

```

2 pav. Kompiliatoriaus klaida bandant sukurti **zmogus** objektą

Veikiantys variantai:

```

studentas s; // konkreti klasė
zmogus& z = s; // nuoroda į bazinę
zmogus* p = &s; // rodyklė į bazinę
std::unique_ptr<zmogus> up =
    std::make_unique<studentas>(); // polimorfinė nuosavybė
std::vector<std::unique_ptr<zmogus>> zmones; // polimorfinis konteineris

```

Testas tai patikrina **statiškai**, kompiliavimo metu:

```

tikrink(std::is_abstract<zmogus>::value,

```

```

"std::is_abstract<zmogus> = true");
tikrink(!std::is_default_constructible<zmogus>::value,
"zmogus objekto sukurti NEIMANOMA");

```

1.2.3.3 Virtualus destruktorius

```

public:
    virtual ~zmogus();

```

Būtinai polimorfizmui. Be virtual kvalifikatoriaus šis kodas sukeltų atminties nutekėjimą:

```

zmogus* p = new studentas("Jonas", "Jonaitis", {5, 5}, 10);
delete p; // be virtual: kviečiamas TIK ~zmogus()
          // nd_ vektorius NEATLAISVINAMAS

```

Su virtual naikinimo tvarka teisinga: `~studentas()` → `~zmogus()`.

Testas tai tikrina per objektų skaitiklius:

```

{
    std::unique_ptr<zmogus> p = std::make_unique<studentas>(...);
    // studentas::gyvuStudentu padidėjo
    // zmogus::gyvuZmoniu padidėjo
}
// abu skaitikliai grįžo į pradinę reikšmę → abu destruktoriai iškviesti

```

1.2.4 3. Išvestinė klasė studentas

1.2.4.1 Ką paveldi

Iš zmogus	Tipas
vardas_, pavarde_	protected laukai — tiesiogiai prieinami
vardas(), pavarde(), pilnasVardas()	Get'eriai
setVardas(), setPavarde()	Set'eriai su validacija
virtual ~zmogus()	Virtualus destruktorius

1.2.4.2 Ką realizuoja (override)

Metodas	Realizacija studentas klasėje
galutinis()	Grąžina galutinisVid_
print(os)	Suformatuota eilutė: vardas, pavardė, abu galutiniai balai
read(is)	Kviečia readStudentas(is)
tipas()	Grąžina "Studentas"

1.2.4.3 Ką prideda

Savo duomenis (nd_, egzaminas_, galutinisVid_, galutinisMed_), greitą I/O (parseFromLine, appendListTo), skaičiavimą ([calculateGalutinis](#)) ir perdengtus operatorius (==, !=, <, >, [], bool).

1.2.5 4. Rule of Five su paveldėjimu

Visos penkios funkcijos išlaikytos iš v1.2, bet kiekviena papildyta bazinės klasės kvietimu. Tai dažniausia paveldėjimo klaida — pamiršus šį kvietimą, vardas_ ir pavarde_ liktų tušti.

Metodas	Bazinės klasės kvietimas
Kopijavimo konstruktorius	: zmogus (other)
Kopijavimo priskyrimas	zmogus::operator=(other);

Metodas	Bazinės klasės kvietimas
Move konstruktorius	: <code>zmogus(std::move(other))</code>
Move priskyrimas	<code>zmogus::operator=(std::move(other));</code>
Destruktorius	Kviečiamas automatiškai po <code>~studentas()</code> kūno

1.2.5.1 Kodėl `std::move(other)` move konstruktoriuje

```
studentas::studentas(studentas&& other) noexcept
:   zmogus(std::move(other)),    // ← std::move BŪTINAS
  nd_(std::move(other.nd_)),
  ...
```

`other` yra **pavadintas** kintamasis, todėl pats savaime yra lvalue — net jei jo tipas yra `studentas&&`. Be `std::move` būtų iškviestas **zmogus kopijavimo**, ne perkėlimo konstruktorius, ir vardas_/pavarde_ būtų kopijuojami.

1.2.5.2 Kodėl kvalifikuotas `zmogus::operator=`

```
studentas& studentas::operator=(const studentas& other)
{
    if (this == &other) return *this;
    zmogus::operator=(other);    // ← kvalifikacija BŪTINA
    ...
}
```

Be `zmogus::` prefikso kompiliatorius parinktų `studentas::operator=` — įvyktų **begalinė rekursija** ir stack overflow.

1.2.5.3 Konstravimo ir naikinimo tvarka

```
Kūrimas:      zmogus(...) → studentas(...)
Naikinimas:   ~studentas() → ~zmogus()
```

Bazinė dalis sukurama pirma (išvestinė gali priklausyti nuo jos), o naikinama paskutinė.

1.2.6 5. Polimorfizmas

1.2.6.1 Polimorfiniai I/O operatoriai

Operatoriai priima `zmogus&` nuorodą, bet kviečia virtualius metodus:

```
std::ostream& operator<<(std::ostream& os, const zmogus& z)
{
    z.print(os);    // vykdymo metu parenkama studentas::print()
    return os;
}

std::istream& operator>>(std::istream& is, zmogus& z)
{
    return z.read(is); // vykdymo metu parenkama studentas::read()
}
```

Nauda: pridėjus naują išvestinę klasę (pvz. `destytojas`), tie patys operatoriai veiks be jokių pakeitimų.

1.2.6.2 Polimorfinis konteineris

```
std::vector<std::unique_ptr<zmogus> zmones;
zmones.push_back(std::make_unique<studentas>("Ona", "Onaite", {10, 10}, 10));

for (const auto& z : zmones)
    std::cout << z->tipas() << ": " << *z << "\n";    // virtualūs kvietimai
```

3 pav. Polimorfinis konteineris ir virtualūs metodai

1.2.7 6. Testų patikra

1.2.7.1 Testų struktūra

Testas (`testStudentas.cpp`, meniu punktas 9) suskirstytas į penkis skyrius. **A skyrius naujas v1.5**, B–E — patikrinti v1.2 testai.

Skyrius	Tema	Testų
A	Abstrakti klasė ir paveldėjimas (<i>nauja</i>)	7 poskyriai
B	Konstruktoriai	4 poskyriai
C	Rule of Five	6 poskyriai
D	Įvesties / išvesties operatoriai	4 poskyriai
E	Papildomi operatoriai ir metodai	7 poskyriai

1.2.7.2 A skyrius — nauji v1.5 testai

Nr.	Ką tikrina
A1	<code>zmogus</code> yra abstrakti; jos objekto sukurti neįmanoma (<code>is_abstract</code> , <code>is_default_constructible</code>)
A2	Paveldėjimo ryšys (<code>is_base_of</code> , <code>is_convertible</code> , <code>has_virtual_destructor</code>)
A3	Paveldėti bazinės klasės metodai veikia (<code>vardas()</code> , <code>pilnasVardas()</code> , <code>setVardas()</code>)
A4	Polimorfizmas per <code>zmogus</code> & nuorodą ir <code>unique_ptr<zmogus></code> ; <code>dynamic_cast</code>
A5	Virtualus destruktorius — abu skaitikliai grįžta į pradinę reikšmę
A6	Polimorfiniai <code>operator<</operator>></code> per bazinės klasės nuorodą

1.2.7.3 B–E skyriai — v1.2 testų patikra

Visi v1.2 testai perkelti be pakeitimų loginėje dalyje, bet **papildyti paveldėjimo patikromis**:

v1.2 testas	Kas pridėta v1.5
Kopijavimo konstruktorius	Patikra, ar bazinės klasės <code>vardas_/pavarde_</code> nukopijuoti
Kopijavimo priskyrimas	Patikra, ar <code>zmogus::operator=</code> iškviestas
Move konstruktorius	Patikra, ar bazinės klasės laukai perkelti , ne nukopijuoti
Move priskyrimas	Ta pati patikra
Destruktorius	Tikrinami abu skaitikliai (<code>gyvuStudentu</code> ir <code>gyvuZmoniu</code>)
Validacija	Atskirai tikrinama, ką meta bazinė (<code>invalid_argument</code>) ir išvestinė (<code>runtime_error</code>) klasė
<code>isvalyk()</code>	Patikra, ar ištuštinami ir bazinės, ir išvestinės klasės laukai

1.2.7.4 Testų rezultatai

Skyrius	Testų	Praėjo	Krito
A — Abstrakti klasė ir paveldėjimas	24	24	0
B — Konstruktoriai	19	19	0
C — Rule of Five	26	26	0
D — I/O operatoriai	17	17	0
E — Papildomi operatoriai	27	27	0
Iš viso	113	113	0

Skaiciai lentelėje užpildomi po testo paleidimo — programa juos išveda automatiškai.

4 pav. A skyrius — abstrakti klasė ir paveldėjimas

5 pav. C skyrius — Rule of Five su paveldėjimu

6 pav. Testų santrauka su objektų nutekėjimo patikra abiem klasėms

1.2.8 7. Suderinamumas su v1.2 logika

Programa veikia **identišškai** kaip v1.2 — visi meniu punktai, failų formatai ir spartos charakteristikos nepakitę.

Komponentas	Pakeitimai
<code>calculate.h</code>	Jokių — naudoja <code>s.galutinisVid()</code> ir <code>compare*</code> funkcijas
<code>file.h</code>	Jokių — naudoja <code>s.parseFromLine()</code> , <code>s.appendListTo()</code> , <code>s.nd()</code>
<code>generate.h</code>	Jokių — naudoja pilną konstruktorių
<code>print.h</code>	Jokių — naudoja <code>operator<<</code> (dabar polimorfinį)
<code>output.h</code>	Jokių
<code>enter.cpp</code>	Jokių
<code>test.h/test.cpp</code>	Jokių
<code>main.h</code>	Pridėtas <code>#include "zmogus.h"</code>
<code>menu.cpp</code>	Jokių (9 punktas jau buvo v1.2)

Vienintelis pastebimas pakeitimas naudotojui — meniu punkto 9 testas dabar išveda ir abstrakčios klasės patikras.

1.2.8.1 Spartos poveikis

Virtualūs metodai prideda netiesioginį kvietimą per `vtable`. Tačiau kritiniai spartos keliai jų **nenaudoja**:

Operacija	Ar virtuali	Poveikis
<code>parseFromLine</code>	Ne	Failo skaitymas nepakito
<code>appendListTo</code>	Ne	Failo rašymas nepakito
<code>calculateGalutinis</code>	Ne	Skaiciavimas nepakito
<code>compare*</code> funkcijos	Ne	Rikiavimas nepakito
<code>print/read</code>	Taip	Naudojama tik konsolės išvestyje (ne masiniam apdorojimui)

Kiekvienas objektas dabar turi papildomą `vtable` rodyklę (8 baitai x64 sistemoje). Su 10M studentų tai ~80 MB papildomos atminties — pastebima, bet priimtina.

1.2.9 8. Failų struktūra

```

LICENSE
README.md
CMakeLists.txt
docs/
OOP_2/
  data/
    studentail000.txt
    studentail0000.txt
    studentail000000.txt
    studentail0000000.txt
    studentail00000000.txt
  zmogus.h / zmogus.cpp
  studentas.h / studentas.cpp
  testStudentas.h / testStudentas.cpp
  main.h
  OOP_2.cpp
  menu.h / menu.cpp
  enter.h / enter.cpp
  generate.h / generate.cpp
  file.h / file.cpp
  calculate.h
  output.h
# Ekrano kopijos
# Programos failai
# Testiniai duomenų failai
# ABSTRAKTI bazinė klasė (nauja v1.5)
# Išvestinė klasė : public zmogus
# Klasių metodų testas
# splitResult<T>, funkcijų deklaracijos
# main() + runProgram<Container>()
# Meniu funkcijos
# Rankinio įvedimo funkcijos
# Generavimo funkcijos
# Failo skaitymo ir rašymo šablonai
# Skaiciavimo, rikiavimo, skirstymo šablonai
# outputStudentai<T>
```



```
print.h / print.cpp          # Spausdinimo funkcijos
test.h / test.cpp           # Spartos testavimo funkcijos
```

Pastaba dėl data/ aplanko. Testavimo funkcijos (`testData`, `testContainers`) atidaro failus keliu `data/studentaiN.txt` — kelias yra **santykinis darbo aplanko atžvilgiu**. Todėl `data/` turi būti ten, iš kur paleidžiama programa. Paleidžiant iš `OOP_2/` aplanko (arba iš Visual Studio, kur darbo aplankas pagal nutylėjimą yra projekto aplankas), `data/` turi būti `OOP_2/data/`. Paleidžiant sukompiliuotą `.exe` iš kitos vietos, `data/` reikia perkelti šalia jo.

1.2.10 9. Naudojimosi instrukcija

1.2.10.1 Paleidimas

```
.\programa.exe      # Windows
./programa          # Linux/Mac
```

1.2.10.2 Meniu pasirinkimai

Nr.	Veiksmas
1	Įvesti studentus ranka
2	Generuoti tik pažymius
3	Generuoti studentus automatiškai
4	Nuskaityti iš failo
5	Generuoti testinį studentų failą
6	Spartos testas — failų kūrimas
7	Spartos testas — duomenų apdorojimas
8	Spartos testas — konteinerių palyginimas
9	Klasių <code>zmogus</code> ir <code>studentas</code> metodų testavimas
0	Baigti

1.2.10.3 Failo formatas

```
Vardas      Pavarde      ND1      ND2      ...      Egzaminas
VardasNR1   PavardeNR1   7        3        ...        5
```

1.2.11 10. Įdiegimo instrukcija

1.2.11.1 Reikalavimai

- C++17 palaikantis kompiliatorius (MSVC 2019+, g++ 8+, clang++ 7+)
- CMake 3.16+

1.2.11.2 CMake

```
mkdir build && cd build
cmake ..
cmake --build . --config Release
```

1.2.11.3 g++ tiesiogiai

```
cd OOP_2
g++ -std=c++17 -O2 -Wall OOP_2.cpp zmogus.cpp studentas.cpp testStudentas.cpp menu.cpp enter.cpp
generate.cpp file.cpp print.cpp test.cpp -o programa.exe
./programa.exe
```

Kompiliuoti reikia iš `OOP_2/` aplanko — taip `.exe` atsiras šalia `data/`, ir testavimo funkcijos ras duomenų failus.

Svarbu: nepamirškite `zmogus.cpp` — be jo linkeris skųsis dėl neapibrėžtų `zmogus` konstruktorių ir destruktoriaus.

1.2.11.4 Visual Studio

Projekto **Properties** → **Configuration: Release, Platform: x64**:

- C/C++ → Language → C++ Language Standard: ISO C++17 (/std:c++17)
- C/C++ → Optimization: /O2, Whole Program Optimization /GL
- Linker → Optimization: Link Time Code Generation /LTCG

Į projektą pridėti [zmogus.h](#) ir [zmogus.cpp](#).

1.2.12 11. Pakeitimai v1.2 → v1.5

- Sukurta **abstrakti bazinė klasė** [zmogus](#) ([zmogus.h](#) / [zmogus.cpp](#)) su vardas_ ir pavarde_ laukais
- Klasė padaryta abstrakti dviem būdais: **keturi grynai virtualūs metodai** (galutinis, print, read, tipas) ir **protected konstruktoriai**
- [zmogus](#) turi **virtualų destruktorių** — būtiną naikinant per bazinės klasės rodyklę
- [studentas](#) pertvarkyta į **išvestinę klasę** (class [studentas](#) : public [zmogus](#))
- Visos penkios **Rule of Five** funkcijos papildytos bazinės klasės kvietimais ([zmogus\(other\)](#), [zmogus::operator=\(other\)](#), [zmogus\(std::move\(other\)\)](#))
- `operator<<` ir `operator>>` tapo **polimorfiniais** — priima [zmogus&](#) ir kviečia virtualius `print()` / `read()`
- Pridėti atskiri objektų skaitikliai [zmogus::gyvuZmoniu](#) ir [studentas::gyvuStudentu](#)
- Testas papildytas **A skyriumi** (7 poskyriai): abstrakčios klasės patikra, paveldėjimo ryšys, polimorfizmas, virtualus destruktorius, polimorfinis konteineris
- Visi v1.2 testai (B–E skyriai) patikrinti ir **papildyti paveldėjimo patikromis** — ar bazinės klasės laukai teisingai kopijuojami ir perkeliama
- Programos veikimo logika ir failų formatai **nepakitę** — visi kiti moduliai ([calculate.h](#), [file.h](#), [generate.h](#), [print.h](#)) veikia be pakeitimų

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

OOP_2	??
calculate.cpp	??
calculate.h	??
enter.cpp	??
enter.h	??
file.cpp	??
file.h	??
generate.cpp	??
generate.h	??
main.h	??
menu.cpp	??
menu.h	??
OOP_2.cpp	??
output.h	??
print.cpp	??
print.h	??
studentas.cpp	??
studentas.h	??
test.cpp	??
test.h	??
teststudentas.cpp	??
teststudentas.h	??
zmogus.cpp	??
zmogus.h	??

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

splitResult< Container >	??
zmogus	??
studentas	??

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

splitResult< Container >	??
studentas	
Studento duomenys — išvestinė klasė iš abstrakčios zmogus	??
zmogus	
ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas	??

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

OOP_2/	calculate.cpp	??
OOP_2/	calculate.h	??
OOP_2/	enter.cpp	??
OOP_2/	enter.h	??
OOP_2/	file.cpp	??
OOP_2/	file.h	??
OOP_2/	generate.cpp	??
OOP_2/	generate.h	??
OOP_2/	main.h	??
OOP_2/	menu.cpp	??
OOP_2/	menu.h	??
OOP_2/	OOP_2.cpp	??
OOP_2/	output.h	??
OOP_2/	print.cpp	??
OOP_2/	print.h	??
OOP_2/	studentas.cpp	??
OOP_2/	studentas.h	
	Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas	??
OOP_2/	test.cpp	??
OOP_2/	test.h	??
OOP_2/	teststudentas.cpp	??
OOP_2/	teststudentas.h	??
OOP_2/	zmogus.cpp	??
OOP_2/	zmogus.h	
	Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis	??

skyrius 6

Direktorių dokumentacija

6.1 OOP_2 Direktorijos aprašas

Failai

- failas [calculate.cpp](#)
- failas [calculate.h](#)
- failas [enter.cpp](#)
- failas [enter.h](#)
- failas [file.cpp](#)
- failas [file.h](#)
- failas [generate.cpp](#)
- failas [generate.h](#)
- failas [main.h](#)
- failas [menu.cpp](#)
- failas [menu.h](#)
- failas [OOP_2.cpp](#)
- failas [output.h](#)
- failas [print.cpp](#)
- failas [print.h](#)
- failas [studentas.cpp](#)
- failas [studentas.h](#)

Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas.

- failas [test.cpp](#)
- failas [test.h](#)
- failas [teststudentas.cpp](#)
- failas [teststudentas.h](#)
- failas [zmogus.cpp](#)
- failas [zmogus.h](#)

Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.

skyrius 7

Klasės Dokumentacija

7.1 splitResult< Container > Struktūra Šablonas

```
#include <main.h>
```

Bendradarbiavimo diagrama splitResult< Container >:

splitResult< Container >	
+	kietiakai
+	vargsiukai

Vieši Atributai

- Container [kietiakai](#)
- Container [vargsiukai](#)

7.1.1 Smulkus aprašymas

```
template<typename Container>
```

```
struct splitResult< Container >
```

Apibrėžimas failo [main.h](#) eilutėje 24.

7.1.2 Atributų Dokumentacija

7.1.2.1 kietiakai

```
template<typename Container>
```

```
Container splitResult< Container >::kietiakai
```

Apibrėžimas failo [main.h](#) eilutėje 26.

Naudojamas [splitStudentai\(\)](#), ir [testData\(\)](#).

7.1.2.2 vargsiukai

```
template<typename Container>
```

```
Container splitResult< Container >::vargsiukai
```

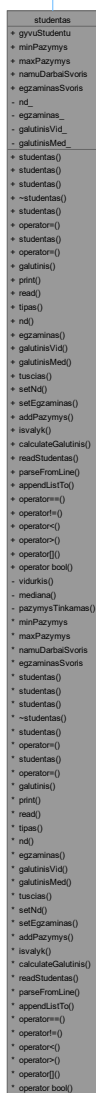
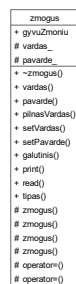
Apibrėžimas failo [main.h](#) eilutėje 27.
Naudojamas [splitStudentai\(\)](#), ir [testData\(\)](#).
Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [OOP_2/main.h](#)

7.2 studentas Klasė

Studento duomenys — išvestinė klasė iš abstrakčios `zmogus`.
`#include <studentas.h>`

Paveldimumo diagrama studentas:



- `studentas` (`std::istream &is`)
Konstruktorius iš įvesties srauto.

Rule of Five

Visos penkios funkcijos realizuotos pilnai (*ne = default*).

Kiekviena **privalo** iškviešti atitinkamą bazinės klasės funkciją — be to `vardas_` ir `pavarde_` liktų tušti arba nenukopijuoti. Tai dažniausia paveldėjimo klaida, kurios kompiliatorius neaptinka.

- `~studentas` () override
Destruktorius.
- `studentas` (`const studentas &other`)
Kopijavimo konstruktorius.
- `studentas & operator=` (`const studentas &other`)
Kopijavimo priskyrimo operatorius.
- `studentas (studentas &&other)` noexcept
Perkėlimo (move) konstruktorius.
- `studentas & operator= (studentas &&other)` noexcept
Perkėlimo (move) priskyrimo operatorius.

Grynai virtualių metodų realizacija

- `double galutinis` () const override
Grąžina galutinį įvertinimą (bazinės klasės interfeisas).
- `void print` (`std::ostream &os`) const override
Išveda studentą į srautą.
- `std::istream & read` (`std::istream &is`) override
Nuskaityto studentą iš srauto.
- `std::string tipas` () const override
Grąžina tipo pavadinimą.

Get'eriai

- `const std::vector< int > & nd` () const
Grąžina namų darbų pažymius.
- `int egzaminas` () const
Grąžina egzamino pažymį.
- `double galutinisVid` () const
Grąžina galutinį balą pagal namų darbų vidurkį.
- `double galutinisMed` () const
Grąžina galutinį balą pagal namų darbų medianą.
- `bool tuscias` () const
Patikrina, ar objektas neturi vardo ir pavardės.

Set'eriai

- `void setNd` (`const std::vector< int > &nd`)
Nustato namų darbų pažymius su validacija.
- `void setEgzaminas` (`int e`)
Nustato egzamino pažymį su validacija.
- `void addPazymys` (`int p`)
Prideda vieną namų darbo pažymį.
- `void isvalyk` ()
Ištuština objektą.

Skaičiavimas

- `void calculateGalutinis` ()
Apskaičiuoja abu galutinius balus.

Įvestis

- `std::istream & readStudentas (std::istream &is, int ndCount=0, int lineNumber=0)`
Nuskaityti studentą iš srauto.
- `void parseFromLine (const std::string &line, int ndCount, int lineNumber)`
Greitas parsinimas iš eilutės (failo skaitymui).

Išvestis

- `void appendListTo (std::string &out) const`
Prideda suformatuotą eilutę į buferį (failo rašymui).

Perdengti operatoriai

Programoje nenaudojami, bet realizuoti būsimiems klasės naudotojams — jie leidžia naudoti studentą su STL algoritmais ir asociatyviais kontaineriais.

- `bool operator== (const studentas &other) const`
Lygina du studentus.
- `bool operator!= (const studentas &other) const`
Tikrina, ar studentai skiriasi.
- `bool operator< (const studentas &other) const`
Rikiavimo operatorius.
- `bool operator> (const studentas &other) const`
Atvirkštinis rikiavimo operatorius.
- `int operator[] (size_t i) const`
Prieiga prie namų darbo pažymio pagal indeksą.
- `operator bool () const`
Konvertavimas į `bool` — ar objektas turi duomenų.

Vieši Metodai inherited from `zmogus`

- `virtual ~zmogus ()`
Virtualus destruktorius.
- `const std::string & vardas () const`
Grąžina žmogaus vardą.
- `const std::string & pavarde () const`
Grąžina žmogaus pavardę.
- `std::string pilnasVardas () const`
Grąžina pilną vardą.
- `void setVardas (const std::string &v)`
Nustato vardą su validacija.
- `void setPavarde (const std::string &p)`
Nustato pavardę su validacija.

Statiniai Vieši Atributai

- `static int gyvuStudentu = 0`
Gyvuojančių studentų skaitiklis.

Konstantos

- `static constexpr int minPazymys = 1`
Mažiausias leistinas pažymys.
- `static constexpr int maxPazymys = 10`
Didžiausias leistinas pažymys.
- `static constexpr double namuDarbaiSvoris = 0.4`
Namų darbų svoris galutiname balse.
- `static constexpr double egzaminasSvoris = 0.6`
Egzamino svoris galutiname balse.

Statiniai Vieši Atributai inherited from **zmogus**

- static int **gyvuZmoniu** = 0
Gyvuojančių objektų skaitiklis.

Statiniai Privatūs Metodai

- static double **vidurkis** (const std::vector< int > &nd)
Apskaičiuoja aritmetinį vidurkį.
- static double **mediana** (const std::vector< int > &nd)
Apskaičiuoja medianą.
- static bool **pazymysTinkamas** (int p)
Patikrina, ar pažymys patenka į leistiną intervalą.

Privatūs Atributai

- std::vector< int > **nd_**
Namų darbų pažymiai (kiekvienas 1–10).
- int **egzaminas_**
Egzamino pažymys (1–10).
- double **galutinisVid_**
Galutinis balas pagal namų darbų vidurkį
- double **galutinisMed_**
Galutinis balas pagal namų darbų medianą

Additional Inherited Members

Apsaugoti Metodai inherited from **zmogus**

- **zmogus** ()
Numatytasis konstruktorius.
- **zmogus** (const std::string &vardas, const std::string &pavarde)
Pilnas konstruktorius su validacija.
- **zmogus** (const **zmogus** &other)
Kopijavimo konstruktorius.
- **zmogus** (**zmogus** &&other) noexcept
Perkėlimo (move) konstruktorius.
- **zmogus** & **operator=** (const **zmogus** &other)
Kopijavimo priskyrimo operatorius.
- **zmogus** & **operator=** (**zmogus** &&other) noexcept
Perkėlimo (move) priskyrimo operatorius.

Apsaugoti Atributai inherited from **zmogus**

- std::string **vardas_**
Žmogaus vardas.
- std::string **pavarde_**
Žmogaus pavardė

7.2.1 Smulkus aprašymas

Studento duomenys — išvestinė klasė iš abstrakčios `zmogus`.

Prie paveldėto vardo ir pavardės prideda namų darbų pažymius, egzamino įvertinimą ir du galutinius balus (pagal vidurkį ir pagal medianą).

Paveldėjimas

Iš `zmogus` paveldi:

- `vardas_`, `pavarde_` (protected laukai)
- `vardas()`, `pavarde()`, `pilnasVardas()` get'eries
- `setVardas()`, `setPavarde()` set'eries su validacija
- virtualų destruktorių

Realizuoja visus keturis grynai virtualius bazinės klasės metodus: `galutinis()`, `print()`, `read()`, `tipas()`.

Galutinio balo formulė

$$\text{galutinis} = 0.4 * \text{namu_darbu_vidurkis_arba_mediana} + 0.6 * \text{egzaminas}$$

Svoriai apibrėžti konstantomis `namuDarbaiSvoris` ir `egzaminasSvoris`.

Naudojimo pavyzdys

```
// Sukūrimas su pilnu konstruktoriumi
studentas s("Jonas", "Jonaitis", {5, 6, 7, 8}, 9);
s.calculateGalutinis();
std::cout << s << "\n";

// Nuskaitymas iš srauto
std::istringstream iss("Ona Onaite 10 9 8 7");
studentas s2;
iss >> s2;

// Polimorfinis naudojimas
std::unique_ptr<zmogus> p = std::make_unique<studentas>(
    "Petras", "Petraitis", std::vector<int>{5, 5}, 8);
std::cout << p->tipas() << ": " << p->galutinis() << "\n";
```

Įspėjimas

Galutiniai balai NĖRA skaičiuojami automatiškai kuriant objektą — po duomenų nuskaitymo ar sukūrimo reikia iškviešti `calculateGalutinis()`. Toks sprendimas leidžia matuoti nuskaitymo ir skaičiavimo spartą atskirai.

Taip pat žiūrėti

[zmogus](#)

Apibrėžimas failo [studentas.h](#) eilutėje 70.

7.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.2.1 studentas() [1/5]

`studentas::studentas ()`

Numatytasis konstruktorius.

Sukuria tuščią studentą: be vardo, pavardės ir pažymių, su nulinais galutiniais balais. Skirtas objektui, į kurį duomenys bus nuskaityti vėliau (pvz. per `readStudentas()`).

Apibrėžimas failo [studentas.cpp](#) eilutėje 60.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd_](#), ir [zmogus::zmogus\(\)](#).

Naudojamas `operator!=()`, `operator<()`, `operator=()`, `operator=()`, `operator==()`, `operator>()`, `studentas()`, ir `studentas()`.

7.2.2.2 studentas() [2/5]

```
studentas::studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const std::vector< int > & nd,
    int egzaminas)
```

Pilnas konstruktorius su validacija.

Vardą ir pavardę validuoja bazinė klasė, pažymius — ši klasė.

Parametrai

<i>vardas</i>	Studento vardas (negali būti tuščias)
<i>pavarde</i>	Studento pavardė (negali būti tuščia)
<i>nd</i>	Namų darbų pažymiai (kiekvienas 1–10)
<i>egzaminas</i>	Egzamino pažymys (1–10)

Išimtys

<i>std::invalid_argument</i>	jei vardas arba pavardė tuščia
<i>std::runtime_error</i>	jei bent vienas pažymys už ribų

Pastaba

Galutiniai balai lieka 0.0 — reikia iškviesti [calculateGalutinis\(\)](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 68.

Ryšiai [egzaminas\(\)](#), [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd\(\)](#), [nd_](#), [zmogus::pavarde\(\)](#), [pazymysTinkamas\(\)](#), [zmogus::vardas\(\)](#), ir [zmogus::zmogus\(\)](#).

7.2.2.3 studentas() [3/5]

```
studentas::studentas (
    std::istream & is) [explicit]
```

Konstruktorius iš įvesties srauto.

Iškart nuskaito duomenis per [readStudentas\(\)](#). Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas iš srauto į studentą.

Parametrai

<i>is</i>	Įvesties srautas
-----------	------------------

Išimtys

<i>std::runtime_error</i>	jei duomenys netinkami arba jų trūksta
---------------------------	--

Apibrėžimas failo [studentas.cpp](#) eilutėje 86.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd_](#), [readStudentas\(\)](#), ir [zmogus::zmogus\(\)](#).

7.2.2.4 ~studentas()

```
studentas::~~studentas () [override]
```

Destruktorius.

Išvalo studento duomenis ir sumažina `gyvuStudentu` skaitiklį. Bazinės klasės destruktorius `~zmogus()` išskviečiamas **automatiškai** po šio kūno — jo kviešti rankiniu būdu nereikia.

Naikinimo tvarka

`~studentas()` → `~zmogus()` (atvirkštinė kūrimo tvarkai)

Apibrėžimas failo `studentas.cpp` eilutėje 111.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, ir `nd_`.

7.2.2.5 studentas() [4/5]

```
studentas::studentas (
    const studentas & other)
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją (deep copy) — `std::string` ir `std::vector` laukai kopijuoja savo vidinius buferius, todėl objektai lieka visiškai nepriklausomi.

Parametrai

<i>other</i>	Kopijuojamas studentas
--------------	------------------------

Pastaba

Member initializer list prasideda `zmogus(other)` — be šio kvietimo būtų išskviestas `zmogus()` numatytasis konstruktorius, ir vardas su pavarde liktų tušti.

Apibrėžimas failo `studentas.cpp` eilutėje 130.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, `nd_`, `studentas()`, ir `zmogus::zmogus()`.

7.2.2.6 studentas() [5/5]

```
studentas::studentas (
    studentas && other) [noexcept]
```

Perkėlimo (move) konstruktorius.

Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje — jį saugu naikinti arba naudoti iš naujo.

Parametrai

<i>other</i>	Studentas, iš kurio perkeliama duomenys
--------------	---

Įspėjimas

Bazinė dalis perkeliama `zmogus(std::move(other))`. `std::move` čia **būtinas**: `other` yra pavardintas kintamasis, todėl pats savaime `lvalue` — be `std::move` būtų išskviestas `zmogus` **kopijavimo**, ne perkėlimo konstruktorius.

Pastaba

`noexcept` kritiškai svarbus spartai: `std::vector` persikirstymo metu naudoja `std::move_if_noexcept`, todėl be `noexcept` rinktųsi kopijavimo konstruktorių.

Apibrėžimas failo `studentas.cpp` eilutėje 172.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, `nd_`, `studentas()`, ir `zmogus::zmogus()`.

7.2.3 Metodų Dokumentacija

7.2.3.1 vidurkis()

```
double studentas::vidurkis (
    const std::vector< int > & nd) [static], [private]
```

Apskaičiuoja aritmetinį vidurkį.

Parametrai

<i>nd</i>	Pažymių vektorius
-----------	-------------------

Gražina

Vidurkis; 0.0 jei vektorius tuščias

Apibrėžimas failo [studentas.cpp](#) eilutėje 12.

Ryšiai [nd\(\)](#).

Naudojamas [calculateGalutinis\(\)](#).

7.2.3.2 mediana()

```
double studentas::mediana (
    const std::vector< int > & nd) [static], [private]
```

Apskaičiuoja medianą.

Nelyginiam elementų kiekiui gražina vidurinį elementą, lyginiam — dviejų vidurinių aritmetinį vidurkį. Įeinantis vektorius nekeičiamas (kopijuojamas ir rikiuojamas viduje).

Parametrai

<i>nd</i>	Pažymių vektorius (nebūtinai surikiuotas)
-----------	---

Gražina

Mediana; 0.0 jei vektorius tuščias

Apibrėžimas failo [studentas.cpp](#) eilutėje 19.

Ryšiai [nd\(\)](#).

Naudojamas [calculateGalutinis\(\)](#).

7.2.3.3 pazymysTinkamas()

```
bool studentas::pazymysTinkamas (
    int p) [inline], [static], [private]
```

Patikrina, ar pažymys patenka į leistiną intervalą.

Sąmoningai gražina `bool`, o ne meta išimtį: klaidos pranešimo formavimas (`std::string` konkatencija) yra brangus, o su 10 mln. studentų ši funkcija kviečiama ~160 mln. kartų. Pranešimas formuojamas tik ten, kur išimtis tikrai metama.

Parametrai

<i>p</i>	Tikrinamas pažymys
----------	--------------------

Gražina

`true` jei `minPazymys` `p` `maxPazymys`

Apibrėžimas failo [studentas.h](#) eilutėje 107.

Ryšiai [maxPazymys](#), ir [minPazymys](#).

Naudojamas [addPazymys\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setEgzaminas\(\)](#), [setNd\(\)](#), ir [studentas\(\)](#).

7.2.3.4 operator=() [1/2]

```
studentas & studentas::operator= (
    const studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Priskiriamas studentas
--------------	------------------------

Gražina

Nuoroda į šį objektą (grandininiam priskyrimui $a = b = c$)

Pastaba

Turi apsaugą nuo priskyrimo sau (`this == &other`) — be jos `a = a` galėtų sugadinti duomenis.

Bazinė dalis priskiriama kvalifikuotu kvietimu `zmogus::operator=(other)`. Be kvalifikacijos kompiliatorius parinktų `studentas::operator=` ir įvyktų begalinė rekursija.

Apibrėžimas failo `studentas.cpp` eilutėje 147.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `nd_`, `zmogus::operator=()`, ir `studentas()`.

7.2.3.5 operator=() [2/2]

```
studentas & studentas::operator= (
    studentas && other) [noexcept]
```

Perkėlimo (move) priskyrimo operatorius.

Parametrai

<i>other</i>	Studentas, iš kurio perkeliami duomenys
--------------	---

Gražina

Nuoroda į šį objektą

Pastaba

Bazinė dalis perkeliama `zmogus::operator=(std::move(other))`.

Apibrėžimas failo `studentas.cpp` eilutėje 190.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `nd_`, `zmogus::operator=()`, ir `studentas()`.

7.2.3.6 galutinis()

```
double studentas::galutinis () const [override], [virtual]
```

Gražina galutinį įvertinimą (bazinės klasės interfeisas).

Studentui tai galutinis balas pagal namų darbų **vidurkį**. Medianos variantas prieinamas per `galutinisMed()`.

Gražina

Galutinis balas pagal vidurkį

Įspėjimas

Gražina 0.0, jei nebuvo iškviesta `calculateGalutinis()`.

Realizuoja `zmogus`.

Apibrėžimas failo `studentas.cpp` eilutėje 217.

Ryšiai `galutinisVid_`.

7.2.3.7 print()

```
void studentas::print (
    std::ostream & os) const [override], [virtual]
```

Išveda studentą į srautą.

Formatas: vardas (20 simbolių), pavardė (20), galutinis pagal vidurkį (20), galutinis pagal medianą (20). Realizuoja polimorfinį operator<<.

Parametrai

<i>os</i>	Išvesties srautas
-----------	-------------------

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 225.

Ryšiai [galutinisMed_](#), [galutinisVid_](#), [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.3.8 read()

```
std::istream & studentas::read (
    std::istream & is) [override], [virtual]
```

Nuskaito studentą iš srauto.

Realizuoja polimorfinį operator>>. Deleguoja į [readStudentas\(\)](#).

Parametrai

<i>is</i>	Išvesties srautas
-----------	-------------------

Gražina

Nuoroda į tą patį srautą

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 237.

Ryšiai [readStudentas\(\)](#).

7.2.3.9 tipas()

```
std::string studentas::tipas () const [override], [virtual]
```

Gražina tipo pavadinimą.

Gražina

"Studentas"

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 245.

7.2.3.10 nd()

```
const std::vector< int > & studentas::nd () const [inline]
```

Gražina namų darbų pažymius.

Gražina

Konstanti nuoroda į vektorių (be kopijavimo)

Apibrėžimas failo [studentas.h](#) eilutėje 316.

Ryšiai [nd_](#).

Naudojamas [appendListTo\(\)](#), [mediana\(\)](#), [setNd\(\)](#), [studentas\(\)](#), [testStudentasKlase\(\)](#), ir [vidurkis\(\)](#).

7.2.3.11 egzaminas()

```
int studentas::egzaminas () const [inline]
```

Gražina egzamino pažymį.

Apibrėžimas failo [studentas.h](#) eilutėje 319.

Ryšiai [egzaminas_](#).

Naudojamas [comparePagalEgzamina\(\)](#), [studentas\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.12 galutinisVid()

```
double studentas::galutinisVid () const [inline]
```

Gražina galutinį balą pagal namų darbų vidurkį.

Apibrėžimas failo [studentas.h](#) eilutėje 322.

Ryšiai [galutinisVid_](#).

Naudojamas [comparePagalVidurki\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.13 galutinisMed()

```
double studentas::galutinisMed () const [inline]
```

Gražina galutinį balą pagal namų darbų medianą.

Apibrėžimas failo [studentas.h](#) eilutėje 325.

Ryšiai [galutinisMed_](#).

Naudojamas [comparePagalMediana\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.14 tuscias()

```
bool studentas::tuscias () const [inline]
```

Patikrina, ar objektas neturi vardo ir pavardės.

Gražina

```
true
```

jei ir vardas, ir pavardė tušti

Apibrėžimas failo [studentas.h](#) eilutėje 331.

Ryšiai [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.3.15 setNd()

```
void studentas::setNd (
    const std::vector< int > & nd)
```

Nustato namų darbų pažymius su validacija.

Parametrai

<i>nd</i>	Pažymių vektorius
-----------	-------------------

Išimtys

<i>std::runtime_error</i>	jei bent vienas pažymys už ribų
---------------------------	---------------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 254.

Ryšiai [nd\(\)](#), [nd_](#), ir [pazymysTinkamas\(\)](#).

Naudojamas [testStudentasKlase\(\)](#).

7.2.3.16 setEgzaminas()

```
void studentas::setEgzaminas (
    int e)
```

Nustato egzamino pažymį su validacija.

Parametrai

<i>e</i>	Egzamino pažymys
----------	------------------

Išimtys

<code>std::runtime_error</code>	jei pažymys už ribų
---------------------------------	---------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.
Ryšiai [egzaminas_](#), ir [pazymysTinkamas\(\)](#).
Naudojamas [testStudentasKlase\(\)](#).

7.2.3.17 addPazymys()

```
void studentas::addPazymys (
    int p)
```

Prideda vieną namų darbo pažymį.

Parametrai

<i>p</i>	Pridedamas pažymys
----------	--------------------

Išimtys

<code>std::runtime_error</code>	jei pažymys už ribų
---------------------------------	---------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 267.
Ryšiai [nd_](#), ir [pazymysTinkamas\(\)](#).
Naudojamas [testStudentasKlase\(\)](#).

7.2.3.18 isvalyk()

```
void studentas::isvalyk ()
```

Ištuština objektą.

Išvalo **ir bazinės, ir išvestinės** klasės laukus. Vektoriaus atmintis faktiškai atlaisvinama (`shrink_to_fit`).

Apibrėžimas failo [studentas.cpp](#) eilutėje 273.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [nd_](#), [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

Naudojamas [testStudentasKlase\(\)](#).

7.2.3.19 calculateGalutinis()

```
void studentas::calculateGalutinis ()
```

Apskaičiuoja abu galutinius balus.

Užpildo [galutinisVid_](#) ir [galutinisMed_](#) pagal formulę:

```
galutinisVid = 0.4 * vidurkis(nd) + 0.6 * egzaminas
galutinisMed = 0.4 * mediana(nd) + 0.6 * egzaminas
```

Tuščiam namų darbų sąrašui vidurkis ir mediana laikomi 0.0, todėl galutinis balas yra $0.6 * \text{egzaminas}$.

Pastaba

Kviečiamas atskirai, ne konstruktoriuje — taip nuskaitymo ir skaičiavimo spartą galima matuoti nepriklausomai.

Apibrėžimas failo [studentas.cpp](#) eilutėje 287.

Ryšiai [egzaminas_](#), [egzaminasSvoris](#), [galutinisMed_](#), [galutinisVid_](#), [mediana\(\)](#), [namuDarbaiSvoris](#), [nd_](#), ir [vidurkis\(\)](#).

Naudojamas [testStudentasKlase\(\)](#).

7.2.3.20 readStudentas()

```
std::istream & studentas::readStudentas (
    std::istream & is,
    int ndCount = 0,
    int lineNumber = 0)
```

Nuskaityto studentą iš srauto.

Veikia dviem režimais priklausomai nuo `ndCount`:

- **ndCount > 0** — nuskaityti tiek namų darbų pažymių, po jų — egzaminą. Naudojama, kai stulpelių skaičius žinomas iš failo antraštės.
- **ndCount == 0** — skaityti iki eilutės galo, paskutinį laiko egzaminu. Naudojama interaktyviai įvesčiai.

Parametrai

<i>is</i>	Įvesties srautas
<i>ndCount</i>	Namų darbų skaičius (0 = skaityti iki galo)
<i>lineNumber</i>	Eilutės numeris klaidos pranešimui (0 = nenaudoti)

Gražina

Nuoroda į tą patį srautą

Išimtys

<code>std::runtime_error</code>	jei trūksta duomenų arba pažymys už ribų
---------------------------------	--

Pastaba

Failo skaitymui naudokite [parseFromLine\(\)](#) — jis kelis kartus greitesnis, nes neturi srauto mechanizmo.

Apibrėžimas failo [studentas.cpp](#) eilutėje 355.

Ryšiai [egzaminas_](#), [nd_](#), [zmogus::pavarde_](#), [pazymysTinkamas\(\)](#), ir [zmogus::vardas_](#).

Naudojamas [read\(\)](#), ir [studentas\(\)](#).

7.2.3.21 parseFromLine()

```
void studentas::parseFromLine (
    const std::string & line,
    int ndCount,
    int lineNumber)
```

Greitas parsinimas iš eilutės (failo skaitymui).

Optimizuota alternatyva [readStudentas\(\)](#) metodui masiniam duomenų apdorojimui:

- **nekuriamas** `std::istringstream` objektas kiekvienai eilutei
- skaičiai parsinami su `std::from_chars` (C++17) — be locale, be stream sentry, be būsenos tikrinimo
- klaidos pranešimai formuojami tik metant išimtį

Su 10 mln. studentų šie trys pakeitimai sutrumpina failo nuskaitymą nuo ~90 s iki ~30 s.

Parametrai

<i>line</i>	Nuskaitoma eilutė
<i>ndCount</i>	Namų darbų stulpelių skaičius (iš failo antraštės)
<i>lineNumber</i>	Eilutės numeris klaidos pranešimui

Išimtys

<code>std::runtime_error</code>	jei trūksta vardo, pavardės ar pažymio, arba pažymys už ribų
---------------------------------	--

Taip pat žiūrėti

[appendListTo\(\)](#) — atvirkštinė operacija

Apibrėžimas failo `studentas.cpp` eilutėje 300.

Ryšiai `egzaminas_`, `nd_`, `zmogus::pavarde_`, `pazymysTinkamas()`, ir `zmogus::vardas_`.

Naudojamas `readStudentaiFromFile()`, ir `testStudentasKlase()`.

7.2.3.22 appendListTo()

```
void studentas::appendListTo (
    std::string & out) const
```

Prideda suformatuotą eilutę į buferį (failo rašymui).

Optimizuota alternatyva `operator<<` masiniam rašymui:

- skaičiai formatuojami su `std::to_chars` — be `std::setw` manipuliatorių
- rašoma į bendrą `std::string` buferį, ne į srautą

Kviečianti funkcija kaupia eilutes ir išskviečia vieną `file.write()` kas ~1 MB. Buferis nedidėja proporcingai studentų skaičiui — `clear()` išlaiko `capacity`.

Formatas: vardas (25), pavardė (25), kiekvienas ND pažymys (10), egzaminas (10), naujos eilutės simbolis.

Parametrai

<i>out</i>	Buferis, į kurį pridedama eilutė
------------	----------------------------------

Taip pat žiūrėti

[parseFromLine\(\)](#) — atvirkštinė operacija; kartu jos sudaro round-trip: išvesta eilutė nuskaitoma atgal be pakeitimų

Apibrėžimas failo `studentas.cpp` eilutėje 407.

Ryšiai `egzaminas_`, `nd()`, `nd_`, `zmogus::pavarde_`, ir `zmogus::vardas_`.

Naudojamas `testStudentasKlase()`.

7.2.3.23 operator==()

```
bool studentas::operator== (
    const studentas & other) const
```

Lygina du studentus.

Lyginami vardas, pavardė, namų darbų pažymiai ir egzaminas. Galutiniai balai **nelyginami**, nes jie išvedami iš kitų laukų.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`true` jei visi duomenys sutampa

Apibrėžimas failo [studentas.cpp](#) eilutėje 434.

Ryšiai [egzaminas_](#), [nd_](#), [zmogus::pavarde_](#), [studentas\(\)](#), ir [zmogus::vardas_](#).

7.2.3.24 operator"!="()

```
bool studentas::operator!= (
    const studentas & other) const
```

Tikrina, ar studentai skiriasi.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`!(*this == other)`

Apibrėžimas failo [studentas.cpp](#) eilutėje 441.

Ryšiai [studentas\(\)](#).

7.2.3.25 operator<()

```
bool studentas::operator< (
    const studentas & other) const
```

Rikiavimo operatorius.

Lygina pagal pavardę; esant vienodoms pavardėms — pagal vardą. Leidžia naudoti `std::sort` be komparatoriaus ir dėti studentus į `std::set` ar `std::map`.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`true` jei šis studentas eina anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 446.

Ryšiai [zmogus::pavarde_](#), [studentas\(\)](#), ir [zmogus::vardas_](#).

7.2.3.26 operator>()

```
bool studentas::operator> (
    const studentas & other) const
```

Atvirkštinis rikiavimo operatorius.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

```
other < *this
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 452.
Ryšiai [studentas\(\)](#).

7.2.3.27 operator[]()

```
int studentas::operator[] (
    size_t i) const
```

Prieiga prie namų darbo pažymio pagal indeksą.

Parametrai

<i>i</i>	Pažymio indeksas (nuo 0)
----------	--------------------------

Gražina

Pažymys

Išimtys

<code>std::out_of_range</code>	jei indeksas už ribų
--------------------------------	----------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 456.
Ryšiai [nd_](#).

7.2.3.28 operator bool()

```
studentas::operator bool () const [explicit]
```

Konvertavimas į `bool` — ar objektas turi duomenų.

Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas į `int` (pvz. `s + 1` nesikompiliuos, kaip ir turi būti).

Gražina

```
true
```

jei vardas ir pavardė nėra tušti

Pavyzdys

```
studentas s;
if (!s) std::cout << "Objektas tuscias\n";
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 462.
Ryšiai [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.4 Atributų Dokumentacija**7.2.4.1 nd_**

```
std::vector<int> studentas::nd_ [private]
```

Namų darbų pažymiai (kiekvienas 1–10).

Apibrėžimas failo [studentas.h](#) eilutėje 72.

Naudojamas [addPažymys\(\)](#), [appendListTo\(\)](#), [calculateGalutinis\(\)](#), [isvalyk\(\)](#), [nd\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [operator\[\]\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setNd\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.2 egzaminas_

```
int studentas::egzaminas_ [private]
```

Egzamino pažymys (1–10).

Apibrėžimas failo [studentas.h](#) eilutėje 73.

Naudojamas [appendListTo\(\)](#), [calculateGalutinis\(\)](#), [egzaminas\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setEgzaminas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.3 galutinisVid_

```
double studentas::galutinisVid_ [private]
```

Galutinis balas pagal namų darbų vidurkį

Apibrėžimas failo [studentas.h](#) eilutėje 74.

Naudojamas [calculateGalutinis\(\)](#), [galutinis\(\)](#), [galutinisVid\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.4 galutinisMed_

```
double studentas::galutinisMed_ [private]
```

Galutinis balas pagal namų darbų medianą

Apibrėžimas failo [studentas.h](#) eilutėje 75.

Naudojamas [calculateGalutinis\(\)](#), [galutinisMed\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.5 minPazymys

```
int studentas::minPazymys = 1 [static], [constexpr]
```

Mažiausias leistinas pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 114.

Naudojamas [enterEgzaminas\(\)](#), [enterPazymys\(\)](#), [generatePazymiai\(\)](#), ir [pazymysTinkamas\(\)](#).

7.2.4.6 maxPazymys

```
int studentas::maxPazymys = 10 [static], [constexpr]
```

Didžiausias leistinas pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 115.

Naudojamas [enterEgzaminas\(\)](#), [enterPazymys\(\)](#), [generatePazymiai\(\)](#), ir [pazymysTinkamas\(\)](#).

7.2.4.7 namuDarbaiSvoris

```
double studentas::namuDarbaiSvoris = 0.4 [static], [constexpr]
```

Namų darbų svoris galutiniame balse.

Apibrėžimas failo [studentas.h](#) eilutėje 116.

Naudojamas [calculateGalutinis\(\)](#).

7.2.4.8 egzaminasSvoris

```
double studentas::egzaminasSvoris = 0.6 [static], [constexpr]
```

Egzamino svoris galutiniame balse.

Apibrėžimas failo [studentas.h](#) eilutėje 117.

Naudojamas [calculateGalutinis\(\)](#).

7.2.4.9 gyvuStudentu

```
int studentas::gyvuStudentu = 0 [static]
```

Gyvuojančių studentų skaitiklis.

Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje. Naudojamas testuose objektų nutekėjimui aptikti — atskiras nuo [zmogus::gyvuZmoniu](#), todėl galima patikrinti, ar iškviesti abu destruktoriai ([~studentas\(\)](#) ir [~zmogus\(\)](#)).

Apibrėžimas failo [studentas.h](#) eilutėje 128.

Naudojamas [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [testStudentasKlase\(\)](#), ir [~studentas\(\)](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

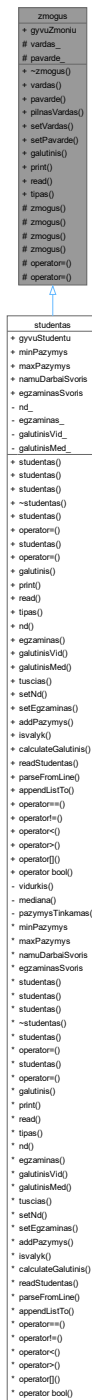
- [OOP_2/studentas.h](#)
- [OOP_2/studentas.cpp](#)

7.3 zmogus Klasė

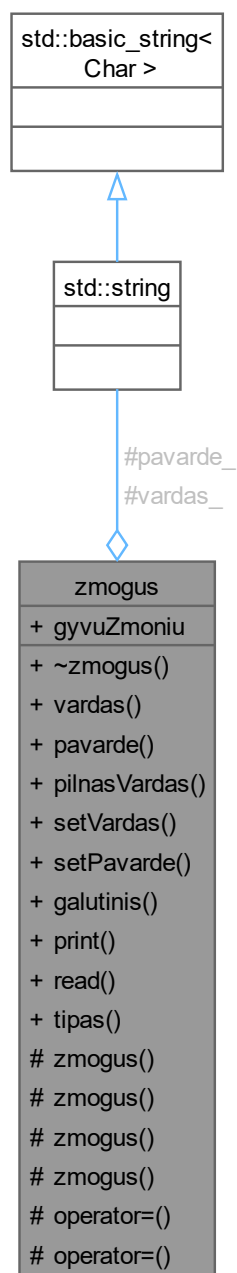
ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas.

```
#include <zmogus.h>
```

Paveldimumo diagrama žmogus:



Bendradarbiavimo diagrama zmogus:



Vieši Metodai

- virtual `~zmogus ()`
Virtualus destruktorius.
- const std::string & `vardas ()` const
Grąžina žmogaus vardą.
- const std::string & `pavarde ()` const
Grąžina žmogaus pavardę.

- `std::string pilnasVardas () const`
Grąžina pilną vardą.
- `void setVardas (const std::string &v)`
Nustato vardą su validacija.
- `void setPavarde (const std::string &p)`
Nustato pavardę su validacija.
- `virtual double galutinis () const =0`
Grąžina galutinį įvertinimą.
- `virtual void print (std::ostream &os) const =0`
Išveda objektą į srautą.
- `virtual std::istream & read (std::istream &is)=0`
Nuskaityto objektą iš srauto.
- `virtual std::string tipas () const =0`
Grąžina žmogaus tipo pavadinimą.

Statiniai Vieši Atributai

- `static int gyvuZmoniu = 0`
Gyvuojančių objektų skaitiklis.

Apsaugoti Metodai

- `zmogus ()`
Numatytasis konstruktorius.
- `zmogus (const std::string &vardas, const std::string &pavarde)`
Pilnas konstruktorius su validacija.
- `zmogus (const zmogus &other)`
Kopijavimo konstruktorius.
- `zmogus (zmogus &&other) noexcept`
Perkėlimo (move) konstruktorius.
- `zmogus & operator= (const zmogus &other)`
Kopijavimo priskyrimo operatorius.
- `zmogus & operator= (zmogus &&other) noexcept`
Perkėlimo (move) priskyrimo operatorius.

Apsaugoti Atributai

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė

7.3.1 Smulkus aprašymas

ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas.

Laiko vardą ir pavardę bei apibrėžia interfeisą, kurį privalo realizuoti kiekviena išvestinė klasė.

Kodėl klasė abstrakti

Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais**:

1. **Grynai virtualūs metodai** (= 0) — `galutinis()`, `print()`, `read()`, `tipas()`. Kompiliatorius neleidžia kurti objektų iš klasės, kuri turi bent vieną neapibrėžtą virtualų metodą.
2. **protected konstruktoriai** — prieinami tik išvestinėms klasėms. Tai antras apsaugos sluoksnis.

Naudojimo pavyzdys

```
// zmogus z; // KLAIDA: abstrakti klasė
// zmogus* p = new zmogus(); // KLAIDA: abstrakti klasė

studentas s("Jonas", "Jonaitis", {5, 6, 7}, 8); // OK
zmogus& z = s; // OK -- nuoroda
std::cout << z.tipas() << "\n"; // "Studentas"

std::unique_ptr<zmogus> p =
    std::make_unique<studentas>("Ona", "Onaite", {10}, 10); // OK
```

Įspėjimas

Destruktorius PRIVALO būti virtualus — kitaip naikinant objektą per bazinės klasės rodyklę išvestinės klasės laukai liktų neatlaisvinti.

Taip pat žiūrėti

[studentas](#)

Apibrėžimas failo [zmogus.h](#) eilutėje 50.

7.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.3.2.1 zmogus() [1/4]

```
zmogus::zmogus () [protected]
```

Numatytasis konstruktorius.

Sukuria objektą su tuščiu vardu ir pavarde. `protected` — kviečiamas tik iš išvestinių klasių.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 12.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), ir [vardas_](#).

Naudojamas [operator=\(\)](#), [operator=\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [zmogus\(\)](#), ir [zmogus\(\)](#).

7.3.2.2 zmogus() [2/4]

```
zmogus::zmogus (
    const std::string & vardas,
    const std::string & pavarde) [protected]
```

Pilnas konstruktorius su validacija.

Parametrai

<i>vardas</i>	Žmogaus vardas (negali būti tuščias)
<i>pavarde</i>	Žmogaus pavardė (negali būti tuščia)

Išimtys

<i>std::invalid_argument</i>	jei vardas arba pavardė tuščia
------------------------------	--------------------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 18.

Ryšiai [gyvuZmoniu](#), [pavarde\(\)](#), [pavarde_](#), [vardas\(\)](#), ir [vardas_](#).

7.3.2.3 zmogus() [3/4]

```
zmogus::zmogus (
    const zmogus & other) [protected]
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją — `std::string` laukai kopijuoja savo vidinius buferius.

Parametrai

<i>other</i>	Kopijuojamas objektas
--------------	-----------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 32.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

7.3.2.4 zmogus() [4/4]

```
zmogus::zmogus (
    zmogus && other) [protected], [noexcept]
```

Perkėlimo (move) konstruktorius.

Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje.

Parametrai

<i>other</i>	Objektas, iš kurio perkeliama duomenys
--------------	--

Pastaba

`noexcept` būtinai: be `std::vector` persikirstymo metu naudotų kopijavimo konstruktorių (`std::vector` naudoja `move_if_noexcept`).

Apibrėžimas failo [zmogus.cpp](#) eilutėje 42.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

7.3.2.5 ~zmogus()

```
zmogus::~zmogus () [virtual]
```

Virtualus destruktorius.

Ispėjimas

`virtual` kvalifikatorius kritiškai svarbus. Be jo:

```
zmogus* p = new studentas(...);
delete p;      // kviečiamas TIK ~zmogus()
               // studentas laukai NEATLAISVINAMI
```

Su `virtual` naikinimo tvarka teisinga: `~studentas()` → `~zmogus()`.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 82.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), ir [vardas_](#).

7.3.3 Metodų Dokumentacija**7.3.3.1 operator=() [1/2]**

```
zmogus & zmogus::operator= (
    const zmogus & other) [protected]
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Priskiriamas objektas
--------------	-----------------------

Gražina

Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)

Pastaba

Turi apsaugą nuo priskyrimo sau (`this == &other`).

Apibrėžimas failo `zmogus.cpp` eilutėje 51.

Ryšiai `pavarde_`, `vardas_`, ir `zmogus()`.

Naudojamas `studentas::operator=()`, ir `studentas::operator=()`.

7.3.3.2 operator=() [2/2]

```
zmogus & zmogus::operator= (
    zmogus && other) [protected], [noexcept]
```

Perkėlimo (move) priskyrimo operatorius.

Parametrai

<i>other</i>	Objektas, iš kurio perkeliama duomenys
--------------	--

Gražina

Nuoroda į šį objektą

Apibrėžimas failo `zmogus.cpp` eilutėje 65.

Ryšiai `pavarde_`, `vardas_`, ir `zmogus()`.

7.3.3.3 vardas()

```
const std::string & zmogus::vardas () const [inline]
```

Gražina žmogaus vardą.

Gražina

Konstanti nuoroda į vardą (be kopijavimo)

Apibrėžimas failo `zmogus.h` eilutėje 141.

Ryšiai `vardas_`.

Naudojamas `comparePagalVarda()`, `studentas::studentas()`, `testStudentasKlase()`, ir `zmogus()`.

7.3.3.4 pavarde()

```
const std::string & zmogus::pavarde () const [inline]
```

Gražina žmogaus pavardę.

Gražina

Konstanti nuoroda į pavardę (be kopijavimo)

Apibrėžimas failo `zmogus.h` eilutėje 147.

Ryšiai `pavarde_`.

Naudojamas `comparePagalPavarde()`, `studentas::studentas()`, `testStudentasKlase()`, ir `zmogus()`.

7.3.3.5 pilnasVardas()

```
std::string zmogus::pilnasVardas () const [inline]
```

Gražina pilną vardą.

Gražina

Vardas ir pavardė, atskirti tarpu

Apibrėžimas failo `zmogus.h` eilutėje 153.

Ryšiai `pavarde_`, ir `vardas_`.

Naudojamas `testStudentasKlase()`.

7.3.3.6 setVardas()

```
void zmogus::setVardas (
    const std::string & v)
```

Nustato vardą su validacija.

Parametrai

<i>v</i>	Naujas vardas
----------	---------------

Išimtys

<i>std::invalid_argument</i>	jei vardas tuščias
------------------------------	--------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 94.

Ryšiai [vardas_](#).

Naudojamas [testStudentasKlase\(\)](#).

7.3.3.7 setPavarde()

```
void zmogus::setPavarde (
    const std::string & p)
```

Nustato pavardę su validacija.

Parametrai

<i>p</i>	Nauja pavardė
----------	---------------

Išimtys

<i>std::invalid_argument</i>	jei pavardė tuščia
------------------------------	--------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 101.

Ryšiai [pavarde_](#).

Naudojamas [testStudentasKlase\(\)](#).

7.3.3.8 galutinis()

```
virtual double zmogus::galutinis () const [pure virtual]
```

Gražina galutinį įvertinimą.

Grynai virtualus — kiekvienas žmogaus tipas skaičiuoja jį savaip. Studentui tai galutinis balas pagal namų darbų vidurkį ir egzaminą.

Gražina

Galutinis įvertinimas

Realizuota [studentas](#).

Naudojamas [testStudentasKlase\(\)](#).

7.3.3.9 print()

```
virtual void zmogus::print (
    std::ostream & os) const [pure virtual]
```

Išveda objektą į srautą.

Grynai virtualus. Realizuoja polimorfinį operator<<.

Parametrai

<i>os</i>	Išvesties srautas
-----------	-------------------

Realizuota [studentas](#).

Naudojamas [operator<<\(\)](#).

7.3.3.10 read()

```
virtual std::istream & zmogus::read (
    std::istream & is) [pure virtual]
```

Nuskaityti objektą iš srauto.

Grynai virtualus. Realizuoja polimorfinį [operator>>](#).

Parametrai

<i>is</i>	Išvesties srautas
-----------	-------------------

Gražina

Nuoroda į tą patį srautą (grandininiam skaitymui)

Realizuota [studentas](#).

Naudojamas [operator>>\(\)](#).

7.3.3.11 tipas()

```
virtual std::string zmogus::tipas () const [pure virtual]
```

Gražina žmogaus tipo pavadinimą.

Grynai virtualus. Naudojamas diagnostikai ir logams.

Gražina

Tipo pavadinimas, pvz. "Studentas"

Realizuota [studentas](#).

Naudojamas [testStudentasKlase\(\)](#).

7.3.4 Atributų Dokumentacija**7.3.4.1 vardas_**

```
std::string zmogus::vardas_ [protected]
```

Žmogaus vardas.

Apibrėžimas failo [zmogus.h](#) eilutėje 52.

Naudojamas [studentas::appendListTo\(\)](#), [studentas::isvalyk\(\)](#), [studentas::operator bool\(\)](#), [studentas::operator<\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [studentas::operator==\(\)](#), [studentas::parseFromLine\(\)](#), [pilnasVardas\(\)](#), [studentas::print\(\)](#), [studentas::readStudentas\(\)](#), [setVardas\(\)](#), [studentas::tuscias\(\)](#), [vardas\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

7.3.4.2 pavarde_

```
std::string zmogus::pavarde_ [protected]
```

Žmogaus pavardė

Apibrėžimas failo [zmogus.h](#) eilutėje 53.

Naudojamas [studentas::appendListTo\(\)](#), [studentas::isvalyk\(\)](#), [studentas::operator bool\(\)](#), [studentas::operator<\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [studentas::operator==\(\)](#), [studentas::parseFromLine\(\)](#), [pavarde\(\)](#), [pilnasVardas\(\)](#), [studentas::print\(\)](#), [studentas::readStudentas\(\)](#), [setPavarde\(\)](#), [studentas::tuscias\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

7.3.4.3 gyvuZmoniu

```
int zmogus::gyvuZmoniu = 0 [static]
```

Gyvuojančių objektų skaitiklis.

Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje. Naudojamas testuose objektų nutekėjimui aptikti.

Apibrėžimas failo [zmogus.h](#) eilutėje 121.

Naudojamas [testStudentasKlase\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [OOP_2/zmogus.h](#)
- [OOP_2/zmogus.cpp](#)

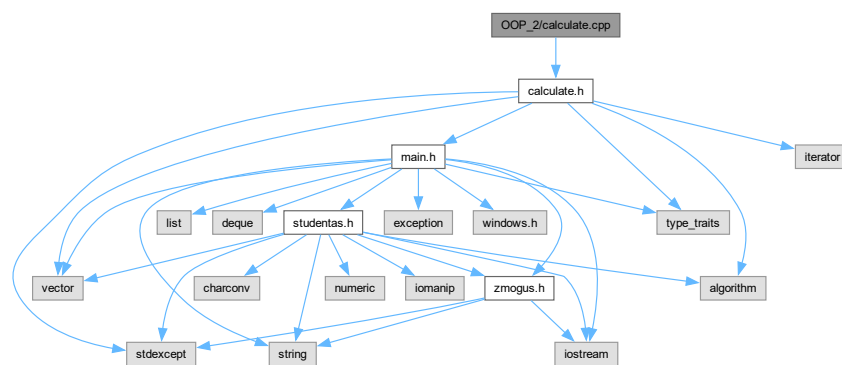
skyrius 8

Failo Dokumentacija

8.1 OOP_2/calculate.cpp Failo Nuoroda

```
#include "calculate.h"
```

Įtraukimo priklausomybių diagrama calculate.cpp:



Funkcijos

- double [mediana](#) (const std::vector< int > &namuDarbai)

8.1.1 Funkcijos Dokumentacija

8.1.1.1 mediana()

```
double mediana (
    const std::vector< int > & namuDarbai)
```

Apibrėžimas failo [calculate.cpp](#) eilutėje 3.

Ryšiai [mediana\(\)](#).

Naudojamas [mediana\(\)](#).

8.2 calculate.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "calculate.h"
00002
00003 double mediana(const std::vector<int>& namuDarbai)
00004 {
00005     double mediana{ 0.0 };
00006     if (!namuDarbai.empty())
```

```

00007     {
00008         std::vector<int> sorted = namuDarbai;
00009         std::sort(sorted.begin(), sorted.end());
00010         size_t mid{ sorted.size() / 2 };
00011         mediana = (sorted.size() % 2 == 0) ? (sorted[mid - 1] + sorted[mid]) / 2.0 : sorted[mid];
00012         return mediana;
00013     }
00014     else
00015         return 0.0;
00016 }

```

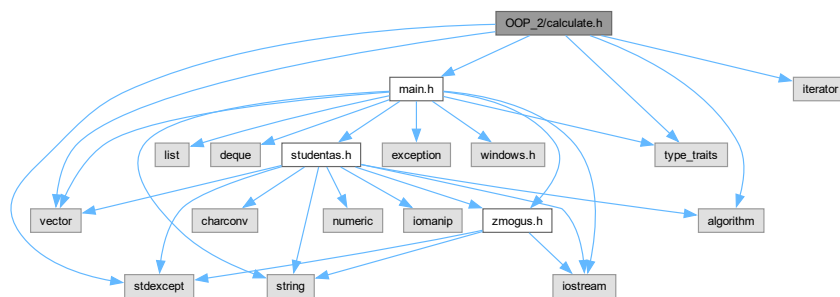
8.3 OOP_2/calculate.h Failo Nuoroda

```

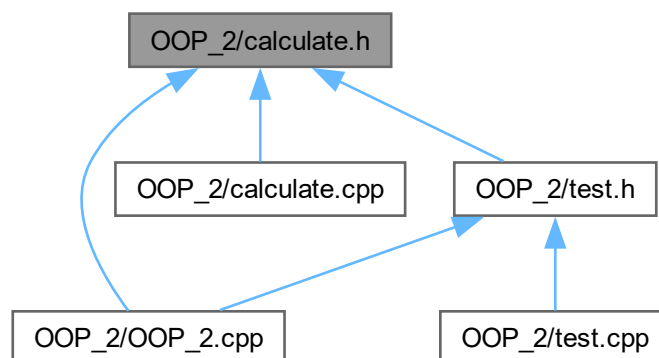
#include "main.h"
#include <vector>
#include <algorithm>
#include <iterator>
#include <type_traits>
#include <stdexcept>

```

Įtraukimo priklausomybių diagrama calculate.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
void `calculateGalutinis` (Container &studentai)

- `template<typename Container>`
void `sortStudentai` (Container &studentai, char sortBy)
- `template<typename Container>`
`splitResult`< Container > `splitStudentai` (const Container &studentai)
- `template<typename Container>`
Container `partitionStudentai` (Container &studentai)

8.3.1 Funkcijos Dokumentacija

8.3.1.1 calculateGalutinis()

```
template<typename Container>
void calculateGalutinis (
    Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 15.

Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.3.1.2 sortStudentai()

```
template<typename Container>
void sortStudentai (
    Container & studentai,
    char sortBy)
```

Apibrėžimas failo `calculate.h` eilutėje 28.

Ryšiai `comparePagalMediana()`, `comparePagalPavarde()`, `comparePagalVarda()`, ir `comparePagalVidurki()`.

Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.3.1.3 splitStudentai()

```
template<typename Container>
splitResult< Container > splitStudentai (
    const Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 54.

Ryšiai `splitResult< Container >::kietiakai`, ir `splitResult< Container >::vargsiukai`.

Naudojamas `testData()`.

8.3.1.4 partitionStudentai()

```
template<typename Container>
Container partitionStudentai (
    Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 73.

Naudojamas `runProgram()`, ir `testContainer()`.

8.4 calculate.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef CALCULATE_H_DEFINED
00002 #define CALCULATE_H_DEFINED
00003
00004 #include "main.h"
00005 #include <vector>           // vector
00006 #include <algorithm>       // sort, partition_copy, stable_partition
00007 #include <iterator>        // back_inserter, make_move_iterator
00008 #include <type_traits>     // is_same_v
00009 #include <stdexcept>       // runtime_error
00010
00011 // -----
00012 // Template: calculateGalutinis
00013 // -----
00014 template<typename Container>
00015 void calculateGalutinis(Container& studentai)
```

```

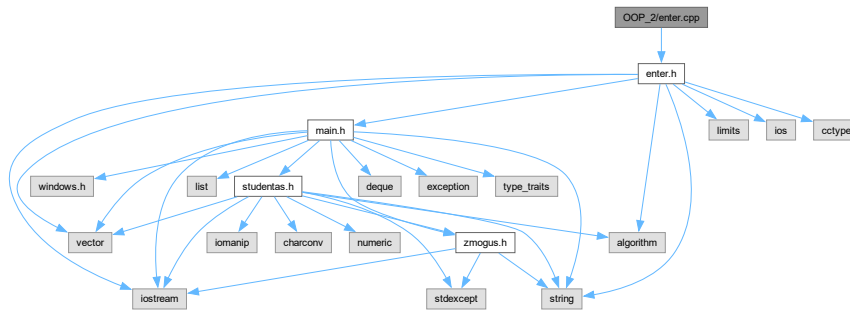
00016 {
00017     if (studentai.empty())
00018         throw std::runtime_error("Studentų sąrašas tuščias -- nėra ką skaičiuoti.");
00019
00020     for (auto& s : studentai)
00021         s.calculateGalutinis();
00022 }
00023
00024 // -----
00025 // Template: sortStudentai
00026 // -----
00027 template<typename Container>
00028 void sortStudentai(Container& studentai, char sortBy)
00029 {
00030     auto cmp = [sortBy](const studentas& a, const studentas& b) -> bool
00031     {
00032         switch (sortBy)
00033         {
00034             case '1': return comparePagalVarda(a, b);
00035             case '2': return comparePagalPavarde(a, b);
00036             case '3': return comparePagalVidurki(a, b);
00037             case '4': return comparePagalMediana(a, b);
00038             default: return false;
00039         }
00040     };
00041
00042     if constexpr (std::is_same_v<Container, std::list<studentas>>)
00043         studentai.sort(cmp); // list narys
00044     else
00045         std::sort(studentai.begin(), studentai.end(), cmp); // vector / deque
00046 }
00047
00048 // -----
00049 // Template: splitStudentai (1 strategija)
00050 // Skaido į DU naujus konteinerius, originalas nekeičiamas.
00051 // Neefektyvu atminties atžvilgiu -- duomenys dublikuojami.
00052 // -----
00053 template<typename Container>
00054 splitResult<Container> splitStudentai(const Container& studentai)
00055 {
00056     splitResult<Container> result;
00057
00058     std::partition_copy(studentai.begin(), studentai.end(),
00059         std::back_inserter(result.kietiakai),
00060         std::back_inserter(result.vargsiukai),
00061         [](const studentas& s) { return s.galutinisVid() >= 5.0; });
00062
00063     return result;
00064 }
00065
00066 // -----
00067 // Template: partitionStudentai (3 strategija -- optimaliausia)
00068 // Perskirsto originalų konteinerį vietoje, vargsiukus perkelia
00069 // į naują konteinerį ir blokiniu erase pašalina iš originalo.
00070 // Po iškvietimo studentai konteineryje lieka tik kietiakai.
00071 // -----
00072 template<typename Container>
00073 Container partitionStudentai(Container& studentai)
00074 {
00075     auto it = std::stable_partition(studentai.begin(), studentai.end(),
00076         [](const studentas& s) { return s.galutinisVid() >= 5.0; });
00077
00078     Container vargsiukai(std::make_move_iterator(it),
00079         std::make_move_iterator(studentai.end()));
00080     studentai.erase(it, studentai.end());
00081
00082     return vargsiukai;
00083 }
00084
00085 #endif // CALCULATE_H_DEFINED

```

8.5 OOP_2/enter.cpp Failo Nuoroda

```
#include "enter.h"
```

Įtraukimo priklausomybių diagrama enter.cpp:



Funkcijos

- `studentas enterStudentas (int n)`
- `std::string enterName (int n)`
- `std::string enterSurname (int n)`
- `int enterNumberOfPazymys (int n)`
- `int enterPazymys (int n)`
- `std::vector< int > enterPazymiai (int n, int m)`
- `int enterEgzaminas (int n)`
- `bool isAllLetters (const std::string &input)`

8.5.1 Funkcijos Dokumentacija

8.5.1.1 enterStudentas()

```
studentas enterStudentas (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 7.

Ryšiai `enterEgzaminas()`, `enterName()`, `enterNumberOfPazymys()`, `enterPazymiai()`, ir `enterSurname()`.

Naudojamas `enterStudentai()`.

8.5.1.2 enterName()

```
std::string enterName (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 19.

Ryšiai `isAllLetters()`.

Naudojamas `enterStudentas()`, ir `generateOnlyPazymiai()`.

8.5.1.3 enterSurname()

```
std::string enterSurname (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 34.

Ryšiai `isAllLetters()`.

Naudojamas `enterStudentas()`, ir `generateOnlyPazymiai()`.

8.5.1.4 enterNumberOfPazymys()

```
int enterNumberOfPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 50.
Naudojamas [enterStudentas\(\)](#).

8.5.1.5 enterPazymys()

```
int enterPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 66.
Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).
Naudojamas [enterPazymiai\(\)](#).

8.5.1.6 enterPazymiai()

```
std::vector< int > enterPazymiai (
    int n,
    int m)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 83.
Ryšiai [enterPazymys\(\)](#).
Naudojamas [enterStudentas\(\)](#).

8.5.1.7 enterEgzaminas()

```
int enterEgzaminas (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 94.
Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).
Naudojamas [enterStudentas\(\)](#).

8.5.1.8 isAllLetters()

```
bool isAllLetters (
    const std::string & input)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 111.
Naudojamas [enterName\(\)](#), ir [enterSurname\(\)](#).

8.6 enter.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "enter.h"
00002
00003 // -----
00004 // enterStudentas -- surenka vieno studento duomenis ir sukuria
00005 // objektą per pilną konstruktorį (jis validuoja ir apskaičiuoja).
00006 // -----
00007 studentas enterStudentas(int n)
00008 {
00009     std::cout << '#' << n << u8" studentas:\n";
00010
00011     std::string vardas = enterName(n);
00012     std::string pavarde = enterSurname(n);
00013     std::vector<int> nd = enterPazymiai(n, enterNumberOfPazymys(n));
00014     int egzaminas = enterEgzaminas(n);
00015
00016     return studentas(vardas, pavarde, nd, egzaminas);
00017 }
00018
00019 std::string enterName(int n)
00020 {
00021     std::string v;
00022
00023     std::cout << u8"Iveskite studento #" << n << u8" vardą: ";
00024     while (!(std::cin > v) || !isAllLetters(v))
00025     {
```



```

00026         std::cin.clear();
00027         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00028         std::cout << u8"Naudokite raides. Iveskite studento vardą: ";
00029     }
00030
00031     return v;
00032 }
00033
00034 std::string enterSurname(int n)
00035 {
00036     std::string v;
00037
00038     std::cout << u8"Iveskite studento #" << n << u8" pavardę: ";
00039     while (!(std::cin > v) || !isAllLetters(v))
00040     {
00041         std::cin.clear();
00042         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00043         std::cout << u8"Naudokite raides.\n";
00044         std::cout << u8"Iveskite studento #" << n << u8" pavardę: ";
00045     }
00046
00047     return v;
00048 }
00049
00050 int enterNumberOfPazymys(int n)
00051 {
00052     int tmp{};
00053
00054     std::cout << u8"Iveskite kiek #" << n << u8" studentas turėjo namų darbų: ";
00055     while (!(std::cin > tmp) || tmp < 0)
00056     {
00057         std::cin.clear();
00058         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00059         std::cout << u8"Iveskite neneigiamą skaičių.\n";
00060         std::cout << u8"Iveskite kiek #" << n << u8" studentas turėjo namų darbų: ";
00061     }
00062
00063     return tmp;
00064 }
00065
00066 int enterPazymys(int n)
00067 {
00068     int tmp{};
00069
00070     std::cout << n << u8" pažymys: ";
00071     while (!(std::cin > tmp) || tmp < studentas::minPazymys || tmp > studentas::maxPazymys)
00072     {
00073         std::cin.clear();
00074         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00075         std::cout << u8"Iveskite skaičių nuo " << studentas::minPazymys
00076             << u8" iki " << studentas::maxPazymys << u8".\n";
00077         std::cout << n << u8" pažymys: ";
00078     }
00079
00080     return tmp;
00081 }
00082
00083 std::vector<int> enterPazymiai(int n, int m) // n - studento numeris, m - pažiūjų skaičius
00084 {
00085     std::vector<int> tmp;
00086     std::cout << u8"Iveskite #" << n << u8" studento namų darbų pažymius.\n";
00087     for (int i{ 0 }; i < m; i++)
00088     {
00089         tmp.push_back(enterPazymys(i + 1));
00090     }
00091     return tmp;
00092 }
00093
00094 int enterEgzaminas(int n)
00095 {
00096     int tmp{};
00097
00098     std::cout << u8"Iveskite kiek #" << n << u8" studentas gavo iš egzamino.\n";
00099     while (!(std::cin > tmp) || tmp < studentas::minPazymys || tmp > studentas::maxPazymys)
00100     {
00101         std::cin.clear();
00102         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00103         std::cout << u8"Iveskite skaičių nuo " << studentas::minPazymys
00104             << u8" iki " << studentas::maxPazymys << u8".\n";
00105         std::cout << u8"Iveskite kiek #" << n << u8" studentas gavo iš egzamino.\n";
00106     }
00107
00108     return tmp;
00109 }
00110
00111 bool isAllLetters(const std::string& input) {
00112     return std::all_of(input.begin(), input.end(), ::isalpha);

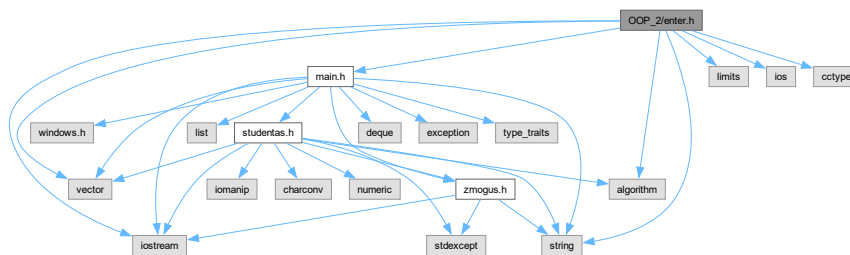
```

```
00113 }
```

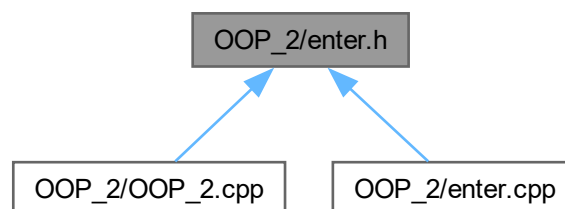
8.7 OOP_2/enter.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <limits>
#include <ios>
#include <vector>
#include <string>
#include <algorithm>
#include <cctype>
```

Įtraukimo priklausomybių diagrama enter.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
`Container enterStudentai ()`

8.7.1 Funkcijos Dokumentacija

8.7.1.1 enterStudentai()

```
template<typename Container>
```

```
Container enterStudentai ()
```

Apibrėžimas failo `enter.h` eilutėje 21.

Ryšiai `askIfMoreStudents()`, ir `enterStudentas()`.

Naudojamas `runProgram()`.

8.8 enter.h

Eiti į šio failo dokumentaciją.

```

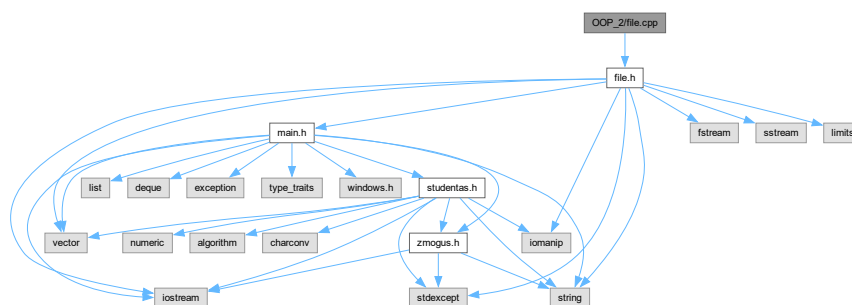
00001 #ifndef ENTER_H_DEFINED
00002 #define ENTER_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <limits>   // numeric_limits
00008 #include <ios>      // streamsize
00009 #include <vector>    // vector
00010 #include <string>    // string
00011 #include <algorithm> // all_of
00012 #include <cctype>    // isalpha
00013
00014 // -----
00015 // Template: enterStudentai<Container>
00016 // Įveda studentus ranka į bet kokį konteinerį
00017 // (vector, list, deque) naudojant push_back.
00018 // -----
00019
00020 template<typename Container>
00021 Container enterStudentai()
00022 {
00023     Container tmp;
00024     int n{ 1 };
00025     do
00026     {
00027         tmp.push_back(enterStudentas(n));
00028         n++;
00029     } while (askIfMoreStudents());
00030
00031     return tmp;
00032 }
00033
00034 #endif // ENTER_H_DEFINED

```

8.9 OOP_2/file.cpp Failo Nuoroda

```
#include "file.h"
```

Įtraukimo priklausomybių diagrama file.cpp:



Funkcijos

- std::string [enterFileName\(\)](#)
- std::string [enterOutputFileName\(\)](#) (std::string desc)

8.9.1 Funkcijos Dokumentacija

8.9.1.1 enterFileName()

```
std::string enterFileName()
```

Apibrėžimas failo [file.cpp](#) eilutėje 3.

Naudojamas [runProgram\(\)](#).

8.9.1.2 enterOutputFileName()

```
std::string enterOutputFileName (
    std::string desc)
```

Apibrėžimas failo [file.cpp](#) eilutėje 16.

Naudojamas [outputStudentai\(\)](#), ir [runProgram\(\)](#).

8.10 file.cpp

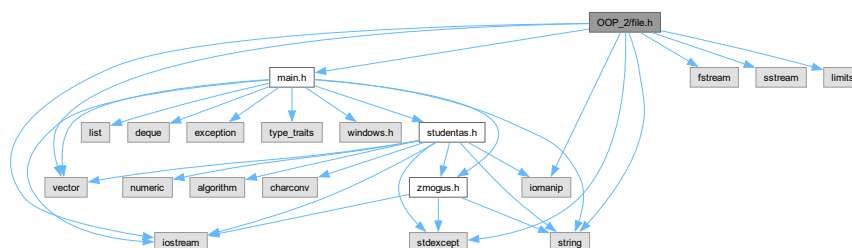
Eiti į šio failo dokumentaciją.

```
00001 #include "file.h"
00002
00003 std::string enterFileName()
00004 {
00005     std::string tmp;
00006     std::cout << u8"Iveskite failo pavadinimą (su .txt plėtiniu): ";
00007     while (!(std::cin > tmp) || tmp.size() < 5 || tmp.substr(tmp.size() - 4) != ".txt")
00008     {
00009         std::cin.clear();
00010         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00011         std::cout << u8"Netinkama įvestis. Įveskite failo pavadinimą su .txt plėtiniu: ";
00012     }
00013     return tmp;
00014 }
00015
00016 std::string enterOutputFileName(std::string desc)
00017 {
00018     std::string tmp;
00019     std::cout << u8"Iveskite " << desc << u8" failo pavadinimą (su .txt plėtiniu): ";
00020     while (!(std::cin > tmp) || tmp.size() < 5 || tmp.substr(tmp.size() - 4) != ".txt")
00021     {
00022         std::cin.clear();
00023         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00024         std::cout << u8"Netinkama įvestis. Įveskite išvesties failo pavadinimą su .txt plėtiniu: ";
00025     }
00026     return tmp;
00027 }
```

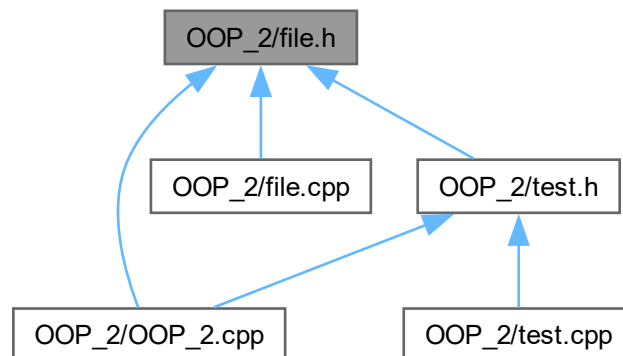
8.11 OOP_2/file.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <limits>
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama file.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
Container `readStudentaiFromFile` (const std::string &filename)
- `template<typename Container>`
void `writeStudentaiToFile` (const Container &studentai, const std::string &filename)
- `template<typename Container>`
void `writeStudentaiListToFile` (const Container &studentai, const std::string &filename)

Kintamieji

- `constexpr size_t FILE_BUFFER_SIZE = 64 * 1024`

8.11.1 Funkcijos Dokumentacija

8.11.1.1 readStudentaiFromFile()

```
template<typename Container>
Container readStudentaiFromFile (
    const std::string & filename)
```

Apibrėžimas failo `file.h` eilutėje 29.

Ryšiai `FILE_BUFFER_SIZE`, ir `studentas::parseFromLine()`.

Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.11.1.2 writeStudentaiToFile()

```
template<typename Container>
void writeStudentaiToFile (
    const Container & studentai,
    const std::string & filename)
```

Apibrėžimas failo `file.h` eilutėje 76.

Ryšiai `FILE_BUFFER_SIZE`.

Naudojamas `outputStudentai()`.

8.11.1.3 writeStudentaiListToFile()

```
template<typename Container>
void writeStudentaiListToFile (
    const Container & studentai,
    const std::string & filename)
```

Apibrėžimas failo [file.h](#) eilutėje 107.

Ryšiai [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#), [testData\(\)](#), ir [testGenerateStudentai\(\)](#).

8.11.2 Kintamojo Dokumentacija

8.11.2.1 FILE_BUFFER_SIZE

```
size_t FILE_BUFFER_SIZE = 64 * 1024 [constexpr]
```

Apibrėžimas failo [file.h](#) eilutėje 16.

Naudojamas [readStudentaiFromFile\(\)](#), ir [writeStudentaiToFile\(\)](#).

8.12 file.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef FILE_H_DEFINED
00002 #define FILE_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <fstream> // ifstream, ofstream
00008 #include <sstream> // istringstream
00009 #include <vector> // vector
00010 #include <string> // string
00011 #include <iomanip> // setprecision, setw
00012 #include <limits> // numeric_limits
00013 #include <stdexcept> // runtime_error
00014
00015 // Rašymo buferio dydis -- 64 KB.
00016 constexpr size_t FILE_BUFFER_SIZE = 64 * 1024;
00017
00018 // -----
00019 // Template: readStudentaiFromFile<Container>
00020 // Nuskaityti studentus iš .txt failo į bet kokį konteinerį.
00021 //
00022 // Optimizacijos:
00023 // - nenaudojamas std::istringstream (nekuriamas objektas eilutei)
00024 // - studentas::parseFromLine naudoja std::from_chars
00025 // - line buferis pernaudojamas visoms eilutėms
00026 // - studentas objektas kuriamas vietoje ir perkeliamas (move)
00027 // -----
00028 template<typename Container>
00029 Container readStudentaiFromFile(const std::string& filename)
00030 {
00031     Container studentai;
00032     std::ifstream file(filename);
00033
00034     if (!file.is_open())
00035         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00036
00037     // Didesnis skaitymo buferis -- mažiau OS kvietimų
00038     std::vector<char> rdbuf(FILE_BUFFER_SIZE);
00039     file.rdbuf()->pubsetbuf(rdbuf.data(), rdbuf.size());
00040
00041     std::string line;
00042     line.reserve(256); // eilutės buferis pernaudojamas -- jokių perskirstymų
00043
00044     if (!std::getline(file, line))
00045         throw std::runtime_error("Failas tuščias arba sugadintas: " + filename);
00046
00047     // Apskaičiuojame ND stulpelių skaičių iš antraštės
00048     // (antraštė nuskaityta tik vieną kartą, todėl istringstream čia pigus)
00049     std::istringstream headerStream(line);
00050     std::string token;
00051     int ndCount = 0;
00052     while (headerStream >> token)
00053         if (token.find("ND") != std::string::npos) ndCount++;
00054
00055     studentas s; // objektas pernaudojamas -- nekuriamas naujas kiekvienai eilutei
00056     int lineNumber = 1;
```

```

00057
00058     while (std::getline(file, line))
00059     {
00060         lineNumber++;
00061         if (line.empty()) continue;
00062
00063         s.parseFromLine(line, ndCount, lineNumber);
00064         studentai.push_back(std::move(s));
00065     }
00066
00067     return studentai;
00068 }
00069
00070 // -----
00071 // Template: writeStudentaiToFile
00072 // Išveda studentus su galutiniais balais.
00073 // Formatavimą atlieka klasės operator«.
00074 // -----
00075 template<typename Container>
00076 void writeStudentaiToFile(const Container& studentai, const std::string& filename)
00077 {
00078     std::ofstream file(filename);
00079     if (!file.is_open())
00080         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00081
00082     std::vector<char> wrbuf(FILE_BUFFER_SIZE);
00083     file.rdbuf()->pubsetbuf(wrbuf.data(), wrbuf.size());
00084
00085     file << u8"Vardas          Pavardė          Galutinis (Vid.)    Galutinis (Med.)\n";
00086     file << u8"-----\n";
00087
00088     for (const auto& s : studentai)
00089         file << s << "\n";
00090
00091     file.close();
00092 }
00093
00094 // -----
00095 // Template: writeStudentaiListToFile
00096 // Išveda studentus su visais namų darbų pažymiais.
00097 // Šis formatas suderinamas su readStudentaiFromFile skaitymui.
00098 //
00099 // Optimizacijos:
00100 // - eilutės formuojamos į std::string buferį (studentas::appendListTo)
00101 // - naudojant std::to_chars vietoj std::setw manipuliatorių
00102 // - buferis išvalomas kas ~1 MB -- atmintis NEauga proporcingai
00103 //   studentų skaičiui (skirtingai nei kaupiant viską į vieną string)
00104 // - vienas file.write() vietoj kelių « operacijų eilutei
00105 // -----
00106 template<typename Container>
00107 void writeStudentaiListToFile(const Container& studentai, const std::string& filename)
00108 {
00109     std::ofstream file(filename, std::ios::binary);
00110     if (!file.is_open())
00111         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00112
00113     // Antraštė
00114     std::string out;
00115     out.reserve(1024 * 1024 + 1024); // ~1 MB darbinis buferis
00116
00117     out.append("Vardas");
00118     out.append(25 - 6, ' ');
00119     out.append("Pavarde");
00120     out.append(25 - 7, ' ');
00121
00122     for (int i = 1; i <= NUMBER_OF_PAZYMYS; ++i)
00123     {
00124         std::string nd = "ND" + std::to_string(i);
00125         out.append(10 - nd.size(), ' ');
00126         out.append(nd);
00127     }
00128     out.append(10 - 9, ' ');
00129     out.append("Egzaminas");
00130     out.push_back('\n');
00131
00132     // Eilutės -- buferis išvalomas pasiekus ~1 MB ribą
00133     constexpr size_t FLUSH_LIMIT = 1024 * 1024;
00134
00135     for (const auto& s : studentai)
00136     {
00137         s.appendListTo(out);
00138
00139         if (out.size() >= FLUSH_LIMIT)
00140         {
00141             file.write(out.data(), static_cast<std::streamsize>(out.size()));
00142             out.clear(); // capacity išlieka -- jokių naujų alokacijų
00143         }
00144     }

```

```

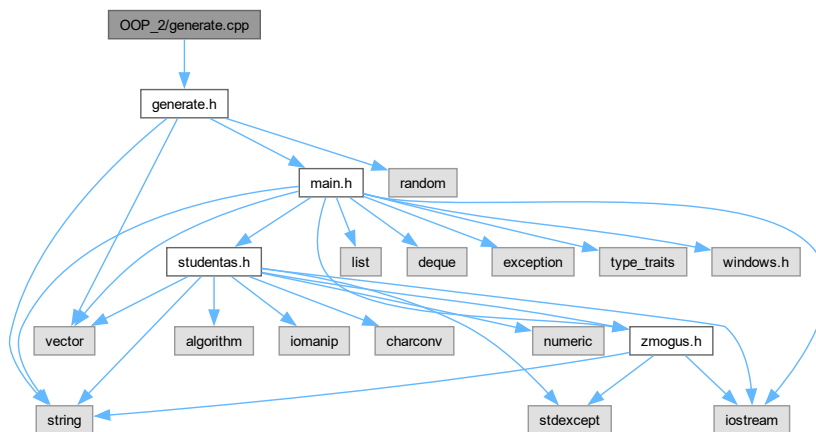
00144     }
00145
00146     // Likutis
00147     if (!out.empty())
00148         file.write(out.data(), static_cast<std::streamsize>(out.size()));
00149
00150     file.close();
00151 }
00152
00153 #endif // FILE_H_DEFINED

```

8.13 OOP_2/generate.cpp Failo Nuoroda

```
#include "generate.h"
```

Įtraukimo priklausomybių diagrama generate.cpp:



Funkcijos

- `std::mt19937 & getRng ()`
- `std::vector< int > generatePazymiai (int m)`

8.13.1 Funkcijos Dokumentacija

8.13.1.1 getRng()

```
std::mt19937 & getRng ()
```

Apibrėžimas failo [generate.cpp](#) eilutėje 3.

Naudojamas [generatePazymiai\(\)](#).

8.13.1.2 generatePazymiai()

```
std::vector< int > generatePazymiai (
    int m)
```

Apibrėžimas failo [generate.cpp](#) eilutėje 9.

Ryšiai [getRng\(\)](#), [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [generateOnlyPazymiai\(\)](#), ir [generateStudentai\(\)](#).

8.14 generate.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "generate.h"
```



```

00002
00003 std::mt19937& getRng()
00004 {
00005     static std::mt19937 rng(std::random_device{}());
00006     return rng;
00007 }
00008
00009 std::vector<int> generatePazymiai(int m) // m - pažymių skaičius
00010 {
00011     std::vector<int> tmp;
00012     tmp.reserve(m);
00013     std::uniform_int_distribution<int> dist(studentas::minPazymys, studentas::maxPazymys);
00014     std::mt19937& rng = getRng();
00015     for (int i{ 0 }; i < m; i++)
00016     {
00017         tmp.push_back(dist(rng));
00018     }
00019     return tmp;
00020 }

```

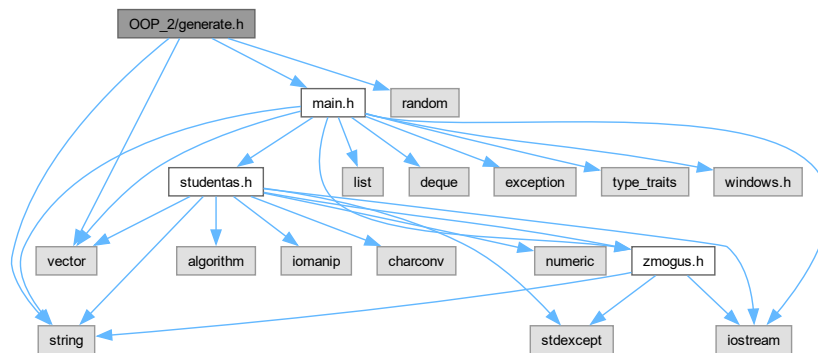
8.15 OOP_2/generate.h Failo Nuoroda

```

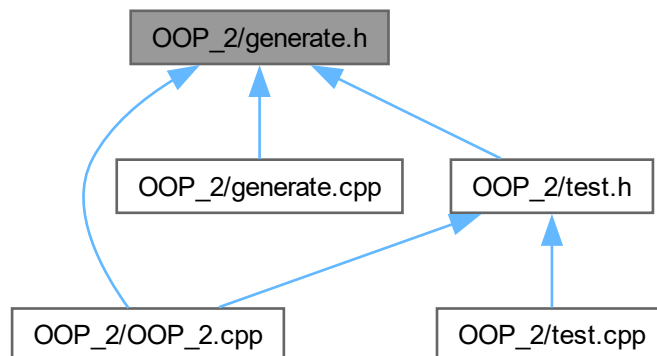
#include "main.h"
#include <vector>
#include <string>
#include <random>

```

Įtraukimo priklausomybių diagrama generate.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
Container [generateStudentai](#) (int n)
- `template<typename Container>`
Container [generateOnlyPazymiai](#) (int n)

8.15.1 Funkcijos Dokumentacija

8.15.1.1 generateStudentai()

```
template<typename Container>
Container generateStudentai (
    int n)
```

Apibrėžimas failo [generate.h](#) eilutėje 15.

Ryšiai [generatePazymiai\(\)](#), ir [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#), ir [testGenerateStudentai\(\)](#).

8.15.1.2 generateOnlyPazymiai()

```
template<typename Container>
Container generateOnlyPazymiai (
    int n)
```

Apibrėžimas failo [generate.h](#) eilutėje 34.

Ryšiai [enterName\(\)](#), [enterSurname\(\)](#), [generatePazymiai\(\)](#), ir [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#).

8.16 generate.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef GENERATE_H_DEFINED
00002 #define GENERATE_H_DEFINED
00003
00004 #include "main.h"
00005 #include <vector> // vector
00006 #include <string> // string
00007 #include <random> // mt19937, random_device
00008
```

```

00009 // -----
00010 // Template: generateStudentai
00011 // Generuoja studentus su šabloniniais vardais (VardasNR1,
00012 // PavardėNR1, ...) ir atsitiktiniais pažymiais.
00013 // -----
00014 template<typename Container>
00015 Container generateStudentai(int n)
00016 {
00017     Container studentai;
00018     for (int i = 0; i < n; i++)
00019     {
00020         studentai.push_back(studentas(
00021             "VardasNR" + std::to_string(i + 1),
00022             "PavardėNR" + std::to_string(i + 1),
00023             generatePazymiai(NUMBER_OF_PAZYMYS),
00024             generatePazymiai(1)[0]));
00025     }
00026     return studentai;
00027 }
00028
00029 // -----
00030 // Template: generateOnlyPazymiai
00031 // Vardai ir pavardės įvedami rankiniu būdu, pažymiai generuojami.
00032 // -----
00033 template<typename Container>
00034 Container generateOnlyPazymiai(int n)
00035 {
00036     Container studentai;
00037     for (int i = 0; i < n; i++)
00038     {
00039         studentai.push_back(studentas(
00040             enterName(i + 1),
00041             enterSurname(i + 1),
00042             generatePazymiai(NUMBER_OF_PAZYMYS),
00043             generatePazymiai(1)[0]));
00044     }
00045     return studentai;
00046 }
00047
00048 #endif // GENERATE_H_DEFINED

```

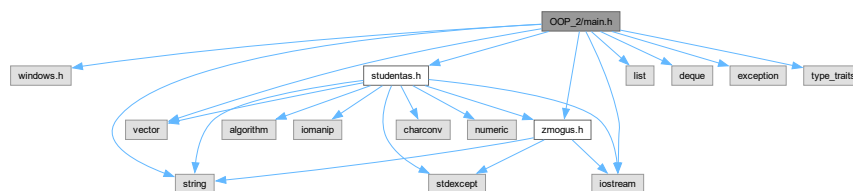
8.17 OOP_2/main.h Failo Nuoroda

```

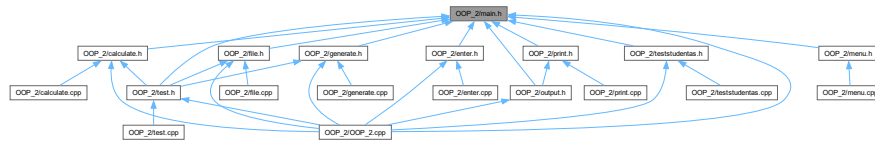
#include <windows.h>
#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <exception>
#include <type_traits>
#include "zmogus.h"
#include "studentas.h"

```

Įtraukimo priklausomybių diagrama main.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- struct [splitResult< Container >](#)

Apibrėžimai

- #define [NUMBER_OF_PAZYMYS](#) 15

Funkcijos

- char [askMenuChoice](#) ()
- int [enterNumberOfStudents](#) ()
- char [askSortBy](#) ()
- bool [askIfMoreStudents](#) ()
- char [askOutputChoice](#) ()
- char [askContainerChoice](#) ()
- std::vector< int > [generatePazymiai](#) (int m)
- void [printWelcome](#) ()
- void [printNameAsk](#) ()
- [studentas](#) [enterStudentas](#) (int n)
- std::string [enterName](#) (int n)
- std::string [enterSurname](#) (int n)
- int [enterNumberOfPazymys](#) (int n)
- int [enterPazymys](#) (int n)
- std::vector< int > [enterPazymiai](#) (int n, int m)
- int [enterEgzaminas](#) (int n)
- std::string [enterFileName](#) ()
- std::string [enterOutputFileName](#) (std::string desc="išvesties")
- void [testContainers](#) (int n)
- bool [testStudentasKlase](#) ()
- bool [isAllLetters](#) (const std::string &input)

8.17.1 Apibrėžimų Dokumentacija

8.17.1.1 NUMBER_OF_PAZYMYS

#define [NUMBER_OF_PAZYMYS](#) 15

Apibrėžimas failo [main.h](#) eilutėje 18.

Naudojamas [generateOnlyPazymiai\(\)](#), [generateStudentai\(\)](#), ir [writeStudentaiListToFile\(\)](#).

8.17.2 Funkcijos Dokumentacija

8.17.2.1 askMenuChoice()

char [askMenuChoice](#) ()

Apibrėžimas failo [menu.cpp](#) eilutėje 3.

Naudojamas [runProgram\(\)](#).

8.17.2.2 enterNumberOfStudents()

```
int enterNumberOfStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 27.

Naudojamas [runProgram\(\)](#).

8.17.2.3 askSortBy()

```
char askSortBy ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 41.

Naudojamas [runProgram\(\)](#).

8.17.2.4 askIfMoreStudents()

```
bool askIfMoreStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 73.

Naudojamas [enterStudentai\(\)](#).

8.17.2.5 askOutputChoice()

```
char askOutputChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 87.

Naudojamas [runProgram\(\)](#).

8.17.2.6 askContainerChoice()

```
char askContainerChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 103.

Naudojamas [main\(\)](#).

8.17.2.7 generatePazymiai()

```
std::vector< int > generatePazymiai (  
    int m)
```

Apibrėžimas failo [generate.cpp](#) eilutėje 9.

Ryšiai [getRng\(\)](#), [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [generateOnlyPazymiai\(\)](#), ir [generateStudentai\(\)](#).

8.17.2.8 printWelcome()

```
void printWelcome ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 3.

Naudojamas [main\(\)](#).

8.17.2.9 printNameAsk()

```
void printNameAsk ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 10.

8.17.2.10 enterStudentas()

```
studentas enterStudentas (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 7.

Ryšiai [enterEgzaminas\(\)](#), [enterName\(\)](#), [enterNumberOfPazymys\(\)](#), [enterPazymiai\(\)](#), ir [enterSurname\(\)](#).

Naudojamas [enterStudentai\(\)](#).

8.17.2.11 enterName()

```
std::string enterName (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 19.

Ryšiai [isAllLetters\(\)](#).

Naudojamas [enterStudentas\(\)](#), ir [generateOnlyPazymiai\(\)](#).

8.17.2.12 enterSurname()

```
std::string enterSurname (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 34.

Ryšiai [isAllLetters\(\)](#).

Naudojamas [enterStudentas\(\)](#), ir [generateOnlyPazymiai\(\)](#).

8.17.2.13 enterNumberOfPazymys()

```
int enterNumberOfPazymys (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 50.

Naudojamas [enterStudentas\(\)](#).

8.17.2.14 enterPazymys()

```
int enterPazymys (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 66.

Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [enterPazymiai\(\)](#).

8.17.2.15 enterPazymiai()

```
std::vector< int > enterPazymiai (  
    int n,  
    int m)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 83.

Ryšiai [enterPazymys\(\)](#).

Naudojamas [enterStudentas\(\)](#).

8.17.2.16 enterEgzaminas()

```
int enterEgzaminas (  
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 94.

Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [enterStudentas\(\)](#).

8.17.2.17 enterFileName()

```
std::string enterFileName ()
```

Apibrėžimas failo [file.cpp](#) eilutėje 3.

Naudojamas [runProgram\(\)](#).

8.17.2.18 enterOutputFileName()

```
std::string enterOutputFileName (  
    std::string desc = "išvesties")
```

Apibrėžimas failo [file.cpp](#) eilutėje 16.

Naudojamas [outputStudentai\(\)](#), ir [runProgram\(\)](#).

8.17.2.19 testContainers()

```
void testContainers (
    int n)
```

Apibrėžimas failo [test.cpp](#) eilutėje 50.

Ryšiai [testContainer\(\)](#).

Naudojamas [runProgram\(\)](#).

8.17.2.20 testStudentasKlase()

```
bool testStudentasKlase ()
```

Apibrėžimas failo [teststudentas.cpp](#) eilutėje 45.

Ryšiai [studentas::addPazymys\(\)](#), [studentas::appendListTo\(\)](#), [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [zmogus::galutinis\(\)](#), [studentas::galutinisMed\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::gyvuStudentu](#), [zmogus::gyvuZmoniu](#), [studentas::isvalyk\(\)](#), [studentas::nd\(\)](#), [studentas::parseFromLine\(\)](#), [zmogus::pavarde\(\)](#), [zmogus::pilnasVardas\(\)](#), [studentas::setEgzaminas\(\)](#), [studentas::setNd\(\)](#), [zmogus::setPavarde\(\)](#), [zmogus::setVardas\(\)](#), [zmogus::tipas\(\)](#), ir [zmogus::vardas\(\)](#).

Naudojamas [runProgram\(\)](#).

8.17.2.21 isAllLetters()

```
bool isAllLetters (
    const std::string & input)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 111.

Naudojamas [enterName\(\)](#), ir [enterSurname\(\)](#).

8.18 main.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef _WIN32
00002 #include <windows.h>
00003 #endif
00004
00005 #ifndef MAIN_H_DEFINED
00006 #define MAIN_H_DEFINED
00007
00008 #include <iostream> // cout, cin
00009 #include <string> // string
00010 #include <vector> // vector
00011 #include <list> // list
00012 #include <deque> // deque
00013 #include <exception> // exception
00014 #include <type_traits> // is_abstract, is_base_of, is_nothrow_move_constructible
00015 #include "zmogus.h" // abstrakti bazinė klasė
00016 #include "studentas.h" // išvestinė klasė
00017
00018 #define NUMBER_OF_PAZYMYS 15
00019
00020 // -----
00021 // splitResult -- 1 strategijos rezultatas (du nauji konteineriai)
00022 // -----
00023 template<typename Container>
00024 struct splitResult
00025 {
00026     Container kietiakai;
00027     Container vargsiukai;
00028 };
00029
00030 // Function declarations
00031
00032 // Menu functions
00033 char askMenuChoice();
00034 int enterNumberOfStudents();
00035 char askSortBy();
00036 bool askIfMoreStudents();
00037 char askOutputChoice();
00038 char askContainerChoice();
00039
00040 // Generate functions
00041 std::vector<int> generatePazymiai(int m);
00042
00043 // Print functions
00044 void printWelcome();
```

```

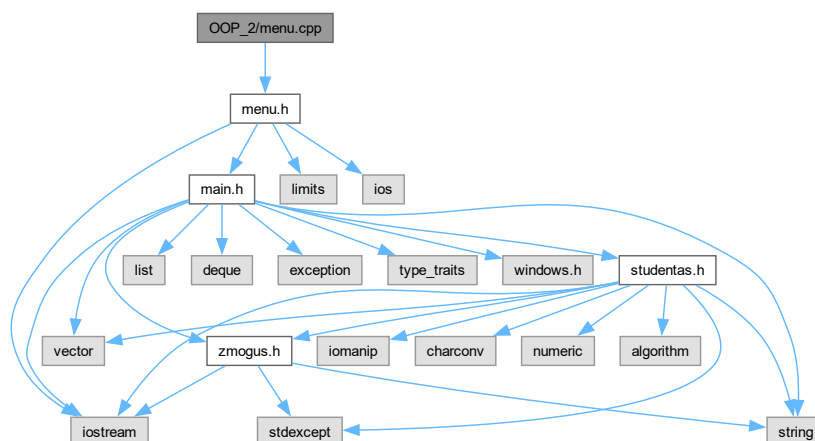
00045 void printNameAsk();
00046
00047 // User input functions
00048 studentas enterStudentas(int n);
00049 std::string enterName(int n);
00050 std::string enterSurname(int n);
00051 int enterNumberOfPazymys(int n);
00052 int enterPazymys(int n);
00053 std::vector<int> enterPazymiai(int n, int m);
00054 int enterEgzaminas(int n);
00055
00056 // File handling functions
00057 std::string enterFileName();
00058 std::string enterOutputFileName(std::string desc = "išvesties");
00059
00060 // Test functions
00061 void testContainers(int n);
00062 bool testStudentasKlase(); // klasių metodų testas (v1.2 / v1.5)
00063
00064 // Utility functions
00065 bool isAllLetters(const std::string& input);
00066
00067 #endif

```

8.19 OOP_2/menu.cpp Failo Nuoroda

```
#include "menu.h"
```

Įtraukimo priklausomybių diagrama menu.cpp:



Funkcijos

- char `askMenuChoice()`
- int `enterNumberOfStudents()`
- char `askSortBy()`
- char `askAverageOrMedian()`
- bool `askIfMoreStudents()`
- char `askOutputChoice()`
- char `askContainerChoice()`

8.19.1 Funkcijos Dokumentacija

8.19.1.1 askMenuChoice()

```
char askMenuChoice ()
```

Apibrėžimas failo `menu.cpp` eilutėje 3.

Naudojamas `runProgram()`.

8.19.1.2 enterNumberOfStudents()

```
int enterNumberOfStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 27.

Naudojamas [runProgram\(\)](#).

8.19.1.3 askSortBy()

```
char askSortBy ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 41.

Naudojamas [runProgram\(\)](#).

8.19.1.4 askAverageOrMedian()

```
char askAverageOrMedian ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 59.

8.19.1.5 askIfMoreStudents()

```
bool askIfMoreStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 73.

Naudojamas [enterStudentai\(\)](#).

8.19.1.6 askOutputChoice()

```
char askOutputChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 87.

Naudojamas [runProgram\(\)](#).

8.19.1.7 askContainerChoice()

```
char askContainerChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 103.

Naudojamas [main\(\)](#).

8.20 menu.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "menu.h"
00002
00003 char askMenuChoice()
00004 {
00005     char choice;
00006     std::cout << u8"Pasirinkite veiksmą:\n";
00007     std::cout << u8"1. Įvesti studentus ranka\n";
00008     std::cout << u8"2. Generuoti tik pažymius\n";
00009     std::cout << u8"3. Generuoti studentų vardus ir pažymius\n";
00010     std::cout << u8"4. Nuskaityti studentus iš failo\n";
00011     std::cout << u8"5. Generuoti studentų failą\n";
00012     std::cout << u8"6. Studentų failų generavimo testavimas\n";
00013     std::cout << u8"7. Duomenų apdorojimo testavimas\n";
00014     std::cout << u8"8. Konteinerių palyginimo testavimas\n";
00015     std::cout << u8"9. Klasės studentas metodų testavimas\n";
00016     std::cout << u8"0. Baigti darbą\n";
00017     std::cout << u8"Jūsų pasirinkimas: ";
00018     do
00019     {
00020         std::cin >> choice;
00021         if (choice < '0' || choice > '9')
00022             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00023     } while (choice < '0' || choice > '9');
00024     return choice;
00025 }
00026
00027 int enterNumberOfStudents()
00028 {
00029     int n;
00030     std::cout << u8"Įveskite studentų skaičių: ";
00031     while (!(std::cin >> n) || n <= 0)
```

```

00032     {
00033         std::cin.clear();
00034         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00035         std::cout << u8"Iveskite teigiamą skaičių.\n";
00036         std::cout << u8"Iveskite studentų skaičių: ";
00037     }
00038     return n;
00039 }
00040
00041 char askSortBy()
00042 {
00043     char choice;
00044     std::cout << u8"Rikiavimas pagal:\n"
00045         << u8"1. Varda\n"
00046         << u8"2. Pavardę\n"
00047         << u8"3. Galutinį įvertinimą pagal vidurkį\n"
00048         << u8"4. Galutinį įvertinimą pagal medianą\n"
00049         << u8"Jūsų pasirinkimas: ";
00050     do
00051     {
00052         std::cin >> choice;
00053         if (choice < '1' || choice > '4')
00054             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00055     } while (choice < '1' || choice > '4');
00056     return choice;
00057 }
00058
00059 char askAverageOrMedian()
00060 {
00061     char choice;
00062     std::cout << u8"Ar norite naudoti vidurkį (v) ar medianą (m) galutiniam įvertinimui apskaičiuoti?
00063     (v/m): ";
00064     do
00065     {
00066         std::cin >> choice;
00067         choice = static_cast<char>(std::tolower(static_cast<unsigned char>(choice)));
00068         if (choice != 'v' && choice != 'm')
00069             std::cout << u8"Netinkama įvestis, bandykite 'v' arba 'm': ";
00070     } while (choice != 'v' && choice != 'm');
00071     return choice;
00072 }
00073 bool askIfMoreStudents()
00074 {
00075     char choice;
00076     std::cout << u8"Ar norite įvesti dar vieną studentą? (y/n): ";
00077     do
00078     {
00079         std::cin >> choice;
00080         choice = static_cast<char>(std::tolower(static_cast<unsigned char>(choice)));
00081         if (choice != 'y' && choice != 'n')
00082             std::cout << u8"Netinkama įvestis, bandykite 'y' arba 'n': ";
00083     } while (choice != 'y' && choice != 'n');
00084     return choice == 'y';
00085 }
00086
00087 char askOutputChoice()
00088 {
00089     char choice;
00090     std::cout << u8"Pasirinkite išvesties formatą:\n"
00091         << u8"1. Išvesti į konsolę\n"
00092         << u8"2. Išvesti į failą\n"
00093         << u8"Jūsų pasirinkimas: ";
00094     do
00095     {
00096         std::cin >> choice;
00097         if (choice < '1' || choice > '2')
00098             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00099     } while (choice < '1' || choice > '2');
00100     return choice;
00101 }
00102
00103 char askContainerChoice()
00104 {
00105     char choice;
00106     std::cout << u8"Pasirinkite konteinerį:\n"
00107         << u8"1. vector\n"
00108         << u8"2. list\n"
00109         << u8"3. deque\n"
00110         << u8"Jūsų pasirinkimas: ";
00111     do
00112     {
00113         std::cin >> choice;
00114         if (choice < '1' || choice > '3')
00115             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00116     } while (choice < '1' || choice > '3');
00117     return choice;

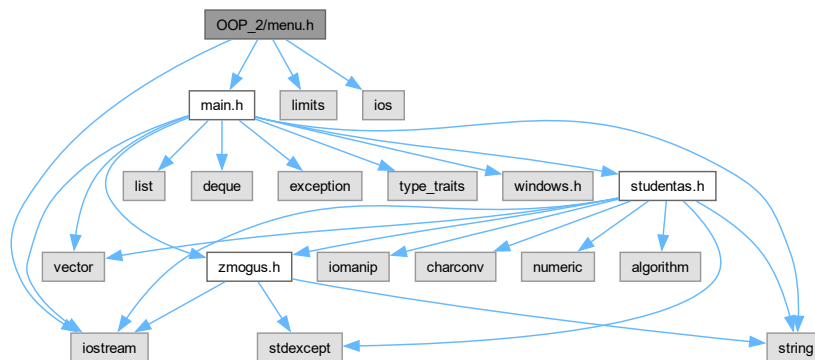
```

```
00118 }
```

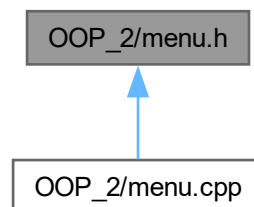
8.21 OOP_2/menu.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <limits>
#include <ios>
```

Įtraukimo priklausomybių diagrama menu.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



8.22 menu.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef MENU_H_DEFINED
00002 #define MENU_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <limits>   // numeric_limits
00008 #include <ios>      // streamsize
00009
00010 #endif // MENU_H_DEFINED
```



```

00011 // runProgram<Container> -- pagrindinis programos ciklas.
00012 // Vienas kodo kelias veikia su vector, list ir deque.
00013 // -----
00014 template<typename Container>
00015 void runProgram()
00016 {
00017     char choice{};
00018     char sortChoice{};
00019     char outputChoice{};
00020     Container studentai{}, vargsiukai{};
00021
00022     while (true)
00023     {
00024         choice = askMenuChoice();
00025         switch (choice)
00026         {
00027             case '0':
00028                 std::cout << u8"Programa baigta.\n";
00029                 return;
00030             case '1':
00031                 studentai = enterStudentai<Container>();
00032                 calculateGalutinis(studentai);
00033                 sortChoice = askSortBy();
00034                 sortStudentai<Container>(studentai, sortChoice);
00035                 vargsiukai = partitionStudentai(studentai);
00036                 outputChoice = askOutputChoice();
00037                 outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00038                 outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00039                 break;
00040             case '2':
00041                 studentai = generateOnlyPazymiai<Container>(enterNumberOfStudents());
00042                 calculateGalutinis(studentai);
00043                 sortChoice = askSortBy();
00044                 sortStudentai<Container>(studentai, sortChoice);
00045                 vargsiukai = partitionStudentai(studentai);
00046                 outputChoice = askOutputChoice();
00047                 outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00048                 outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00049                 break;
00050             case '3':
00051                 studentai = generateStudentai<Container>(enterNumberOfStudents());
00052                 calculateGalutinis(studentai);
00053                 sortChoice = askSortBy();
00054                 sortStudentai<Container>(studentai, sortChoice);
00055                 vargsiukai = partitionStudentai(studentai);
00056                 outputChoice = askOutputChoice();
00057                 outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00058                 outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00059                 break;
00060             case '4':
00061                 studentai = readStudentaiFromFile<Container>(enterFileName());
00062                 calculateGalutinis(studentai);
00063                 sortChoice = askSortBy();
00064                 sortStudentai<Container>(studentai, sortChoice);
00065                 vargsiukai = partitionStudentai(studentai);
00066                 outputChoice = askOutputChoice();
00067                 outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00068                 outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00069                 break;
00070             case '5':
00071                 studentai = generateStudentai<Container>(enterNumberOfStudents());
00072                 writeStudentaiListToFile<Container>(studentai, enterOutputFileName());
00073                 break;
00074             case '6':
00075                 testGenerateStudentai<Container>(1000);
00076                 testGenerateStudentai<Container>(10000);
00077                 testGenerateStudentai<Container>(100000);
00078                 testGenerateStudentai<Container>(1000000);
00079                 testGenerateStudentai<Container>(10000000);
00080                 break;
00081             case '7':
00082                 testData<Container>(100000);
00083                 testData<Container>(1000000);
00084                 break;
00085             case '8':
00086                 testContainers(1000);
00087                 testContainers(10000);
00088                 testContainers(100000);
00089                 testContainers(1000000);
00090                 testContainers(10000000);
00091                 break;
00092             case '9':
00093                 // Klasės studentas metodų testas (v1.2)
00094                 if (testStudentasKlase())
00095                     std::cout << u8"Visi klases testai praejo sekmingai.\n\n";
00096                 else
00097                     std::cout << u8"DEMESIO: kai kurie klases testai nepraejo!\n\n";

```

```

00098         break;
00099     default:
00100         std::cout << u8"Neteisingas pasirinkimas. Bandykite dar kartą.\n";
00101         break;
00102     }
00103 }
00104 }
00105
00106 int main()
00107 {
00108     #ifdef _WIN32
00109         SetConsoleOutputCP(CP_UTF8);
00110         SetConsoleCP(CP_UTF8);
00111     #endif
00112
00113     printWelcome();
00114
00115     try
00116     {
00117         switch (askContainerChoice())
00118         {
00119             case '1':
00120                 runProgram<std::vector<studentas>>>();
00121                 break;
00122             case '2':
00123                 runProgram<std::list<studentas>>>();
00124                 break;
00125             case '3':
00126                 runProgram<std::deque<studentas>>>();
00127                 break;
00128             default:
00129                 std::cout << u8"Neteisingas pasirinkimas. Programa baigta.\n";
00130                 return 1;
00131         }
00132     }
00133     catch (const std::exception& e)
00134     {
00135         std::cerr << "Klaida: " << e.what() << std::endl;
00136         return 1;
00137     }
00138     return 0;
00139 }

```

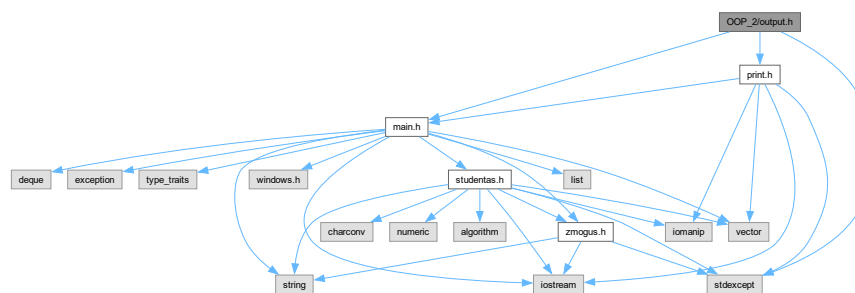
8.25 OOP_2/output.h Failo Nuoroda

```

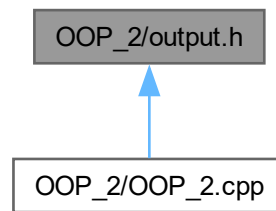
#include "main.h"
#include "print.h"
#include <stdexcept>

```

Įtraukimo priklausomybių diagrama output.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
`void outputStudentai (const Container &studentai, char choice, std::string desc="išvesties")`

8.25.1 Funkcijos Dokumentacija

8.25.1.1 outputStudentai()

```

template<typename Container>
void outputStudentai (
    const Container & studentai,
    char choice,
    std::string desc = "išvesties")
  
```

Apibrėžimas failo `output.h` eilutėje 14.

Ryšiai `enterOutputFileName()`, `printStudentai()`, ir `writeStudentaiToFile()`.

Naudojamas `runProgram()`.

8.26 output.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef OUTPUT_H_DEFINED
00002 #define OUTPUT_H_DEFINED
00003
00004 #include "main.h"
00005 #include "print.h"
00006
00007 #include <stdexcept> // runtime_error
00008
00009 // -----
00010 // Template: outputStudentai
00011 // Išveda studentus į konsolę arba į failą, priklausomai nuo vartotojo pasirinkimo.
00012 // -----
00013 template<typename Container>
00014 void outputStudentai(const Container& studentai, char choice, std::string desc = "išvesties")
00015 {
00016     switch (choice)
00017     {
00018     case '1':
00019         printStudentai<Container>(studentai);
00020         break;
00021     case '2':
00022     {
00023         std::string outputFileName = enterOutputFileName(desc);
00024         writeStudentaiToFile<Container>(studentai, outputFileName);
00025     }
00026     break;
00027     default:
00028         throw std::runtime_error("Neteisingas pasirinkimas išvesties formatui.");
00029     }
  
```

```

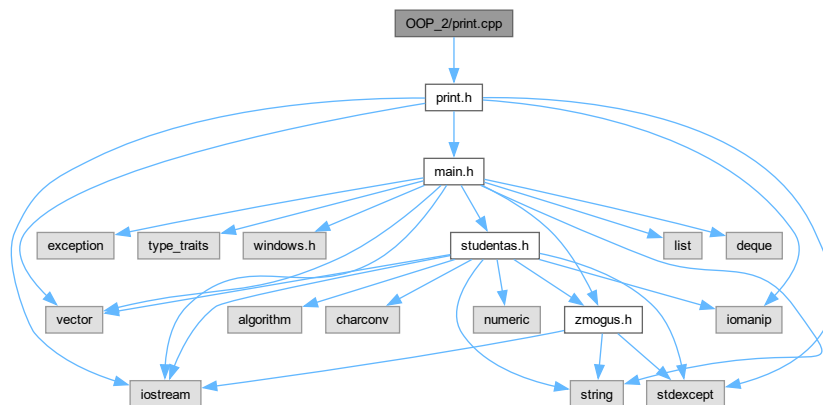
00030 }
00031
00032 #endif // OUTPUT_H_DEFINED

```

8.27 OOP_2/print.cpp Failo Nuoroda

```
#include "print.h"
```

Įtraukimo priklausomybių diagrama print.cpp:



Funkcijos

- void [printWelcome](#) ()
- void [printNameAsk](#) ()

8.27.1 Funkcijos Dokumentacija

8.27.1.1 printWelcome()

```
void printWelcome ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 3.

Naudojamas [main\(\)](#).

8.27.1.2 printNameAsk()

```
void printNameAsk ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 10.

8.28 print.cpp

Eiti į šio failo dokumentaciją.

```

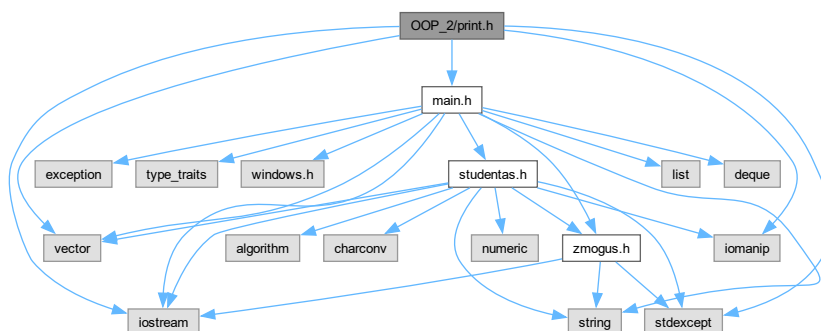
00001 #include "print.h"
00002
00003 void printWelcome ()
00004 {
00005     std::cout << u8"\n-----\n";
00006     std::cout << u8" Studentų informacinė sistema \n";
00007     std::cout << u8"-----\n\n";
00008 }
00009
00010 void printNameAsk ()
00011 {
00012     std::cout << u8"Įveskite naujus studentus.\n";
00013 }

```

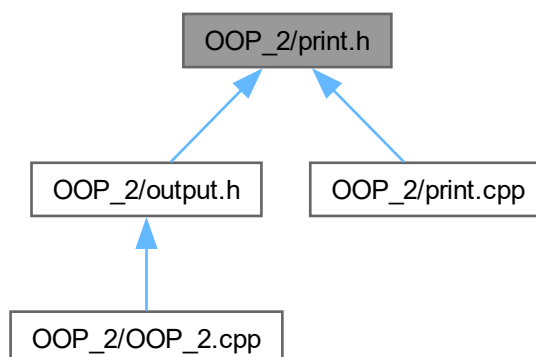

8.29 OOP_2/print.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama print.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
void `printStudentai` (const Container &studentai)

8.29.1 Funkcijos Dokumentacija

8.29.1.1 printStudentai()

```
template<typename Container>
void printStudentai (
    const Container & studentai)
```

Apibrėžimas failo `print.h` eilutėje 14.
Naudojamas `outputStudentai()`.

8.30 print.h

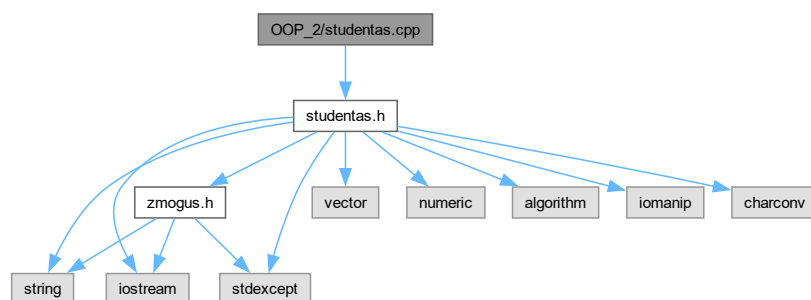
Eiti į šio failo dokumentaciją.

```
00001 #ifndef PRINT_H_DEFINED
00002 #define PRINT_H_DEFINED
00003
00004 #include "main.h"
00005 #include <iostream> // cout, cin
00006 #include <iomanip> // setprecision, setw
00007 #include <vector> // vector
00008 #include <stdexcept> // runtime_error
00009
00010 // -----
00011 // printStudentai -- išveda studentus į konsolę su antrašte.
00012 // -----
00013 template<typename Container>
00014 void printStudentai(const Container& studentai)
00015 {
00016     if (studentai.empty())
00017         throw std::runtime_error("Studentų sąrašas tuščias -- nėra ko spausdinti.");
00018
00019     std::cout << std::left << std::setw(20) << u8"Vardas"
00020               << std::left << std::setw(20) << u8"Pavardė"
00021               << std::left << std::setw(20) << u8"Galutinis (Vid.)"
00022               << std::left << std::setw(20) << u8"Galutinis (Med.)" << "\n";
00023     std::cout << u8"-----\n";
00024
00025     for (const auto& s : studentai)
00026         std::cout << s << "\n";
00027 }
00028
00029 #endif // PRINT_H_DEFINED
```

8.31 OOP_2/studentas.cpp Failo Nuoroda

```
#include "studentas.h"
```

Įtraukimo priklausomybių diagrama `studentas.cpp`:



Funkcijos

- bool `comparePagalVarda` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal vardą (didėjančiai).
- bool `comparePagalPavarde` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal pavardę (didėjančiai).
- bool `comparePagalEgzamina` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal egzamino pažymį (mažėjančiai).
- bool `comparePagalVidurki` (const `studentas` &s1, const `studentas` &s2)

Lygina pagal galutinį balą su vidurkiu (mažėjančiai).

- bool `comparePagalMediana` (const `studentas` &`s1`, const `studentas` &`s2`)

Lygina pagal galutinį balą su mediana (mažėjančiai).

8.31.1 Funkcijos Dokumentacija

8.31.1.1 `comparePagalVarda()`

```
bool comparePagalVarda (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal vardą (didėjančiai).

Parametrai

<code>s1</code>	Pirmas studentas
<code>s2</code>	Antras studentas

Gražina

`true` jei `s1` vardas abėcėliškai anksčiau

Apibrėžimas failo `studentas.cpp` eilutėje 470.

Ryšiai `zmogus::vardas()`.

Naudojamas `sortStudentai()`.

8.31.1.2 `comparePagalPavarde()`

```
bool comparePagalPavarde (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal pavardę (didėjančiai).

Parametrai

<code>s1</code>	Pirmas studentas
<code>s2</code>	Antras studentas

Gražina

`true` jei `s1` pavardė abėcėliškai anksčiau

Apibrėžimas failo `studentas.cpp` eilutėje 474.

Ryšiai `zmogus::pavarde()`.

Naudojamas `sortStudentai()`.

8.31.1.3 `comparePagalEgzamina()`

```
bool comparePagalEgzamina (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal egzamino pažymį (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true jei s1 egzaminas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 478.

Ryšiai [studentas::egzaminas\(\)](#).

8.31.1.4 comparePagalVidurki()

```
bool comparePagalVidurki (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal galutinį balą su vidurkiu (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true jei s1 galutinis balas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 482.

Ryšiai [studentas::galutinisVid\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.31.1.5 comparePagalMediana()

```
bool comparePagalMediana (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal galutinį balą su mediana (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true jei s1 galutinis balas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 486.

Ryšiai [studentas::galutinisMed\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.32 studentas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "studentas.h"
```

```

00002
00003 // -----
00004 // Statinis narys
00005 // -----
00006 int studentas::gyvuStudentu = 0;
00007
00008 // -----
00009 // Privatūs pagalbiniai metodai
00010 // -----
00011
00012 double studentas::vidurkis(const std::vector<int>& nd) {
00013     if (nd.empty())
00014         return 0.0;
00015     double sum = std::accumulate(nd.begin(), nd.end(), 0.0);
00016     return sum / nd.size();
00017 }
00018
00019 double studentas::mediana(const std::vector<int>& nd) {
00020     if (nd.empty())
00021         return 0.0;
00022     std::vector<int> sortedNd = nd;
00023     std::sort(sortedNd.begin(), sortedNd.end());
00024     size_t size = sortedNd.size();
00025     if (size % 2 == 0)
00026         return (sortedNd[size / 2 - 1] + sortedNd[size / 2]) / 2.0;
00027     else
00028         return sortedNd[size / 2];
00029 }
00030
00031 // -----
00032 // Vidinės pagalbinės funkcijos (tik šiam failui)
00033 // -----
00034
00035 namespace {
00036
00037     inline std::string vietosTekstas(int lineNumber) {
00038         return (lineNumber > 0)
00039             ? "Eilutėje " + std::to_string(lineNumber)
00040             : std::string("Įvestyje");
00041     }
00042
00043     inline std::string ribuKlaida(int lineNumber, const char* ko, int p) {
00044         return vietosTekstas(lineNumber) + " " + ko + " pažymys už ribų ("
00045             + std::to_string(studentas::minPazymys) + "-"
00046             + std::to_string(studentas::maxPazymys) + "): "
00047             + std::to_string(p);
00048     }
00049
00050 } // anoniminis namespace
00051
00052 // =====
00053 // 1. KONSTRUKTORIAI
00054 //
00055 // Kiekvienas kviečia atitinkamą BAZINĖS klasės konstruktorių
00056 // per member initializer list -- zmogus() arba zmogus(v, p).
00057 // Kūrimo tvarka: zmogus konstruktorius → studentas konstruktorius
00058 // =====
00059
00060 studentas::studentas()
00061     : zmogus(), // bazinės klasės konstruktorius
00062     nd_(), egzaminas_(0),
00063     galutinisVid_(0.0), galutinisMed_(0.0)
00064 {
00065     ++gyvuStudentu;
00066 }
00067
00068 studentas::studentas(const std::string& vardas,
00069     const std::string& pavarde,
00070     const std::vector<int>& nd,
00071     int egzaminas)
00072     : zmogus(vardas, pavarde), // bazinė validuoja vardą ir pavardę
00073     nd_(nd), egzaminas_(egzaminas),
00074     galutinisVid_(0.0), galutinisMed_(0.0)
00075 {
00076     // Studentui specifinė validacija
00077     for (int p : nd_)
00078         if (!pazymysTinkamas(p))
00079             throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00080     if (!pazymysTinkamas(egzaminas_))
00081         throw std::runtime_error("Egzamino pažymys už ribų: " + std::to_string(egzaminas_));
00082
00083     ++gyvuStudentu;
00084 }
00085
00086 studentas::studentas(std::istream& is)
00087     : zmogus(),
00088     nd_(), egzaminas_(0),

```

```

00089     galutinisVid_(0.0), galutinisMed_(0.0)
00090 {
00091     ++gyvuStudentu;
00092     readStudentas(is);
00093 }
00094
00095 // =====
00096 // 2. RULE OF FIVE
00097 //
00098 // Visos penkios funkcijos kviečia atitinkamas BAZINĖS klasės
00099 // funkcijas -- kitaip vardas_ ir pavarde_ nebūtų nukopijuoti
00100 // arba perkelti (dažna klaida paveldėjime!).
00101 // =====
00102
00103 // -----
00104 // 2.1 DESTRUKTORIUS (override)
00105 //
00106 // Naikinimo tvarka atvirkštinė kūrimui:
00107 // ~studentas() + ~zmogus()
00108 // Bazinės klasės destruktorius kviečiamas AUTOMATIŠKAI --
00109 // jo rankiniu būdu kviešti nereikia ir negalima.
00110 // -----
00111 studentas::~studentas()
00112 {
00113     nd_.clear();
00114     nd_.shrink_to_fit();
00115     egzaminas_ = 0;
00116     galutinisVid_ = 0.0;
00117     galutinisMed_ = 0.0;
00118
00119     --gyvuStudentu;
00120     // ~zmogus() iškviečiamas automatiškai po šio kūno
00121 }
00122
00123 // -----
00124 // 2.2 KOPIJAVIMO KONSTRUKTORIUS
00125 //
00126 // zmogus(other) -- bazinės dalies kopijavimas.
00127 // Be šio kvietimo būtų iškviestas zmogus() numatytasis
00128 // konstruktorius, ir vardas_/pavarde_ liktų tušti.
00129 // -----
00130 studentas::studentas(const studentas& other)
00131 : zmogus(other), // bazinės dalies kopija
00132   nd_(other.nd_),
00133   egzaminas_(other.egzaminas_),
00134   galutinisVid_(other.galutinisVid_),
00135   galutinisMed_(other.galutinisMed_)
00136 {
00137     ++gyvuStudentu;
00138 }
00139
00140 // -----
00141 // 2.3 KOPIJAVIMO PRISKYRIMO OPERATORIUS
00142 //
00143 // zmogus::operator=(other) -- bazinės dalies priskyrimas.
00144 // Kvalifikuotas kvietimas būtinas, nes be jo įvyktų
00145 // begalinė rekursija (kvietų save patį).
00146 // -----
00147 studentas& studentas::operator=(const studentas& other)
00148 {
00149     if (this == &other) // apsauga nuo priskyrimo sau
00150         return *this;
00151
00152     zmogus::operator=(other); // bazinės dalies priskyrimas
00153
00154     nd_ = other.nd_;
00155     egzaminas_ = other.egzaminas_;
00156     galutinisVid_ = other.galutinisVid_;
00157     galutinisMed_ = other.galutinisMed_;
00158
00159     return *this;
00160 }
00161
00162 // -----
00163 // 2.4 PERKĖLIMO (MOVE) KONSTRUKTORIUS
00164 //
00165 // std::move(other) būtinas: other yra pavadintas kintamasis,
00166 // todėl pats savaime yra lvalue -- be std::move būtų iškviestas
00167 // zmogus KOPIJAVIMO, ne perkėlimo konstruktorius.
00168 //
00169 // noexcept -- kad std::vector persikirstymo metu naudotų
00170 // perkėlimą, ne kopijavimą (std::move_if_noexcept).
00171 // -----
00172 studentas::studentas(studentas&& other) noexcept
00173 : zmogus(std::move(other)), // bazinės dalies perkėlimas
00174   nd_(std::move(other.nd_)),
00175   egzaminas_(other.egzaminas_),

```

```

00176     galutinisVid_(other.galutinisVid_),
00177     galutinisMed_(other.galutinisMed_)
00178 {
00179     // Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje
00180     other.egzaminas_ = 0;
00181     other.galutinisVid_ = 0.0;
00182     other.galutinisMed_ = 0.0;
00183
00184     ++gyvuStudentu;
00185 }
00186
00187 // -----
00188 // 2.5 PERKĖLIMO (MOVE) PRISKYRIMO OPERATORIUS
00189 // -----
00190 studentas& studentas::operator=(studentas&& other) noexcept
00191 {
00192     if (this == &other)                // apsauga nuo priskyrimo sau
00193         return *this;
00194
00195     zmogus::operator=(std::move(other)); // bazinės dalies perkėlimas
00196
00197     nd_ = std::move(other.nd_);
00198     egzaminas_ = other.egzaminas_;
00199     galutinisVid_ = other.galutinisVid_;
00200     galutinisMed_ = other.galutinisMed_;
00201
00202     other.egzaminas_ = 0;
00203     other.galutinisVid_ = 0.0;
00204     other.galutinisMed_ = 0.0;
00205
00206     return *this;
00207 }
00208
00209 // =====
00210 // 3. GRYNAI VIRTUALIŲ METODŲ REALIZACIJA (override)
00211 // =====
00212
00213 // -----
00214 // galutinis -- bazinės klasės reikalaujamas įvertinimas.
00215 // Studentui tai galutinis balas pagal vidurkį.
00216 // -----
00217 double studentas::galutinis() const
00218 {
00219     return galutinisVid_;
00220 }
00221
00222 // -----
00223 // print -- realizuoja polimorfinį operator«
00224 // -----
00225 void studentas::print(std::ostream& os) const
00226 {
00227     os << std::left << std::setw(20) << vardas_
00228         << std::left << std::setw(20) << pavarde_
00229         << std::fixed << std::setprecision(2)
00230         << std::left << std::setw(20) << galutinisVid_
00231         << std::left << std::setw(20) << galutinisMed_;
00232 }
00233
00234 // -----
00235 // read -- realizuoja polimorfinį operator«
00236 // -----
00237 std::istream& studentas::read(std::istream& is)
00238 {
00239     return readStudentas(is);
00240 }
00241
00242 // -----
00243 // tipas -- žmogaus tipo pavadinimas
00244 // -----
00245 std::string studentas::tipas() const
00246 {
00247     return "Studentas";
00248 }
00249
00250 // =====
00251 // 4. SET'ERIAI
00252 // =====
00253
00254 void studentas::setNd(const std::vector<int>& nd) {
00255     for (int p : nd)
00256         if (!pazymysTinkamas(p))
00257             throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00258     nd_ = nd;
00259 }
00260
00261 void studentas::setEgzaminas(int e) {
00262     if (!pazymysTinkamas(e))

```

```

00263         throw std::runtime_error("Egzamino pažymys už ribų: " + std::to_string(e));
00264     egzaminas_ = e;
00265 }
00266
00267 void studentas::addPažymys(int p) {
00268     if (!pažymysTinkamas(p))
00269         throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00270     nd_.push_back(p);
00271 }
00272
00273 void studentas::isvalyk() {
00274     vardas_.clear(); // protected laukas iš zmogus
00275     pavarde_.clear(); // protected laukas iš zmogus
00276     nd_.clear();
00277     nd_.shrink_to_fit();
00278     egzaminas_ = 0;
00279     galutinisVid_ = 0.0;
00280     galutinisMed_ = 0.0;
00281 }
00282
00283 // =====
00284 // 5. SKAIČIAVIMAS
00285 // =====
00286
00287 void studentas::calculateGalutinis() {
00288     galutinisVid_ = namuDarbaiSvoris * vidurkis(nd_) + egzaminasSvoris * egzaminas_;
00289     galutinisMed_ = namuDarbaiSvoris * mediana(nd_) + egzaminasSvoris * egzaminas_;
00290 }
00291
00292 // =====
00293 // 6. ĮVESTIS
00294 // =====
00295
00296 // -----
00297 // parseFromLine -- GREITAS parsinimas failo skaitymui
00298 // Nenaudoja std::istream, naudoja std::from_chars
00299 // -----
00300 void studentas::parseFromLine(const std::string& line, int ndCount, int lineNumber)
00301 {
00302     const char* p = line.data();
00303     const char* end = p + line.size();
00304
00305     auto skipSpace = [&]() {
00306         while (p < end && (*p == ' ' || *p == '\t' || *p == '\r')) ++p;
00307     };
00308
00309     // --- Vardas ---
00310     skipSpace();
00311     const char* start = p;
00312     while (p < end && *p != ' ' && *p != '\t' && *p != '\r') ++p;
00313     if (start == p)
00314         throw std::runtime_error(vietosTekstas(lineNumber) + " trūksta vardo.");
00315     vardas_.assign(start, p - start);
00316
00317     // --- Pavardė ---
00318     skipSpace();
00319     start = p;
00320     while (p < end && *p != ' ' && *p != '\t' && *p != '\r') ++p;
00321     if (start == p)
00322         throw std::runtime_error(vietosTekstas(lineNumber) + " trūksta pavardės.");
00323     pavarde_.assign(start, p - start);
00324
00325     // --- Pažymiai ---
00326     auto readInt = [&](const char* ko) -> int {
00327         skipSpace();
00328         int v = 0;
00329         auto res = std::from_chars(p, end, v);
00330         if (res.ec != std::errc())
00331             throw std::runtime_error(vietosTekstas(lineNumber)
00332                                     + " trūksta " + ko + " pažymio.");
00333         p = res.ptr;
00334         return v;
00335     };
00336
00337     nd_.clear();
00338     nd_.reserve(ndCount);
00339
00340     for (int i = 0; i < ndCount; ++i) {
00341         int v = readInt("namų darbo");
00342         if (!pažymysTinkamas(v))
00343             throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", v));
00344         nd_.push_back(v);
00345     }
00346
00347     egzaminas_ = readInt("egzamino");
00348     if (!pažymysTinkamas(egzaminas_))
00349         throw std::runtime_error(ribuKlaida(lineNumber, "egzamino", egzaminas_));

```



```

00350 }
00351
00352 // -----
00353 // readStudentas -- įvestis per std::istream
00354 // -----
00355 std::istream& studentas::readStudentas(std::istream& is, int ndCount, int lineNumber) {
00356     if (!(is » vardas_ » pavarde_))
00357         throw std::runtime_error(vietosTekstas(lineNumber)
00358             + " trūksta vardo arba pavardės.");
00359     nd_.clear();
00360
00361     if (ndCount > 0) {
00362         nd_.reserve(ndCount);
00363         for (int i = 0; i < ndCount; ++i) {
00364             int p;
00365             if (!(is » p))
00366                 throw std::runtime_error(vietosTekstas(lineNumber)
00367                     + " trūksta namų darbo pažymio.");
00368             if (!pazymysTinkamas(p))
00369                 throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", p));
00370             nd_.push_back(p);
00371         }
00372         if (!(is » egzaminas_))
00373             throw std::runtime_error(vietosTekstas(lineNumber)
00374                 + " trūksta egzamino pažymio.");
00375     }
00376     else {
00377         // Nežinomas skaičius -- skaitome iki galo, paskutinis yra egzaminas
00378         int p;
00379         while (is » p) {
00380             if (!pazymysTinkamas(p))
00381                 throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", p));
00382             nd_.push_back(p);
00383         }
00384         if (nd_.empty())
00385             throw std::runtime_error(vietosTekstas(lineNumber)
00386                 + " trūksta egzamino pažymio.");
00387         egzaminas_ = nd_.back();
00388         nd_.pop_back();
00389         is.clear(); // eilutės pabaiga nėra klaida šiame režime
00390     }
00391     if (!pazymysTinkamas(egzaminas_))
00392         throw std::runtime_error(ribuKlaida(lineNumber, "egzamino", egzaminas_));
00393     return is;
00394 }
00395
00400 // =====
00401 // 7. IŠVESTIS
00402 // =====
00403
00404 // -----
00405 // appendListTo -- GREITAS formatavimas failo rašymui (std::to_chars)
00406 // -----
00407 void studentas::appendListTo(std::string& out) const
00408 {
00409     out.append(vardas_);
00410     if (vardas_.size() < 25) out.append(25 - vardas_.size(), ' ');
00411
00412     out.append(pavarde_);
00413     if (pavarde_.size() < 25) out.append(25 - pavarde_.size(), ' ');
00414
00415     char buf[16];
00416     auto appendInt = [&](int v) {
00417         auto res = std::to_chars(buf, buf + sizeof(buf), v);
00418         size_t len = static_cast<size_t>(res.ptr - buf);
00419         if (len < 10) out.append(10 - len, ' ');
00420         out.append(buf, len);
00421     };
00422
00423     for (int nd : nd_)
00424         appendInt(nd);
00425     appendInt(egzaminas_);
00426
00427     out.push_back('\n');
00428 }
00429
00430 // =====
00431 // 8. PERDENGTI OPERATORIAI
00432 // =====
00433
00434 bool studentas::operator==(const studentas& other) const {
00435     return vardas_ == other.vardas_
00436         && pavarde_ == other.pavarde_

```

```

00437         && nd_ == other.nd_
00438         && egzaminas_ == other.egzaminas_;
00439     }
00440
00441     bool studentas::operator!=(const studentas& other) const {
00442         return !(*this == other);
00443     }
00444
00445     // Rikiavimas pagal pavardę, esant vienodoms -- pagal vardą
00446     bool studentas::operator<(const studentas& other) const {
00447         if (pavarde_ != other.pavarde_)
00448             return pavarde_ < other.pavarde_;
00449         return vardas_ < other.vardas_;
00450     }
00451
00452     bool studentas::operator>(const studentas& other) const {
00453         return other < *this;
00454     }
00455
00456     int studentas::operator[](size_t i) const {
00457         if (i >= nd_.size())
00458             throw std::out_of_range("Namų darbo indeksas už ribų: " + std::to_string(i));
00459         return nd_[i];
00460     }
00461
00462     studentas::operator bool() const {
00463         return !vardas_.empty() && !pavarde_.empty();
00464     }
00465
00466     // =====
00467     // 9. NE-NARIAI LYGINIMO FUNKCIJOS
00468     // =====
00469
00470     bool comparePagalVarda(const studentas& s1, const studentas& s2){
00471         return s1.vardas() < s2.vardas();
00472     }
00473
00474     bool comparePagalPavarde(const studentas& s1, const studentas& s2) {
00475         return s1.pavarde() < s2.pavarde();
00476     }
00477
00478     bool comparePagalEgzamina(const studentas& s1, const studentas& s2) {
00479         return s1.egzaminas() > s2.egzaminas();
00480     }
00481
00482     bool comparePagalVidurki(const studentas& s1, const studentas& s2) {
00483         return s1.galutinisVid() > s2.galutinisVid();
00484     }
00485
00486     bool comparePagalMediana(const studentas& s1, const studentas& s2) {
00487         return s1.galutinisMed() > s2.galutinisMed();
00488     }

```

8.33 OOP_2/studentas.h Failo Nuoroda

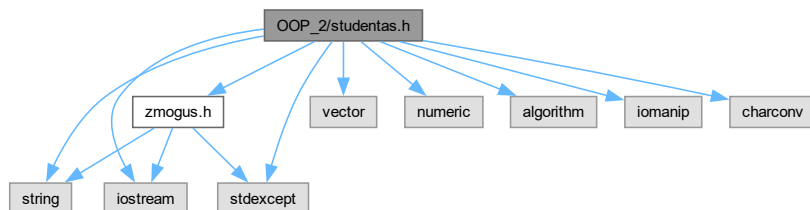
Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas.

```

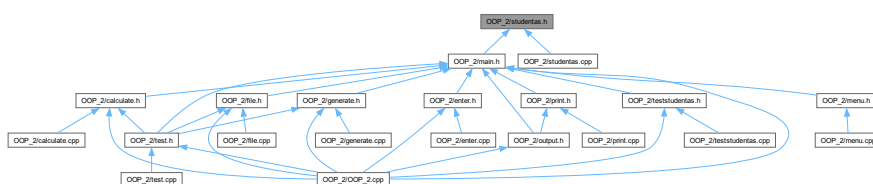
#include "zmogus.h"
#include <string>
#include <vector>
#include <iostream>
#include <numeric>
#include <algorithm>
#include <iomanip>
#include <stdexcept>
#include <charconv>

```

Įtraukimo priklausomybių diagrama studentas.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class **studentas**

Studento duomenys — išvestinė klasė iš abstrakčios zmogus.

Funkcijos

Ne-nariai lyginimo funkcijos

Naudojamos `sortStudentai()` šablone kaip komparatoriai.

Nēra klasēs nariai, nes naudoja tik viešą interfeisą — taip laikomasi principo, kad klasės narių turi būti kuo mažiau.

- bool **comparePagalVarda** (const **studentas** &s1, const **studentas** &s2)
Lygina pagal vardą (didėjančiai).
- bool **comparePagalPavarde** (const **studentas** &s1, const **studentas** &s2)
Lygina pagal pavardę (didėjančiai).
- bool **comparePagalEgzamina** (const **studentas** &s1, const **studentas** &s2)
Lygina pagal egzamino pažymį (mažėjančiai).
- bool **comparePagalVidurki** (const **studentas** &s1, const **studentas** &s2)
Lygina pagal galutinį balą su vidurkiu (mažėjančiai).
- bool **comparePagalMediana** (const **studentas** &s1, const **studentas** &s2)
Lygina pagal galutinį balą su mediana (mažėjančiai).

8.33.1 Smulkus aprašymas

Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas.

Autorius

Marius Augustinas

Data

2026

Apibrēžimas faile [studentas.h.](#)

8.33.2 Funkcijos Dokumentacija

8.33.2.1 comparePagalVarda()

```
bool comparePagalVarda (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal vardą (didėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

`true` jei *s1* vardas abėcėliškai anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 470.

Ryšiai [zmogus::vardas\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.2.2 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal pavardę (didėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

`true` jei *s1* pavardė abėcėliškai anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 474.

Ryšiai [zmogus::pavarde\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.2.3 comparePagalEgzamina()

```
bool comparePagalEgzamina (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal egzamino pažymį (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
    true jei s1 egzaminas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 478.

Ryšiai [studentas::egzaminas\(\)](#).

8.33.2.4 comparePagalVidurki()

```
bool comparePagalVidurki (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal galutinį balą su vidurkiu (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
    true jei s1 galutinis balas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 482.

Ryšiai [studentas::galutinisVid\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.2.5 comparePagalMediana()

```
bool comparePagalMediana (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal galutinį balą su mediana (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
    true jei s1 galutinis balas didesnis
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 486.

Ryšiai [studentas::galutinisMed\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.34 studentas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STUDENTAS_H_DEFINED
00002 #define STUDENTAS_H_DEFINED
00003
00004 #include "zmogus.h"
00005
00006 #include <string>
00007 #include <vector>
00008 #include <iostream>
00009 #include <numeric>
00010 #include <algorithm>
```

```

00011 #include <iomanip>
00012 #include <stdexcept>
00013 #include <charconv>
00014
00015 /**
00016  * @file studentas.h
00017  * @brief Išvestinė klasė studentas -- studento duomenys ir galutinių balų skaičiavimas.
00018  * @author Marius Augustinas
00019  * @date 2026
00020  */
00021
00022 /**
00023  * @class studentas
00024  * @brief Studento duomenys -- išvestinė klasė iš abstrakčios zmogus.
00025  *
00026  * Prie paveldėto vardo ir pavardės prideda namų darbų pažymius,
00027  * egzamino įvertinimą ir du galutinius balus (pagal vidurkį ir pagal medianą).
00028  *
00029  * @par Paveldėjimas
00030  * Iš zmogus paveldi:
00031  * - `vardas_`, `pavarde_` ('protected' laukai)
00032  * - vardas(), pavarde(), pilnasVardas() get'eries
00033  * - setVardas(), setPavarde() set'eries su validacija
00034  * - virtualų destruktorių
00035  *
00036  * Realizuoja visus keturis grynai virtualius bazinės klasės metodus:
00037  * galutinis(), print(), read(), tipas().
00038  *
00039  * @par Galutinio balo formulė
00040  * @code
00041  * galutinis = 0.4 * namų_darbų_vidurkis_arba_mediana + 0.6 * egzaminas
00042  * @endcode
00043  * Svariai apibrėžti konstantomis #namuDarbaiSvoris ir #egzaminasSvoris.
00044  *
00045  * @par Naudojimo pavyzdys
00046  * @code
00047  * // Sukūrimas su pilnu konstruktoriumi
00048  * studentas s("Jonas", "Jonaitis", {5, 6, 7, 8}, 9);
00049  * s.calculateGalutinis();
00050  * std::cout << s << "\n";
00051  *
00052  * // Nuskaitymas iš srauto
00053  * std::istringstream iss("Ona Onaite 10 9 8 7");
00054  * studentas s2;
00055  * iss >> s2;
00056  *
00057  * // Polimorfinis naudojimas
00058  * std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00059  *     "Petras", "Petraitis", std::vector<int>{5, 5}, 8);
00060  * std::cout << p->tipas() << ": " << p->galutinis() << "\n";
00061  * @endcode
00062  *
00063  * @warning Galutiniai balai NĖRA skaičiuojami automatiškai kuriant objektą --
00064  * po duomenų nuskaitymo ar sukūrimo reikia iškviešti
00065  * calculateGalutinis(). Toks sprendimas leidžia matuoti nuskaitymo
00066  * ir skaičiavimo spartą atskirai.
00067  *
00068  * @see zmogus
00069  */
00070 class studentas : public zmogus {
00071 private:
00072     std::vector<int> nd_;           ///< Namų darbų pažymiai (kiekvienas 1-10)
00073     int egzaminas_;               ///< Egzamino pažymys (1-10)
00074     double galutinisVid_;         ///< Galutinis balas pagal namų darbų vidurkį
00075     double galutinisMed_;         ///< Galutinis balas pagal namų darbų medianą
00076
00077     /**
00078      * @brief Apskaičiuoja aritmetinį vidurkį.
00079      * @param nd Pažymių vektorius
00080      * @return Vidurkis; 0.0 jei vektorius tuščias
00081      */
00082     static double vidurkis(const std::vector<int>& nd);
00083
00084     /**
00085      * @brief Apskaičiuoja medianą.
00086      *
00087      * Nelyginiam elementų kiekiui gražina vidurinį elementą,
00088      * lyginiam -- dviejų vidurinių aritmetinį vidurkį.
00089      * Įeinantis vektorius nekeičiamas (kopijuojamas ir rikiuojamas viduje).
00090      *
00091      * @param nd Pažymių vektorius (nebūtinai surikiuotas)
00092      * @return Mediana; 0.0 jei vektorius tuščias
00093      */
00094     static double mediana(const std::vector<int>& nd);
00095
00096     /**
00097      * @brief Patikrina, ar pažymys patenka į leistiną intervalą.

```

```

00098     *
00099     * Samoningai gražina `bool`, o ne meta išimtį: klaidos pranešimo
00100     * formavimas (`std::string` konkatenacija) yra brangus, o su 10 mln.
00101     * studentų ši funkcija kviečiama ~160 mln. kartų. Pranešimas
00102     * formuojamas tik ten, kur išimtis tikrai metama.
00103     *
00104     * @param p Tikrinamas pažymys
00105     * @return `true` jei #minPazymys p #maxPazymys
00106     */
00107     static inline bool pazymysTinkamas(int p) {
00108         return p >= minPazymys && p <= maxPazymys;
00109     }
00110
00111 public:
00112     /// @name Konstantos
00113     /// @{
00114     static constexpr int minPazymys = 1;           ///< Mažiausias leistinas pažymys
00115     static constexpr int maxPazymys = 10;          ///< Didžiausias leistinas pažymys
00116     static constexpr double namuDarbaiSvoris = 0.4; ///< Namų darbų svoris galutiniame balse
00117     static constexpr double egzaminasSvoris = 0.6; ///< Egzamino svoris galutiniame balse
00118     /// @}
00119
00120     /**
00121     * @brief Gyvuojančių studentų skaitiklis.
00122     *
00123     * Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje.
00124     * Naudojamas testuose objektų nutekėjimui aptikti -- atskiras nuo
00125     * zmogus::gyvuZmoniu, todėl galima patikrinti, ar iškviešti abu
00126     * destruktoriai (~studentas() ir ~zmogus()).
00127     */
00128     static int gyvuStudentu;
00129
00130     // =====
00131     /// @name Konstruktoriai
00132     /// @{
00133     // =====
00134
00135     /**
00136     * @brief Numatytasis konstruktorius.
00137     *
00138     * Sukuria tuščią studentą: be vardo, pavardės ir pažymių,
00139     * su nuliniiais galutiniais balais. Skirtas objektui, į kurį
00140     * duomenys bus nuskaityti vėliau (pvz. per readStudentas()).
00141     */
00142     studentas();
00143
00144     /**
00145     * @brief Pilnas konstruktorius su validacija.
00146     *
00147     * Vardą ir pavardę validuoja bazinė klasė, pažymius -- ši klasė.
00148     *
00149     * @param vardas Studento vardas (negali būti tuščias)
00150     * @param pavarde Studento pavardė (negali būti tuščia)
00151     * @param nd Namų darbų pažymiai (kiekvienas 1-10)
00152     * @param egzaminas Egzamino pažymys (1-10)
00153     *
00154     * @throws std::invalid_argument jei vardas arba pavardė tuščia
00155     * @throws std::runtime_error jei bent vienas pažymys už ribų
00156     *
00157     * @note Galutiniai balai lieka 0.0 -- reikia iškviešti calculateGalutinis().
00158     */
00159     studentas(const std::string& vardas,
00160               const std::string& pavarde,
00161               const std::vector<int>& nd,
00162               int egzaminas);
00163
00164     /**
00165     * @brief Konstruktorius iš įvesties srauto.
00166     *
00167     * Iškart nuskaito duomenis per readStudentas().
00168     * Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas
00169     * iš srauto į studentą.
00170     *
00171     * @param is Įvesties srautas
00172     * @throws std::runtime_error jei duomenys netinkami arba jų trūksta
00173     */
00174     explicit studentas(std::istream& is);
00175
00176     /// @}
00177
00178     // =====
00179     /// @name Rule of Five
00180     ///
00181     /// Visos penkios funkcijos realizuotos pilnai (ne `= default`).
00182     /// Kiekviena **privalo** iškviešti atitinkamą bazinės klasės funkciją --
00183     /// be to `vardas` ir `pavarde` liktų tušti arba nenukopijuoti.
00184     /// Tai dažniausia paveldėjimo klaida, kurios kompiliatorius neaptinka.

```

```

00185     /// @{
00186     // =====
00187
00188     /**
00189     * @brief Destruktorius.
00190     *
00191     * Išvalo studento duomenis ir sumažina #gyvuStudentu skaitiklį.
00192     * Bazinės klasės destruktoriaus ~zmogus() iškviečiamas
00193     * **automatiškai** po šio kūno -- jo kviesti rankiniu būdu nereikia.
00194     *
00195     * @par Naikinimo tvarka
00196     * ~studentas() + ~zmogus() (atvirkštinė kūrimo tvarkai)
00197     */
00198     ~studentas() override;
00199
00200     /**
00201     * @brief Kopijavimo konstruktorius.
00202     *
00203     * Sukuria gilia kopiją (deep copy) -- `std::string` ir `std::vector`
00204     * laukai kopijuoja savo vidinius buferius, todėl objektai lieka
00205     * visiškai nepriklausomi.
00206     *
00207     * @param other Kopijuojamas studentas
00208     *
00209     * @note Member initializer list prasideda `zmogus(other)` -- be šio
00210     * kvietimo būtų iškviestas `zmogus()` numatytasis konstruktorius,
00211     * ir vardas su pavarde liktų tušti.
00212     */
00213     studentas(const studentas& other);
00214
00215     /**
00216     * @brief Kopijavimo priskyrimo operatorius.
00217     *
00218     * @param other Priskiriamas studentas
00219     * @return Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)
00220     *
00221     * @note Turi apsaugą nuo priskyrimo sau (`this == &other`) -- be jos
00222     * `a = a` galėtų sugadinti duomenis.
00223     * @note Bazinė dalis priskiriama kvalifikuotu kvietimu
00224     * `zmogus::operator=(other)`. Be kvalifikacijos kompiliatorius
00225     * parinktų `studentas::operator=` ir įvyktų begalinė rekursija.
00226     */
00227     studentas& operator=(const studentas& other);
00228
00229     /**
00230     * @brief Perkėlimo (move) konstruktorius.
00231     *
00232     * Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas
00233     * galiojančioje, bet tuščioje būsenoje -- jį saugu naikinti arba
00234     * naudoti iš naujo.
00235     *
00236     * @param other Studentas, iš kurio perkeliami duomenys
00237     *
00238     * @warning Bazinė dalis perkeliama `zmogus(std::move(other))`.
00239     * `std::move` čia būtinas: `other` yra pavadintas
00240     * kintamasis, todėl pats savaime lvalue -- be `std::move`
00241     * būtų iškviestas `zmogus` **kopijavimo**, ne perkėlimo
00242     * konstruktorius.
00243     *
00244     * @note `noexcept` kritiškai svarbus spartai: `std::vector`
00245     * persikirstymo metu naudoja `std::move_if_noexcept`, todėl
00246     * be `noexcept` rinktųsi kopijavimo konstruktorių.
00247     */
00248     studentas(studentas&& other) noexcept;
00249
00250     /**
00251     * @brief Perkėlimo (move) priskyrimo operatorius.
00252     *
00253     * @param other Studentas, iš kurio perkeliami duomenys
00254     * @return Nuoroda į šį objektą
00255     *
00256     * @note Bazinė dalis perkeliama `zmogus::operator=(std::move(other))`.
00257     */
00258     studentas& operator=(studentas&& other) noexcept;
00259
00260     /// @}
00261
00262     // =====
00263     /// @name Grynai virtualių metodų realizacija
00264     /// @{
00265     // =====
00266
00267     /**
00268     * @brief Gražina galutinį įvertinimą (bazinės klasės interfeisas).
00269     *
00270     * Studentui tai galutinis balas pagal namų darbų **vidurkį**.
00271     * Medianos variantas prieinamas per galutinisMed().

```



```

00272     *
00273     * @return Galutinis balas pagal vidurkį
00274     * @warning Gražina 0.0, jei nebuvo iškviesta calculateGalutinis().
00275     */
00276     double galutinis() const override;
00277
00278     /**
00279     * @brief Išveda studentą į srautą.
00280     *
00281     * Formatas: vardas (20 simbolių), pavardė (20), galutinis pagal
00282     * vidurkį (20), galutinis pagal medianą (20).
00283     * Realizuoja polimorfinį `operator«`.
00284     *
00285     * @param os Išvesties srautas
00286     */
00287     void print(std::ostream& os) const override;
00288
00289     /**
00290     * @brief Nuskaito studentą iš srauto.
00291     *
00292     * Realizuoja polimorfinį `operator»`. Deleguoja į readStudentas().
00293     *
00294     * @param is Įvesties srautas
00295     * @return Nuoroda į tą patį srautą
00296     */
00297     std::istream& read(std::istream& is) override;
00298
00299     /**
00300     * @brief Gražina tipo pavadinimą.
00301     * @return `"Studentas"`
00302     */
00303     std::string tipas() const override;
00304
00305     /// @}
00306
00307     // =====
00308     /// @name Get'eriai
00309     /// @{
00310     // =====
00311
00312     /**
00313     * @brief Gražina namų darbų pažymius.
00314     * @return Konstanti nuoroda į vektorių (be kopijavimo)
00315     */
00316     inline const std::vector<int>& nd() const { return nd_; }
00317
00318     /// @brief Gražina egzamino pažymį.
00319     inline int egzaminas() const { return egzaminas_; }
00320
00321     /// @brief Gražina galutinį balą pagal namų darbų vidurkį.
00322     inline double galutinisVid() const { return galutinisVid_; }
00323
00324     /// @brief Gražina galutinį balą pagal namų darbų medianą.
00325     inline double galutinisMed() const { return galutinisMed_; }
00326
00327     /**
00328     * @brief Patikrina, ar objektas neturi vardo ir pavardės.
00329     * @return `true` jei ir vardas, ir pavardė tušti
00330     */
00331     inline bool tuscias() const { return vardas_.empty() && pavarde_.empty(); }
00332
00333     /// @}
00334
00335     // =====
00336     /// @name Set'eriai
00337     /// @{
00338     // =====
00339
00340     /**
00341     * @brief Nustato namų darbų pažymius su validacija.
00342     * @param nd Pažymių vektorius
00343     * @throws std::runtime_error jei bent vienas pažymys už ribų
00344     */
00345     void setNd(const std::vector<int>& nd);
00346
00347     /**
00348     * @brief Nustato egzamino pažymį su validacija.
00349     * @param e Egzamino pažymys
00350     * @throws std::runtime_error jei pažymys už ribų
00351     */
00352     void setEgzaminas(int e);
00353
00354     /**
00355     * @brief Prideda vieną namų darbo pažymį.
00356     * @param p Pridedamas pažymys
00357     * @throws std::runtime_error jei pažymys už ribų
00358     */

```

```

00359 void addPazymys(int p);
00360
00361 /**
00362  * @brief Ištuština objektą.
00363  *
00364  * Išvalo **ir bazinės, ir išvestinės** klasės laukus. Vektoriaus
00365  * atmintis faktiškai atlaisvinama ('shrink_to_fit').
00366  */
00367 void isvalyk();
00368
00369 /// @}
00370
00371 // =====
00372 /// @name Skaičiavimas
00373 /// @{
00374 // =====
00375
00376 /**
00377  * @brief Apskaičiuoja abu galutinius balus.
00378  *
00379  * Užpildo `galutinisVid` ir `galutinisMed` pagal formulę:
00380  * @code
00381  * galutinisVid = 0.4 * vidurkis(nd) + 0.6 * egzaminas
00382  * galutinisMed = 0.4 * mediana(nd) + 0.6 * egzaminas
00383  * @endcode
00384  *
00385  * Tuščiam namų darbų sąrašui vidurkis ir mediana laikomi 0.0,
00386  * todėl galutinis balas yra `0.6 * egzaminas`.
00387  *
00388  * @note Kviečiamas atskirai, ne konstruktoriuje -- taip nuskaitymo
00389  * ir skaičiavimo spartą galima matuoti nepriklausomai.
00390  */
00391 void calculateGalutinis();
00392
00393 /// @}
00394
00395 // =====
00396 /// @name Įvestis
00397 /// @{
00398 // =====
00399
00400 /**
00401  * @brief Nuskaito studentą iš srauto.
00402  *
00403  * Veikia dviem režimais priklausomai nuo @p ndCount:
00404  * - **ndCount > 0** -- nuskaito tiksliai tiek namų darbų pažymių,
00405  *   po jų -- egzaminą. Naudojama, kai stulpelių skaičius žinomas
00406  *   iš failo antraštės.
00407  * - **ndCount == 0** -- skaito skaičius iki eilutės galo, paskutinį
00408  *   laiko egzaminu. Naudojama interaktyviai įvesčiai.
00409  *
00410  * @param is Įvesties srautas
00411  * @param ndCount Namų darbų skaičius (0 = skaityti iki galo)
00412  * @param lineNumber Eilutės numeris klaidos pranešimui (0 = nenaudoti)
00413  * @return Nuoroda į tą patį srautą
00414  *
00415  * @throws std::runtime_error jei trūksta duomenų arba pažymys už ribų
00416  *
00417  * @note Failo skaitymui naudokite parseFromLine() -- jis kelis kartus
00418  * greitesnis, nes neturi srauto mechanizmo.
00419  */
00420 std::istream& readStudentas(std::istream& is, int ndCount = 0, int lineNumber = 0);
00421
00422 /**
00423  * @brief Greitas parsinimas iš eilutės (failo skaitymui).
00424  *
00425  * Optimizuota alternatyva readStudentas() metodui masiniam
00426  * duomenų apdorojimui:
00427  * - **nekuriamas** `std::istream` objektas kiekvienai eilutei
00428  * - skaičiai parsinami su `std::from_chars` (C++17) -- be locale,
00429  *   be stream sentry, be būsenos tikrinimo
00430  * - klaidos pranešimai formuojami tik metant išimtį
00431  *
00432  * Su 10 mln. studentų šie trys pakeitimai sutrumpina failo
00433  * nuskaitymą nuo ~90 s iki ~30 s.
00434  *
00435  * @param line Nuskaitoma eilutė
00436  * @param ndCount Namų darbų stulpelių skaičius (iš failo antraštės)
00437  * @param lineNumber Eilutės numeris klaidos pranešimui
00438  *
00439  * @throws std::runtime_error jei trūksta vardo, pavardės ar pažymio,
00440  * arba pažymys už ribų
00441  *
00442  * @see appendListTo() -- atvirkštinė operacija
00443  */
00444 void parseFromLine(const std::string& line, int ndCount, int lineNumber);
00445

```

```

00446     /// @}
00447
00448     // =====
00449     /// @name Išvestis
00450     /// @{
00451     // =====
00452
00453     /**
00454      * @brief Prideda suformatuotą eilutę į buferį (failo rašymui).
00455      *
00456      * Optimizuota alternatyva `operator«` masiniam rašymui:
00457      * - skaičiai formatuojami su `std::to_chars` -- be `std::setw`
00458      *   manipuliatorių
00459      * - rašoma į bendrą `std::string` buferį, ne į srautą
00460      *
00461      * Kviečianti funkcija kaupia eilutes ir iškviečia vieną
00462      * `file.write()` kas ~1 MB. Buferis nedidėja proporcingai studentų
00463      * skaičiui -- `clear()` išlaiko `capacity`.
00464      *
00465      * Formatas: vardas (25), pavardė (25), kiekvienas ND pažymys (10),
00466      * egzaminas (10), naujos eilutės simbolis.
00467      *
00468      * @param out Buferis, į kurį pridėjama eilutė
00469      *
00470      * @see parseFromLine() -- atvirkštinė operacija; kartu jos sudaro
00471      *   round-trip: išvesta eilutė nuskaitoma atgal be pakeitimų
00472      */
00473     void appendListTo(std::string& out) const;
00474
00475     /// @}
00476
00477     // =====
00478     /// @name Perdengti operatoriai
00479     ///
00480     /// Programoje nenaudojami, bet realizuoti būsimiems klasės
00481     /// naudotojams -- jie leidžia naudoti studentą su STL algoritmais
00482     /// ir asociatyviais konteineriais.
00483     /// @{
00484     // =====
00485
00486     /**
00487      * @brief Lygina du studentus.
00488      *
00489      * Lyginami vardas, pavardė, namų darbų pažymiai ir egzaminas.
00490      * Galutiniai balai **nelyginami**, nes jie išvedami iš kitų laukų.
00491      *
00492      * @param other Lyginamas studentas
00493      * @return `true` jei visi duomenys sutampa
00494      */
00495     bool operator==(const studentas& other) const;
00496
00497     /**
00498      * @brief Tikrina, ar studentai skiriasi.
00499      * @param other Lyginamas studentas
00500      * @return `!(this == other)`
00501      */
00502     bool operator!=(const studentas& other) const;
00503
00504     /**
00505      * @brief Rikiavimo operatorius.
00506      *
00507      * Lygina pagal pavardę; esant vienodoms pavardėms -- pagal vardą.
00508      * Leidžia naudoti `std::sort` be komparatoriaus ir dėti studentus
00509      * į `std::set` ar `std::map`.
00510      *
00511      * @param other Lyginamas studentas
00512      * @return `true` jei šis studentas eina anksčiau
00513      */
00514     bool operator<(const studentas& other) const;
00515
00516     /**
00517      * @brief Atvirkštinis rikiavimo operatorius.
00518      * @param other Lyginamas studentas
00519      * @return `other < *this`
00520      */
00521     bool operator>(const studentas& other) const;
00522
00523     /**
00524      * @brief Prieiga prie namų darbo pažymio pagal indeksą.
00525      *
00526      * @param i Pažymio indeksas (nuo 0)
00527      * @return Pažymys
00528      * @throws std::out_of_range jei indeksas už ribų
00529      */
00530     int operator[](size_t i) const;
00531
00532     /**

```

```

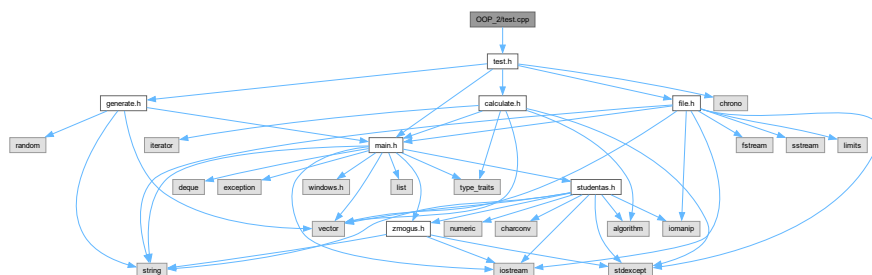
00533     * @brief Konvertavimas į `bool` -- ar objektas turi duomenų.
00534     *
00535     * Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas į `int`
00536     * (pvz. `s + 1` nesikompiliuos, kaip ir turi būti).
00537     *
00538     * @return `true` jei vardas ir pavardė nėra tušti
00539     *
00540     * @par Pavyzdys
00541     * @code
00542     * studentas s;
00543     * if (!s) std::cout << "Objektas tuscias\n";
00544     * @endcode
00545     */
00546     explicit operator bool() const;
00547
00548     /// @}
00549 };
00550
00551 // =====
00552 /// @name Ne-nariai lyginimo funkcijos
00553 ///
00554 /// Naudojamos `sortStudentai()` šablone kaip komparatoriai. Nėra klasės
00555 /// nariai, nes naudoja tik viešą interfeisą -- taip laikomasi principo,
00556 /// kad klasės narių turi būti kuo mažiau.
00557 /// @{
00558 // =====
00559
00560 /**
00561  * @brief Lygina pagal vardą (didėjančiai).
00562  * @param s1 Pirmas studentas
00563  * @param s2 Antras studentas
00564  * @return `true` jei `s1` vardas abėcėliškai anksčiau
00565  */
00566 bool comparePagalVarda(const studentas& s1, const studentas& s2);
00567
00568 /**
00569  * @brief Lygina pagal pavardę (didėjančiai).
00570  * @param s1 Pirmas studentas
00571  * @param s2 Antras studentas
00572  * @return `true` jei `s1` pavardė abėcėliškai anksčiau
00573  */
00574 bool comparePagalPavarde(const studentas& s1, const studentas& s2);
00575
00576 /**
00577  * @brief Lygina pagal egzamino pažymį (mažėjančiai).
00578  * @param s1 Pirmas studentas
00579  * @param s2 Antras studentas
00580  * @return `true` jei `s1` egzaminas didesnis
00581  */
00582 bool comparePagalEgzamina(const studentas& s1, const studentas& s2);
00583
00584 /**
00585  * @brief Lygina pagal galutinį balą su vidurkiu (mažėjančiai).
00586  * @param s1 Pirmas studentas
00587  * @param s2 Antras studentas
00588  * @return `true` jei `s1` galutinis balas didesnis
00589  */
00590 bool comparePagalVidurki(const studentas& s1, const studentas& s2);
00591
00592 /**
00593  * @brief Lygina pagal galutinį balą su mediana (mažėjančiai).
00594  * @param s1 Pirmas studentas
00595  * @param s2 Antras studentas
00596  * @return `true` jei `s1` galutinis balas didesnis
00597  */
00598 bool comparePagalMediana(const studentas& s1, const studentas& s2);
00599
00600 /// @}
00601
00602 #endif // STUDENTAS_H_DEFINED

```

8.35 OOP_2/test.cpp Failo Nuoroda

```
#include "test.h"
```

Įtraukimo priklausomybių diagrama test.cpp:



Funkcijos

- `template<typename Container>`
static void `testContainer` (const std::string &label, const std::string &filename)
- void `testContainers` (int n)

8.35.1 Funkcijos Dokumentacija

8.35.1.1 testContainer()

```
template<typename Container>
void testContainer (
    const std::string & label,
    const std::string & filename) [static]
```

Apibrėžimas failo `test.cpp` eilutėje 12.

Ryšiai `calculateGalutinis()`, `partitionStudentai()`, `readStudentaiFromFile()`, ir `sortStudentai()`.

Naudojamas `testContainers()`.

8.35.1.2 testContainers()

```
void testContainers (
    int n)
```

Apibrėžimas failo `test.cpp` eilutėje 50.

Ryšiai `testContainer()`.

Naudojamas `runProgram()`.

8.36 test.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "test.h"
00002
00003 // -----
00004 // Vidinis šablonas -- naudojamas tik šiame faile.
00005 // Matuoja žingsnius su pasirinktu konteineriu:
00006 // 1. Nuskaitymas iš failo
00007 // 2. Galutinių balų skaičiavimas
00008 // 3. Rikiavimas
00009 // 4. Skirstymas į dvi grupes
00010 // -----
00011 template<typename Container>
00012 static void testContainer(const std::string& label, const std::string& filename)
00013 {
00014     std::cout << u8" [" << label << u8"]\n";
00015
00016     // 1. Nuskaitymas (be skaičiavimo)
00017     auto start = std::chrono::high_resolution_clock::now();
```

```

00018     Container studentai = readStudentaiFromFile<Container>(filename);
00019     auto end = std::chrono::high_resolution_clock::now();
00020     std::chrono::duration<double> elapsed = end - start;
00021     std::cout << u8"      Nuskaitymas: " << elapsed.count() << u8" s\n";
00022
00023     // 2. Galutinių balų skaičiavimas (atskiras žingsnis)
00024     auto start2 = std::chrono::high_resolution_clock::now();
00025     calculateGalutinis(studentai);
00026     auto end2 = std::chrono::high_resolution_clock::now();
00027     std::chrono::duration<double> elapsed2 = end2 - start2;
00028     std::cout << u8"      Skaičiavimas:" << elapsed2.count() << u8" s\n";
00029
00030     // 3. Rikiavimas
00031     // vector/deque: std::sort | list: .sort() (sprendžiama if constexpr calculate.h)
00032     auto start3 = std::chrono::high_resolution_clock::now();
00033     sortStudentai(studentai, '3'); // rikiuojama pagal galutinisVid
00034     auto end3 = std::chrono::high_resolution_clock::now();
00035     std::chrono::duration<double> elapsed3 = end3 - start3;
00036     std::cout << u8"      Rikiavimas: " << elapsed3.count() << u8" s\n";
00037
00038     // 4. Skirstymas į dvi grupes (3 strategija)
00039     auto start4 = std::chrono::high_resolution_clock::now();
00040     Container vargsiukai = partitionStudentai(studentai);
00041     auto end4 = std::chrono::high_resolution_clock::now();
00042     std::chrono::duration<double> elapsed4 = end4 - start4;
00043     std::cout << u8"      Skirstymas: " << elapsed4.count() << u8" s\n";
00044 }
00045
00046 // -----
00047 // testContainers -- išskviečiama iš meniu (pasirinkimas 8)
00048 // Testuoja tą patį failą su visais trimis konteineriais
00049 // -----
00050 void testContainers(int n)
00051 {
00052     const std::string filename = "data/studentai" + std::to_string(n) + ".txt";
00053     std::cout << u8"=== Konteinerių palyginimas: " << n << u8" studentų ===\n";
00054     testContainer<std::vector<studentas>>>("vector", filename);
00055     testContainer<std::list<studentas>>("list", filename);
00056     testContainer<std::deque<studentas>>("deque", filename);
00057     std::cout << "\n";
00058 }

```

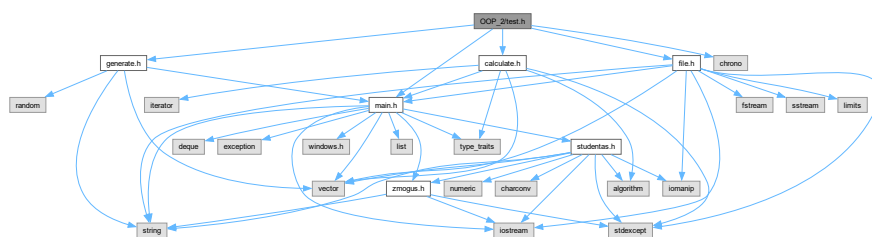
8.37 OOP_2/test.h Failo Nuoroda

```

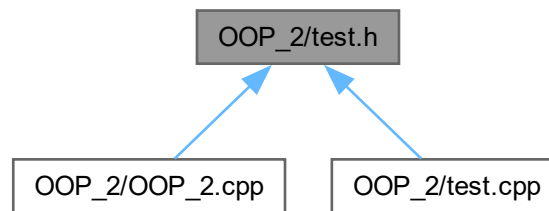
#include "main.h"
#include "file.h"
#include "generate.h"
#include "calculate.h"
#include <chrono>

```

Įtraukimo priklausomybių diagrama test.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
`void testGenerateStudentai (int n)`
- `template<typename Container>`
`void testData (int n)`

8.37.1 Funkcijos Dokumentacija

8.37.1.1 testGenerateStudentai()

```
template<typename Container>
void testGenerateStudentai (
    int n)
```

Apibrėžimas failo `test.h` eilutėje 15.

Ryšiai `generateStudentai()`, ir `writeStudentaiListToFile()`.

Naudojamas `runProgram()`.

8.37.1.2 testData()

```
template<typename Container>
void testData (
    int n)
```

Apibrėžimas failo `test.h` eilutėje 37.

Ryšiai `calculateGalutinis()`, `splitResult< Container >::kietiakai`, `readStudentaiFromFile()`, `sortStudentai()`, `splitStudentai()`, `splitResult< Container >::vargsiukai`, ir `writeStudentaiListToFile()`.

Naudojamas `runProgram()`.

8.38 test.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef TEST_H_DEFINED
00002 #define TEST_H_DEFINED
00003
00004 #include "main.h"
00005 #include "file.h"
00006 #include "generate.h"
00007 #include "calculate.h"
00008 #include <chrono>
00009
00010 // -----
00011 // testGenerateStudentai -- iškviečiama iš meniu (pasirinkimas 6)
00012 // Testuoja studentų failo generavimą su n studentų.
00013 // -----
00014 template<typename Container>
```

```

00015 void testGenerateStudentai(int n)
00016 {
00017     std::cout << u8"=== Studentų failo generavimo testavimas: " << n << u8" studentų ===\n";
00018
00019     auto start = std::chrono::high_resolution_clock::now();
00020     Container studentai = generateStudentai<Container>(n);
00021     auto end = std::chrono::high_resolution_clock::now();
00022     std::chrono::duration<double> elapsed = end - start;
00023     std::cout << u8" Generavimas: " << elapsed.count() << u8" s\n";
00024
00025     start = std::chrono::high_resolution_clock::now();
00026     writeStudentaiListToFile<Container>(studentai, "teststudentai" + std::to_string(n) + ".txt");
00027     end = std::chrono::high_resolution_clock::now();
00028     elapsed = end - start;
00029     std::cout << u8" Išvedimas į failą: " << elapsed.count() << u8" s\n";
00030 }
00031
00032 // -----
00033 // testData -- iškviečiama iš meniu (pasirinkimas 7)
00034 // Testuoja studentų duomenų apdorojimą su n studentų.
00035 // -----
00036 template<typename Container>
00037 void testData(int n)
00038 {
00039     auto startTotal = std::chrono::high_resolution_clock::now();
00040     std::cout << u8"Pradedamas testavimas su " << n << u8" studentų duomenimis.\n";
00041     std::cout << u8"-----\n";
00042
00043     auto start = std::chrono::high_resolution_clock::now();
00044     Container studentai = readStudentaiFromFile<Container>(
00045         "data/studentai" + std::to_string(n) + ".txt");
00046     auto end = std::chrono::high_resolution_clock::now();
00047     std::chrono::duration<double> elapsed = end - start;
00048     std::cout << n << u8" failo skaitymas užtruko: " << elapsed.count() << u8" sekundžių.\n";
00049
00050     auto start2 = std::chrono::high_resolution_clock::now();
00051     calculateGalutinis(studentai);
00052     auto end2 = std::chrono::high_resolution_clock::now();
00053     std::chrono::duration<double> elapsed2 = end2 - start2;
00054     std::cout << n << u8" galutinio balo skaičiavimas užtruko: "
00055         << elapsed2.count() << u8" sekundžių.\n";
00056
00057     auto start3 = std::chrono::high_resolution_clock::now();
00058     sortStudentai(studentai, '4');
00059     auto end3 = std::chrono::high_resolution_clock::now();
00060     std::chrono::duration<double> elapsed3 = end3 - start3;
00061     std::cout << n << u8" rikiavimas pagal medianą užtruko: "
00062         << elapsed3.count() << u8" sekundžių.\n";
00063
00064     auto start4 = std::chrono::high_resolution_clock::now();
00065     splitResult<Container> result = splitStudentai<Container>(studentai);
00066     auto end4 = std::chrono::high_resolution_clock::now();
00067     std::chrono::duration<double> elapsed4 = end4 - start4;
00068     std::cout << n << u8" skirstymas į kietiakai ir vargsiukai užtruko: "
00069         << elapsed4.count() << u8" sekundžių.\n";
00070
00071     auto start5 = std::chrono::high_resolution_clock::now();
00072     writeStudentaiListToFile<Container>(result.kietiakai, "testkietiakai" + std::to_string(n) +
00073         ".txt");
00074     auto end5 = std::chrono::high_resolution_clock::now();
00075     std::chrono::duration<double> elapsed5 = end5 - start5;
00076     std::cout << n << u8" Kietiakai failo generavimas užtruko: "
00077         << elapsed5.count() << u8" sekundžių.\n";
00078     auto start6 = std::chrono::high_resolution_clock::now();
00079     writeStudentaiListToFile<Container>(result.vargsiukai, "testvargsiukai" + std::to_string(n) +
00080         ".txt");
00081     auto end6 = std::chrono::high_resolution_clock::now();
00082     std::chrono::duration<double> elapsed6 = end6 - start6;
00083
00084     auto endTotal = std::chrono::high_resolution_clock::now();
00085     std::chrono::duration<double> elapsedTotal = endTotal - startTotal;
00086     std::cout << n << u8" Vargsiukai failo generavimas užtruko: "
00087         << elapsed6.count() << u8" sekundžių.\n";
00088     std::cout << n << u8" Visas testavimas užtruko: "
00089         << elapsedTotal.count() << u8" sekundžių.\n\n";
00090 }
00091 #endif

```

8.39 OOP_2/teststudentas.cpp Failo Nuoroda

```

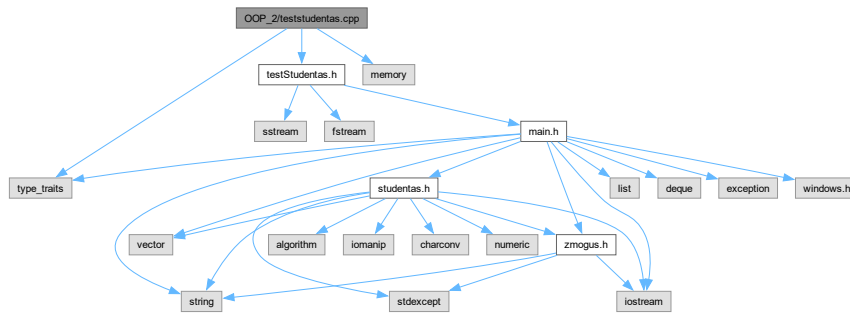
#include "testStudentas.h"
#include <memory>

```



```
#include <type_traits>
```

Įtraukimo priklausomybių diagrama teststudentas.cpp:



Funkcijos

- bool `testStudentasKlase()`

8.39.1 Funkcijos Dokumentacija

8.39.1.1 testStudentasKlase()

```
bool testStudentasKlase ()
```

Apibrėžimas failo `teststudentas.cpp` eilutėje 45.

Ryšiai `studentas::addPazymys()`, `studentas::appendListTo()`, `studentas::calculateGalutinis()`, `studentas::egzaminas()`, `zmogus::galutinis()`, `studentas::galutinisMed()`, `studentas::galutinisVid()`, `studentas::gyvuStudentu`, `zmogus::gyvuZmoniu`, `studentas::isvalyk()`, `studentas::nd()`, `studentas::parseFromLine()`, `zmogus::pavarde()`, `zmogus::pilnasVardas()`, `studentas::setEgzaminas()`, `studentas::setNd()`, `zmogus::setPavarde()`, `zmogus::setVardas()`, `zmogus::tipas()`, ir `zmogus::vardas()`.

Naudojamas `runProgram()`.

8.40 teststudentas.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "testStudentas.h"
00002 #include <memory>           // unique_ptr
00003 #include <type_traits>       // is_abstract, is_base_of
00004
00005 // =====
00006 // Testavimo infrastruktūra
00007 // =====
00008
00009 namespace {
00010
00011     int testuPraejo = 0;
00012     int testuKrito = 0;
00013
00014     void tikrink(bool salyga, const std::string& aprasymas)
00015     {
00016         if (salyga) {
00017             ++testuPraejo;
00018             std::cout << " [ OK ] " << aprasymas << "\n";
00019         }
00020         else {
00021             ++testuKrito;
00022             std::cout << " [KLaida] " << aprasymas << "\n";
00023         }
00024     }
00025
00026     void skyrius(const std::string& pavadinimas)
00027     {
00028         std::cout << "\n--- " << pavadinimas << " ---\n";
00029     }
00030

```

```

00031 // Testinis studentas su žinomomis reikšmėmis
00032 // ND: 5,5,5,5,5 -> vidurkis 5.0, mediana 5.0
00033 // Egzaminas 10 -> galutinis = 0.4*5 + 0.6*10 = 8.0
00034 studentas kurkTestini()
00035 {
00036     return studentas("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00037 }
00038
00039 } // anoniminis namespace
00040
00041 // =====
00042 // Pagrindinė testavimo funkcija
00043 // =====
00044
00045 bool testStudentasKlase()
00046 {
00047     testuPraejo = 0;
00048     testuKrito = 0;
00049
00050     std::cout << "\n=====\\n";
00051     std::cout << " KLASIU zmogus IR studentas TESTAVIMAS (v1.5)\\n";
00052     std::cout << "=====\\n";
00053
00054     const int pradinisStudentu = studentas::gyvuStudentu;
00055     const int pradinisZmoniu = zmogus::gyvuZmoniu;
00056
00057     // =====
00058     // A. ABSTRAKCIJOJI KLASE IR PAVELDEJIMAS (nauja v1.5)
00059     // =====
00060     skyrius("A1. Klase zmogus yra ABSTRAKTI");
00061     {
00062         tikrink(std::is_abstract<zmogus>::value,
00063             "std::is_abstract<zmogus> = true (turi grynai virtualiu metodu)");
00064         tikrink(!std::is_abstract<studentas>::value,
00065             "std::is_abstract<studentas> = false (visi metodai realizuoti)");
00066
00067         // Sie bandymai NESIKOMPILIUOTU -- todel patikra atliekama statiskai:
00068         //     zmogus z; // klaida: abstrakti klase
00069         //     zmogus* p = new zmogus(); // klaida: abstrakti klase
00070         tikrink(!std::is_default_constructible<zmogus>::value,
00071             "zmogus objekto sukurti NEIMANOMA (is_default_constructible = false)");
00072         tikrink(std::is_default_constructible<studentas>::value,
00073             "studentas objekta sukurti galima");
00074     }
00075
00076     skyrius("A2. Paveldejimo rysys");
00077     {
00078         tikrink((std::is_base_of<zmogus, studentas>::value),
00079             "studentas yra isvestine is zmogus");
00080         tikrink((std::is_convertible<studentas*, zmogus*>::value),
00081             "studentas* konvertuojasi i zmogus* (public paveldejimas)");
00082         tikrink(std::has_virtual_destructor<zmogus>::value,
00083             "zmogus turi VIRTUALU destruktoriu");
00084     }
00085
00086     skyrius("A3. Paveldeti bazines klases metodai");
00087     {
00088         studentas s = kurkTestini();
00089
00090         tikrink(s.vardas() == "Jonas", "paveldetas vardas() veikia");
00091         tikrink(s.pavarde() == "Jonaitis", "paveldetas pavarde() veikia");
00092         tikrink(s.pilnasVardas() == "Jonas Jonaitis", "paveldetas pilnasVardas() veikia");
00093
00094         s.setVardas("Petras");
00095         tikrink(s.vardas() == "Petras", "paveldetas setVardas() veikia");
00096
00097         bool metaTuscia = false;
00098         try { s.setPavarde(""); }
00099         catch (const std::invalid_argument&) { metaTuscia = true; }
00100         tikrink(metaTuscia, "paveldetas setPavarde() validuoja tuscia reiksme");
00101     }
00102
00103     skyrius("A4. POLIMORFIZMAS per bazines klases rodykle");
00104     {
00105         studentas s = kurkTestini();
00106         s.calculateGalutinis();
00107
00108         // Nuoroda i bazine klase
00109         zmogus& z = s;
00110         tikrink(z.vardas() == "Jonas", "prieiga prie vardo per zmogus& nuoroda");
00111         tikrink(z.tipas() == "Studentas",
00112             "virtualus tipas() grazina isvestines klases reiksme");
00113         tikrink(std::abs(z.galutinis() - 8.0) < 1e-9,
00114             "virtualus galutinis() = 8.0 (isvestines klases realizacija)");
00115
00116         // Rodykle i bazine klase
00117         std::unique_ptr<zmogus> p = std::make_unique<studentas>{

```

```

00118         "Ona", "Onaite", std::vector<int>{ 10, 10 }, 10);
00119         tikrink(p->tipas() == "Studentas", "virtualus tipas() per unique_ptr<zmogus>");
00120         tikrink(p->pilnasVardas() == "Ona Onaite", "paveldetas metodus per rodykle");
00121
00122         // dynamic_cast atgal i isvestine klase
00123         studentas* sp = dynamic_cast<studentas*>(p.get());
00124         tikrink(sp != nullptr, "dynamic_cast<studentas*> pavyko");
00125         if (sp) tikrink(sp->nd().size() == 2, "prieiga prie isvestines klases duomenu");
00126     }
00127
00128     skyrius("A5. VIRTUALUS destruktorius");
00129     {
00130         const int priesZmoniu = zmogus::gyvuZmoniu;
00131         const int priesStudentu = studentas::gyvuStudentu;
00132
00133         {
00134             // Objektas kuriamas kaip studentas, naikinamas per zmogus rodykle.
00135             // Be virtual destruktoriaus studentas laukai liktu neatlaisvinti.
00136             std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00137                 "Testas", "Testaitis", std::vector<int>{ 5, 5 }, 5);
00138
00139             tikrink(zmogus::gyvuZmoniu == priesZmoniu + 1,
00140                 "sukurus objekta zmogus skaitiklis padidejo");
00141             tikrink(studentas::gyvuStudentu == priesStudentu + 1,
00142                 "sukurus objekta studentas skaitiklis padidejo");
00143         }
00144
00145         tikrink(studentas::gyvuStudentu == priesStudentu,
00146             "~studentas() iskvistas naikinant per zmogus rodykle");
00147         tikrink(zmogus::gyvuZmoniu == priesZmoniu,
00148             "~zmogus() iskvistas po ~studentas()");
00149     }
00150
00151     skyrius("A6. Polimorfiniai operatoriai « ir »");
00152     {
00153         studentas s = kurkTestini();
00154         s.calculateGalutinis();
00155
00156         // operator« priima zmogus& ir kviecia virtualu print()
00157         zmogus& z = s;
00158         std::ostringstream oss;
00159         oss << z;
00160         tikrink(oss.str().find("Jonas") != std::string::npos,
00161             "operator« per zmogus& nuoroda kviecia studentas::print()");
00162
00163         // operator» priima zmogus& ir kviecia virtualu read()
00164         studentas s2;
00165         zmogus& z2 = s2;
00166         std::istringstream iss("Ona Onaite 9 8 7");
00167         iss >> z2;
00168         tikrink(s2.vardas() == "Ona",
00169             "operator» per zmogus& nuoroda kviecia studentas::read()");
00170         tikrink(s2.egzaminas() == 7, "nuskaityti duomenys teisingi");
00171     }
00172
00173     // =====
00174     // B. KONSTRUKTORIAI (patikra is v1.2)
00175     // =====
00176     skyrius("B1. Numatytais konstruktorius");
00177     {
00178         studentas s;
00179         tikrink(s.vardas().empty(), "vardas tuscias");
00180         tikrink(s.pavarde().empty(), "pavarde tuscia");
00181         tikrink(s.nd().empty(), "nd vektorius tuscias");
00182         tikrink(s.egzaminas() == 0, "egzaminas = 0");
00183         tikrink(s.galutinisVid() == 0.0, "galutinisVid = 0.0");
00184         tikrink(s.galutinisMed() == 0.0, "galutinisMed = 0.0");
00185         tikrink(!static_cast<bool>(s), "operator bool grazina false tusciam objektui");
00186     }
00187
00188     skyrius("B2. Pilnas konstruktorius");
00189     {
00190         studentas s("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00191         tikrink(s.vardas() == "Jonas", "vardas issaugotas (per bazine klase)");
00192         tikrink(s.pavarde() == "Jonaitis", "pavarde issaugota (per bazine klase)");
00193         tikrink(s.nd().size() == 5, "nd dydis = 5");
00194         tikrink(s.egzaminas() == 10, "egzaminas = 10");
00195         tikrink(static_cast<bool>(s), "operator bool grazina true");
00196     }
00197
00198     skyrius("B3. Validacija konstruktoriuje");
00199     {
00200         bool metaTusciaVarda = false;
00201         try { studentas s("", "Jonaitis", { 5 }, 10); }
00202         catch (const std::invalid_argument&) { metaTusciaVarda = true; }
00203         tikrink(metaTusciaVarda,
00204             "BAZINE klase meta invalid_argument, kai vardas tuscias");
00205     }

```

```

00205
00206     bool metaTusciaPavarde = false;
00207     try { studentas s("Jonas", "", { 5 }, 10); }
00208     catch (const std::invalid_argument&) { metaTusciaPavarde = true; }
00209     tikrink(metaTusciaPavarde,
00210             "BAZINE klase meta invalid_argument, kai pavarde tuscia");
00211
00212     bool metaBlogaND = false;
00213     try { studentas s("Jonas", "Jonaitis", { 5, 99 }, 10); }
00214     catch (const std::runtime_error&) { metaBlogaND = true; }
00215     tikrink(metaBlogaND,
00216             "ISVESTINE klase meta runtime_error, kai ND pazymys = 99");
00217
00218     bool metaBlogaEgz = false;
00219     try { studentas s("Jonas", "Jonaitis", { 5 }, 0); }
00220     catch (const std::runtime_error&) { metaBlogaEgz = true; }
00221     tikrink(metaBlogaEgz,
00222             "ISVESTINE klase meta runtime_error, kai egzaminas = 0");
00223 }
00224
00225 skyrius("B4. Konstruktorius is srauto");
00226 {
00227     std::istringstream iss("Petras Petraitis 8 9 10 7");
00228     studentas s(iss);
00229     tikrink(s.vardas() == "Petras", "vardas nuskaitytas");
00230     tikrink(s.pavarde() == "Petraitis", "pavarde nuskaityta");
00231     tikrink(s.nd().size() == 3, "nd dydis = 3 (paskutinis skaicius = egzaminas)");
00232     tikrink(s.egzaminas() == 7, "egzaminas = 7");
00233 }
00234
00235 // =====
00236 // C. RULE OF FIVE (patikra is v1.2 su paveldejimu)
00237 // =====
00238 skyrius("C1. Kopijavimo konstruktorius");
00239 {
00240     studentas originalas = kurkTestini();
00241     originalas.calculateGalutinis();
00242
00243     studentas kopija(originalas);
00244
00245     // Svarbiausia paveldejime: ar BAZINES klases laukai nukopijuoti?
00246     tikrink(kopija.vardas() == originalas.vardas(),
00247             "BAZINES klases vardas nukopijuotas (zmogus(other) kvietimas)");
00248     tikrink(kopija.pavarde() == originalas.pavarde(),
00249             "BAZINES klases pavarde nukopijuota");
00250     tikrink(kopija.nd() == originalas.nd(), "nd vektorius nukopijuotas");
00251     tikrink(kopija.egzaminas() == originalas.egzaminas(), "egzaminas nukopijuotas");
00252     tikrink(kopija.galutinisVid() == originalas.galutinisVid(),
00253             "galutinisVid nukopijuotas");
00254
00255     // Deep copy patikra
00256     kopija.setVardas("Pakeistas");
00257     tikrink(originalas.vardas() == "Jonas",
00258             "originalas NEPAKISTA, kai keiciama kopija (deep copy)");
00259 }
00260
00261 skyrius("C2. Kopijavimo priskyrimo operatorius");
00262 {
00263     studentas a = kurkTestini();
00264     a.calculateGalutinis();
00265     studentas b;
00266
00267     b = a;
00268
00269     tikrink(b.vardas() == a.vardas(),
00270             "BAZINES klases vardas priskirtas (zmogus::operator= kvietimas)");
00271     tikrink(b.nd() == a.nd(), "nd vektorius priskirtas");
00272     tikrink(b.galutinisVid() == a.galutinisVid(), "galutinisVid priskirtas");
00273
00274     a = a;
00275     tikrink(a.vardas() == "Jonas" && a.nd().size() == 5,
00276             "priskyrimas sau (a = a) nesugadina objekto");
00277
00278     studentas c, d;
00279     c = d = a;
00280     tikrink(c.vardas() == "Jonas" && d.vardas() == "Jonas",
00281             "grandinis priskyrimas (c = d = a) veikia");
00282 }
00283
00284 skyrius("C3. Perkelimo (move) konstruktorius");
00285 {
00286     studentas saltinis = kurkTestini();
00287     saltinis.calculateGalutinis();
00288     const double tiketinasVid = saltinis.galutinisVid();
00289
00290     studentas tikslas(std::move(saltinis));
00291

```

```

00292         tikrink(tikslas.vardas() == "Jonas",
00293                 "BAZINES klases vardas perkeltas (zmogus(std::move(other)))");
00294         tikrink(tikslas.nd().size() == 5, "nd vektorius perkeltas");
00295         tikrink(tikslas.galutinisVid() == tiketinasVid, "galutinisVid perkeltas");
00296
00297         tikrink(saltinis.vardas().empty(),
00298                 "saltinio BAZINES klases vardas tuscias po perkeliimo");
00299         tikrink(saltinis.nd().empty(), "saltinio nd tuscias po perkeliimo");
00300         tikrink(saltinis.egzaminas() == 0, "saltinio egzaminas = 0 po perkeliimo");
00301         tikrink(!static_cast<bool>(saltinis), "saltinis konvertuojasi i false");
00302     }
00303
00304     skyrius("C4. Perkeliimo (move) priskyrimo operatorius");
00305     {
00306         studentas saltinis = kurkTestini();
00307         saltinis.calculateGalutinis();
00308         studentas tikslas("Senas", "Senaitis", { 1, 1 }, 1);
00309
00310         tikslas = std::move(saltinis);
00311
00312         tikrink(tikslas.vardas() == "Jonas",
00313                 "BAZINES klases vardas perkeltas (zmogus::operator=(std::move))");
00314         tikrink(tikslas.nd().size() == 5, "nd perkeltas priskyrimu");
00315         tikrink(saltinis.vardas().empty(), "saltinis istustintas");
00316
00317         studentas x = kurkTestini();
00318         x = std::move(x);
00319         tikrink(x.vardas() == "Jonas" || x.vardas().empty(),
00320                 "move priskyrimas sau nesukelia luzimo");
00321     }
00322
00323     skyrius("C5. noexcept specifikatorius");
00324     {
00325         tikrink(std::is_nothrow_move_constructible<studentas>::value,
00326                 "studentas move konstruktorius yra noexcept");
00327         tikrink(std::is_nothrow_move_assignable<studentas>::value,
00328                 "studentas move priskyrimas yra noexcept");
00329     }
00330
00331     skyrius("C6. Destruktorius ir naikinimo tvarka");
00332     {
00333         const int priesStudentu = studentas::gyvuStudentu;
00334         const int priesZmoniu = zmogus::gyvuZmoniu;
00335         {
00336             studentas a = kurkTestini();
00337             studentas b = kurkTestini();
00338             studentas c(a);
00339             tikrink(studentas::gyvuStudentu == priesStudentu + 3,
00340                     "sukurti 3 studentai -- studentu skaitiklis padidejo 3");
00341             tikrink(zmogus::gyvuZmoniu == priesZmoniu + 3,
00342                     "kartu sukurtos 3 zmogus bazines dalys");
00343         }
00344         tikrink(studentas::gyvuStudentu == priesStudentu,
00345                 "~studentas() sumazino studentu skaitikli iki pradinio");
00346         tikrink(zmogus::gyvuZmoniu == priesZmoniu,
00347                 "~zmogus() sumazino zmoniu skaitikli iki pradinio");
00348     }
00349
00350     // =====
00351     // D. IVESTIES / ISVESTIES OPERATORIAI (patikra is v1.2)
00352     // =====
00353     skyrius("D1. Ivesties operatorius »");
00354     {
00355         std::stringstream iss("Ona Onaite 10 9 8 6");
00356         studentas s;
00357         iss >> s;
00358
00359         tikrink(s.vardas() == "Ona", "vardas nuskaitytas per »");
00360         tikrink(s.pavarde() == "Onaite", "pavarde nuskaityta per »");
00361         tikrink(s.nd().size() == 3, "nuskaityti 3 ND pazymiai");
00362         tikrink(s.egzaminas() == 6, "egzaminas = 6 (paskutinis skaicius)");
00363     }
00364
00365     skyrius("D2. Ivesties operatorius «");
00366     {
00367         studentas s = kurkTestini();
00368         s.calculateGalutinis();
00369
00370         std::ostringstream oss;
00371         oss << s;
00372         const std::string rez = oss.str();
00373
00374         tikrink(rez.find("Jonas") != std::string::npos, "isvestyje yra vardas");
00375         tikrink(rez.find("Jonaitis") != std::string::npos, "isvestyje yra pavarde");
00376         tikrink(rez.find("8.00") != std::string::npos,
00377                 "isvestyje yra galutinis balas 8.00 (0.4*5 + 0.6*10)");
00378         tikrink(rez.size() >= 60, "isvestis suformatuota stulpeliais");

```

```

00379     }
00380
00381     skyrius("D3. Ivestis is failo (parseFromLine)");
00382     {
00383         studentas s;
00384         s.parseFromLine("Antanas Antanaitis 7 8 9 10 5", 4, 1);
00385
00386         tikrink(s.vardas() == "Antanas", "vardas parsintas");
00387         tikrink(s.pavarde() == "Antanaitis", "pavarde parsinta");
00388         tikrink(s.nd().size() == 4, "parsinti 4 ND pazymiai");
00389         tikrink(s.egzaminas() == 5, "egzaminas = 5");
00390
00391         bool metaBloga = false;
00392         try { studentas bad; bad.parseFromLine("Vardas Pavarde 99 5", 2, 7); }
00393         catch (const std::runtime_error&) { metaBloga = true; }
00394         tikrink(metaBloga, "meta runtime_error, kai pazymys uz ribu");
00395
00396         bool metaTrukstama = false;
00397         try { studentas bad; bad.parseFromLine("TikVardas", 2, 3); }
00398         catch (const std::runtime_error&) { metaTrukstama = true; }
00399         tikrink(metaTrukstama, "meta runtime_error, kai truksta duomenu");
00400     }
00401
00402     skyrius("D4. Iinvestis i faila (appendListTo)");
00403     {
00404         studentas s("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00405
00406         std::string buferis;
00407         s.appendListTo(buferis);
00408
00409         tikrink(buferis.find("Rasa") != std::string::npos, "buferyje yra vardas");
00410         tikrink(buferis.find("Rasaite") != std::string::npos, "buferyje yra pavarde");
00411         tikrink(buferis.back() == '\n', "eilute baigiasi nauja eilute");
00412         tikrink(buferis.size() == 25 + 25 + 4 * 10 + 1,
00413                 "eilutes ilgis atitinka formatavima (25+25+4*10+1)");
00414
00415         studentas s2;
00416         s2.parseFromLine(buferis, 3, 1);
00417         tikrink(s2.vardas() == s.vardas() && s2.nd() == s.nd() &&
00418                 s2.egzaminas() == s.egzaminas(),
00419                 "isvesta eilute korektiskai nuskaitoma atgal (round-trip)");
00420     }
00421
00422     // =====
00423     // E. PAPILDOMI OPERATORIAI (patikra is vl.2)
00424     // =====
00425     skyrius("E1. Lyginimo operatoriai == ir !=");
00426     {
00427         studentas a("Jonas", "Jonaitis", { 5, 6 }, 7);
00428         studentas b("Jonas", "Jonaitis", { 5, 6 }, 7);
00429         studentas c("Petras", "Petraitis", { 5, 6 }, 7);
00430
00431         tikrink(a == b, "vienodi objektai lygus (==)");
00432         tikrink(a != c, "skirtingi objektai nelygus (!=)");
00433         tikrink(!(a == c), "== grazina false skirtingiems");
00434     }
00435
00436     skyrius("E2. Rikiavimo operatoriai < ir >");
00437     {
00438         studentas a("Jonas", "Adamkus", { 5 }, 7);
00439         studentas b("Jonas", "Zukauskas", { 5 }, 7);
00440
00441         tikrink(a < b, "Adamkus < Zukauskas (pagal pavarde)");
00442         tikrink(b > a, "Zukauskas > Adamkus");
00443
00444         studentas c("Antanas", "Petraitis", { 5 }, 7);
00445         studentas d("Zigmas", "Petraitis", { 5 }, 7);
00446         tikrink(c < d, "esant vienodoms pavardems lyginama pagal varda");
00447
00448         std::vector<studentas> v = { b, a };
00449         std::sort(v.begin(), v.end());
00450         tikrink(v[0].pavarde() == "Adamkus", "std::sort su operator< veikia");
00451     }
00452
00453     skyrius("E3. Indeksavimo operatorius []");
00454     {
00455         studentas s("Jonas", "Jonaitis", { 3, 6, 9 }, 10);
00456
00457         tikrink(s[0] == 3, "s[0] = 3");
00458         tikrink(s[1] == 6, "s[1] = 6");
00459         tikrink(s[2] == 9, "s[2] = 9");
00460
00461         bool metaUzRibu = false;
00462         try { int x = s[99]; (void)x; }
00463         catch (const std::out_of_range&) { metaUzRibu = true; }
00464         tikrink(metaUzRibu, "meta out_of_range, kai indeksas uz ribu");
00465     }

```

```

00466     skyrius("E4. Skaiciavimo metodas calculateGalutinis");
00467     {
00468         // ND: 2,4,6,8,10 -> vidurkis 6.0, mediana 6.0
00469         studentas s("Testas", "Testaitis", { 2, 4, 6, 8, 10 }, 5);
00470         s.calculateGalutinis();
00471
00472         tikrink(std::abs(s.galutinisVid() - 5.4) < 1e-9,
00473             "galutinisVid = 5.4 (0.4*6.0 + 0.6*5)");
00474         tikrink(std::abs(s.galutinisMed() - 5.4) < 1e-9,
00475             "galutinisMed = 5.4 (mediana 6.0)");
00476
00477         // Lyginis medianos atvejis: 1,2,3,4 -> (2+3)/2 = 2.5
00478         studentas s2("Testas", "Testaitis", { 1, 2, 3, 4 }, 10);
00479         s2.calculateGalutinis();
00480         tikrink(std::abs(s2.galutinisMed() - (0.4 * 2.5 + 0.6 * 10)) < 1e-9,
00481             "mediana lyginiam kiekiui = (2+3)/2 = 2.5");
00482
00483         // Tuscias ND sarasas
00484         studentas s3;
00485         s3.setVardas("Tuscias");
00486         s3.setPavarde("Tusciaitis");
00487         s3.setEgzaminas(8);
00488         s3.calculateGalutinis();
00489         tikrink(std::abs(s3.galutinisVid() - 4.8) < 1e-9,
00490             "tuscias ND sarasas: galutinis = 0.6*8 = 4.8");
00491     }
00492
00493     skyrius("E5. Set'eriai ir isvalyk");
00494     {
00495         studentas s;
00496         s.setVardas("Naujas");           // paveldetas is zmogus
00497         s.setPavarde("Naujaitis");      // paveldetas is zmogus
00498         s.setNd({ 5, 6, 7 });           // studentas metodas
00499         s.setEgzaminas(9);              // studentas metodas
00500
00501         tikrink(s.vardas() == "Naujas", "paveldetas setVardas veikia");
00502         tikrink(s.nd().size() == 3, "setNd veikia");
00503         tikrink(s.egzaminas() == 9, "setEgzaminas veikia");
00504
00505         s.addPazymys(10);
00506         tikrink(s.nd().size() == 4 && s.nd().back() == 10, "addPazymys veikia");
00507
00508         bool metaBloga = false;
00509         try { s.setEgzaminas(50); }
00510         catch (const std::runtime_error&) { metaBloga = true; }
00511         tikrink(metaBloga, "setEgzaminas meta klaida, kai reiksme uz ribu");
00512
00513         s.isvalyk();
00514         tikrink(s.vardas().empty() && s.nd().empty() && s.egzaminas() == 0,
00515             "isvalyk() istustina ir bazines, ir isvestines klases laukus");
00516     }
00517
00518     skyrius("E6. Veikimas konteineriuose (move semantika)");
00519     {
00520         std::vector<studentas> v;
00521         v.reserve(4);
00522         for (int i = 0; i < 4; ++i)
00523             v.push_back(studentas("Vardas" + std::to_string(i),
00524                 "Pavarde" + std::to_string(i),
00525                 { 5, 6, 7 }, 8));
00526
00527         tikrink(v.size() == 4, "4 objektai ideti i vektoriu");
00528
00529         v.reserve(100);
00530         tikrink(v[0].vardas() == "Vardas0" && v[3].vardas() == "Vardas3",
00531             "duomenys islieka po vektoriaus perskirstymo");
00532
00533         std::vector<studentas> v2 = std::move(v);
00534         tikrink(v2.size() == 4, "vektorius perkeltas su std::move");
00535         tikrink(v2[2].vardas() == "Vardas2", "duomenys teisingi po perkavimo");
00536     }
00537
00538     skyrius("E7. Polimorfinis konteineris (nauja v1.5)");
00539     {
00540         // Konteineris bazines klases rodykliu -- leidzia laikyti
00541         // skirtingu isvestiniu klasiu objektus vietoje
00542         std::vector<std::unique_ptr<zmogus> zmones;
00543         zmones.push_back(std::make_unique<studentas>(
00544             "Pirmas", "Pirmaitis", std::vector<int>{ 10, 10 }, 10));
00545         zmones.push_back(std::make_unique<studentas>(
00546             "Antras", "Antraitis", std::vector<int>{ 1, 1 }, 1));
00547
00548         for (auto& z : zmones) {
00549             studentas* sp = dynamic_cast<studentas*>(z.get());
00550             if (sp) sp->calculateGalutinis();
00551         }
00552     }

```

```

00553
00554     tikrink(zmones.size() == 2, "polimorfiniame konteineryje 2 objektai");
00555     tikrink(zmones[0]->tipas() == "Studentas", "virtualus tipas() veikia");
00556     tikrink(std::abs(zmones[0]->galutinis() - 10.0) < 1e-9,
00557             "pirmojo galutinis() = 10.0");
00558     tikrink(std::abs(zmones[1]->galutinis() - 1.0) < 1e-9,
00559             "antrojo galutinis() = 1.0");
00560
00561     // Isvedimas per polimorfini operator«
00562     std::ostringstream oss;
00563     for (const auto& z : zmones)
00564         oss << *z << "\n";
00565     tikrink(oss.str().find("Pirmas") != std::string::npos &&
00566             oss.str().find("Antras") != std::string::npos,
00567             "polimorfinis operator« isveda visus objektus");
00568 }
00569
00570 // =====
00571 // Rezultatai
00572 // =====
00573 const int galutinisStudentu = studentas::gyvuStudentu;
00574 const int galutinisZmoniu = zmogus::gyvuZmoniu;
00575
00576 std::cout << "\n=====\\n";
00577 std::cout << " REZULTATAI\\n";
00578 std::cout << "-----\\n";
00579 std::cout << " Praejo: " << testuPraejo << "\\n";
00580 std::cout << " Krito: " << testuKrito << "\\n";
00581 std::cout << " Is viso: " << (testuPraejo + testuKrito) << "\\n";
00582 std::cout << "-----\\n";
00583 std::cout << " studentas objektu nutekejimas: "
00584     << (galutinisStudentu == pradinisStudentu ? "OK" : "KLaida") << "\\n";
00585 std::cout << " zmogus objektu nutekejimas: "
00586     << (galutinisZmoniu == pradinisZmoniu ? "OK" : "KLaida") << "\\n";
00587 std::cout << "=====\\n\\n";
00588
00589 return testuKrito == 0
00590     && galutinisStudentu == pradinisStudentu
00591     && galutinisZmoniu == pradinisZmoniu;
00592 }

```

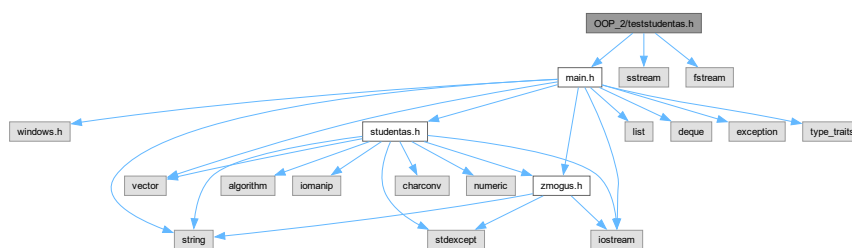
8.41 OOP_2/teststudentas.h Failo Nuoroda

```

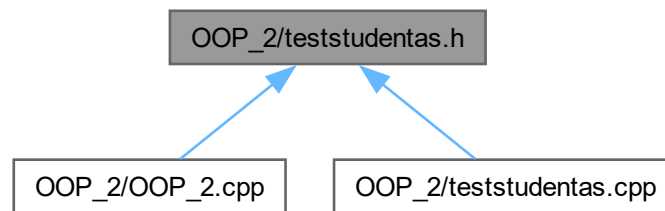
#include "main.h"
#include <sstream>
#include <fstream>

```

Įtraukimo priklausomybių diagrama teststudentas.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- bool `testStudentasKlase` ()

8.41.1 Funkcijos Dokumentacija

8.41.1.1 testStudentasKlase()

bool `testStudentasKlase` ()

Apibrėžimas failo `teststudentas.cpp` eilutėje 45.

8.42 teststudentas.h

Eiti į šio failo dokumentaciją.

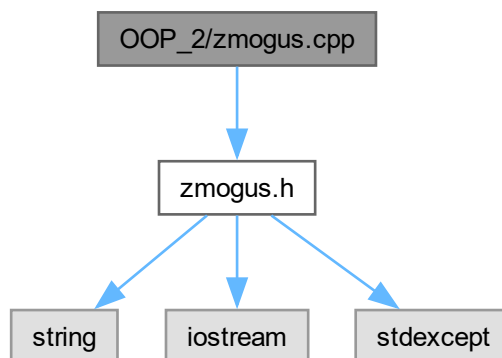
```

00001 #ifndef TESTSTUDENTAS_H_DEFINED
00002 #define TESTSTUDENTAS_H_DEFINED
00003
00004 #include "main.h"
00005 #include <sstream>
00006 #include <fstream>
00007
00008 // -----
00009 // testStudentasKlase -- rankinis klasių metodų testas.
00010 //
00011 // v1.5: papildyta abstrakčios bazinės klasės žmogus ir
00012 // paveldėjimo testais (A skyrius), išlaikant visus v1.2
00013 // testus (B-E skyriai).
00014 //
00015 // Iškviečiama iš meniu (pasirinkimas 9).
00016 // Gražina true, jei visi testai praėjo.
00017 // -----
00018 bool testStudentasKlase();
00019
00020 #endif // TESTSTUDENTAS_H_DEFINED
  
```

8.43 OOP_2/zmogus.cpp Failo Nuoroda

```
#include "zmogus.h"
```

Įtraukimo priklausomybių diagrama zmogus.cpp:



Funkcijos

- `std::ostream & operator<< (std::ostream &os, const zmogus &z)`
Polimorfinis išvesties operatorius.
- `std::istream & operator>> (std::istream &is, zmogus &z)`
Polimorfinis įvesties operatorius.

8.43.1 Funkcijos Dokumentacija

8.43.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const zmogus & z)
```

Polimorfinis išvesties operatorius.

Priima `zmogus` & nuorodą, bet kviečia virtualų `print()` metodą. Todėl veikia su bet kuria išvestine klase — dabartine ir būsimomis.

Parametrai

<i>os</i>	Išvesties srautas
<i>z</i>	Išvedamas objektas

Gražina

Nuoroda į srautą

Pavyzdys

```

studentas s("Jonas", "Jonaitis", {5, 6}, 7);
s.calculateGalutinis();

std::cout << s << "\n";           // į ekraną

std::ofstream f("rezultatai.txt");
f << s << "\n";                   // į failą

zmogus& z = s;
std::cout << z << "\n";           // per bazinės klasės nuorodą

```

Apibrėžimas failo `zmogus.cpp` eilutėje 112.

Ryšiai `zmogus::print()`.

8.43.1.2 operator>>()

```

std::istream & operator>> (
    std::istream & is,
    zmogus & z)

```

Polimorfinis įvesties operatorius.

Priima `zmogus` ir nuorodą, bet kviečia virtualų `read()` metodą.

Parametrai

<i>is</i>	Įvesties srautas
<i>z</i>	Objektas, į kurį nuskaitomi duomenys

Gražina

Nuoroda į srautą

Išimtys

<code>std::runtime_error</code>	jei duomenys netinkami
---------------------------------	------------------------

Apibrėžimas failo `zmogus.cpp` eilutėje 118.

Ryšiai `zmogus::read()`.

8.44 zmogus.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "zmogus.h"
00002
00003 // -----
00004 // Statinis narys -- gyvuojančių objektų skaitiklis
00005 // -----
00006 int zmogus::gyvuZmoniu = 0;
00007
00008 // =====
00009 // Konstruktoriai (protected)
00010 // =====
00011
00012 zmogus::zmogus()
00013     : vardas_(), pavarde_()
00014 {
00015     ++gyvuZmoniu;
00016 }
00017
00018 zmogus::zmogus(const std::string& vardas, const std::string& pavarde)
00019     : vardas_(vardas), pavarde_(pavarde)
00020 {
00021     if (vardas_.empty())
00022         throw std::invalid_argument("Vardas negali būti tuščias");
00023     if (pavarde_.empty())
00024         throw std::invalid_argument("Pavardė negali būti tuščia");

```

```

00025
00026     ++gyvuZmoniu;
00027 }
00028
00029 // -----
00030 // Kopijavimo konstruktorius
00031 // -----
00032 zmogus::zmogus(const zmogus& other)
00033     : vardas_(other.vardas_), pavarde_(other.pavarde_)
00034 {
00035     ++gyvuZmoniu;
00036 }
00037
00038 // -----
00039 // Perkėlimo (move) konstruktorius
00040 // noexcept -- kad std::vector naudotų perkėlimą, ne kopijavimą
00041 // -----
00042 zmogus::zmogus(zmogus&& other) noexcept
00043     : vardas_(std::move(other.vardas_)), pavarde_(std::move(other.pavarde_))
00044 {
00045     ++gyvuZmoniu;
00046 }
00047
00048 // -----
00049 // Kopijavimo priskyrimo operatorius
00050 // -----
00051 zmogus& zmogus::operator=(const zmogus& other)
00052 {
00053     if (this == &other)        // apsauga nuo priskyrimo sau
00054         return *this;
00055
00056     vardas_ = other.vardas_;
00057     pavarde_ = other.pavarde_;
00058
00059     return *this;
00060 }
00061
00062 // -----
00063 // Perkėlimo (move) priskyrimo operatorius
00064 // -----
00065 zmogus& zmogus::operator=(zmogus&& other) noexcept
00066 {
00067     if (this == &other)        // apsauga nuo priskyrimo sau
00068         return *this;
00069
00070     vardas_ = std::move(other.vardas_);
00071     pavarde_ = std::move(other.pavarde_);
00072
00073     return *this;
00074 }
00075
00076 // =====
00077 // VIRTUALUS destruktorius
00078 //
00079 // Kviečiamas PO išvestinės klasės destruktoriaus.
00080 // Naikinimo tvarka: ~studentas() → ~zmogus()
00081 // =====
00082 zmogus::~zmogus()
00083 {
00084     vardas_.clear();
00085     pavarde_.clear();
00086
00087     --gyvuZmoniu;
00088 }
00089
00090 // =====
00091 // Set'eriai su validacija
00092 // =====
00093
00094 void zmogus::setVardas(const std::string& v)
00095 {
00096     if (v.empty())
00097         throw std::invalid_argument("Vardas negali būti tuščias");
00098     vardas_ = v;
00099 }
00100
00101 void zmogus::setPavarde(const std::string& p)
00102 {
00103     if (p.empty())
00104         throw std::invalid_argument("Pavardė negali būti tuščia");
00105     pavarde_ = p;
00106 }
00107
00108 // =====
00109 // POLIMORFINIAI operatoriai
00110 // =====
00111

```

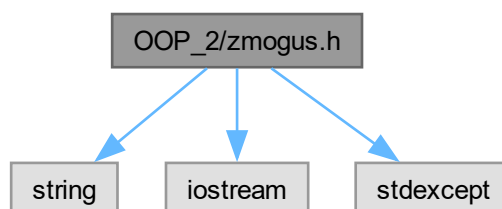
```
00112 std::ostream& operator<<(std::ostream& os, const zmogus& z)
00113 {
00114     z.print(os);                // virtualus kvietimas
00115     return os;
00116 }
00117
00118 std::istream& operator>>(std::istream& is, zmogus& z)
00119 {
00120     return z.read(is);          // virtualus kvietimas
00121 }
```

8.45 OOP_2/zmogus.h Failo Nuoroda

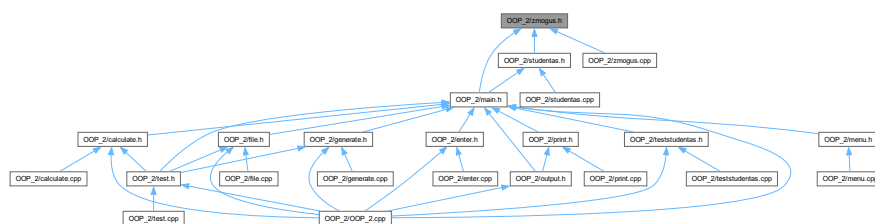
Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.

```
#include <string>
#include <iostream>
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama zmoqus.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class **zmogus**

ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas.

Funkcijos

- `std::ostream & operator<< (std::ostream &os, const zmogus &z)`

Polimorfinis išvesties operatorius.

- `std::istream & operator>> (std::istream &is, zmogus &z)`

Polimorfinis jvesties operatorius.

8.45.1 Smulkus aprašymas

Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.

Autorius

Marius Augustinas

Data

2026

Apibrėžimas faile [zmogus.h](#).

8.45.2 Funkcijos Dokumentacija

8.45.2.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const zmogus & z)
```

Polimorfinis išvesties operatorius.

Priima [zmogus](#) & nuorodą, bet kviečia virtualų print() metodą. Todėl veikia su bet kokia išvestine klase — dabartine ir būsimomis.

Parametrai

<i>os</i>	Išvesties srautas
<i>z</i>	Išvedamas objektas

Gražina

Nuoroda į srautą

Pavyzdys

```
studentas s("Jonas", "Jonaitis", {5, 6}, 7);
s.calculateGalutinis();

std::cout << s << "\n";           // į ekraną

std::ofstream f("rezultatai.txt");
f << s << "\n";                   // į failą

zmogus& z = s;
std::cout << z << "\n";           // per bazinės klasės nuorodą
```

Apibrėžimas failo [zmogus.cpp](#) eilutėje 112.

Ryšiai [zmogus::print\(\)](#).

8.45.2.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    zmogus & z)
```

Polimorfinis įvesties operatorius.

Priima [zmogus](#) & nuorodą, bet kviečia virtualų read() metodą.

Parametrai

<i>is</i>	Įvesties srautas
-----------	------------------

z	Objektas, į kurį nuskaitomi duomenys
---	--------------------------------------

Gražina

Nuoroda į srautą

Išimtys

<code>std::runtime_error</code>	jei duomenys netinkami
---------------------------------	------------------------

Apibrėžimas failo `zmogus.cpp` eilutėje 118.

Ryšiai `zmogus::read()`.

8.46 zmogus.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ZMOGUS_H_DEFINED
00002 #define ZMOGUS_H_DEFINED
00003
00004 #include <string>
00005 #include <iostream>
00006 #include <stdexcept>
00007
00008 /**
00009  * @file zmogus.h
00010  * @brief Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.
00011  * @author Marius Augustinas
00012  * @date 2026
00013  */
00014
00015 /**
00016  * @class zmogus
00017  * @brief ABSTRAKTI bazinė klasė -- bendras bet kokio žmogaus aprašymas.
00018  *
00019  * Laiko vardą ir pavardę bei apibrėžia interfeisą, kurį privalo
00020  * realizuoti kiekviena išvestinė klasė.
00021  *
00022  * @par Kodėl klasė abstrakti
00023  * Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais**:
00024  *
00025  * 1. **Grynai virtualūs metodai** ('= 0') -- galutinis(), print(),
00026  *     read(), tipas(). Kompiliatorius neleidžia kurti objektų iš klasės,
00027  *     kuri turi bent vieną neapibrėžtą virtualų metodą.
00028  * 2. **`protected` konstruktoriai** -- prieinami tik išvestinėms klasėms.
00029  *     Tai antras apsaugos sluoksnis.
00030  *
00031  * @par Naudojimo pavyzdys
00032  * @code
00033  * // zmogus z; // KLAIDA: abstrakti klasė
00034  * // zmogus* p = new zmogus(); // KLAIDA: abstrakti klasė
00035  *
00036  * studentas s("Jonas", "Jonaitis", {5, 6, 7}, 8); // OK
00037  * zmogus& z = s; // OK -- nuoroda
00038  * std::cout << z.tipas() << "\n"; // "Studentas"
00039  *
00040  * std::unique_ptr<zmogus> p =
00041  *     std::make_unique<studentas>("Ona", "Onaite", {10}, 10); // OK
00042  * @endcode
00043  *
00044  * @warning Destruktorius PRIVALO būti virtualus -- kitaip naikinant
00045  *     objektą per bazinės klasės rodyklę išvestinės klasės
00046  *     laukai liktų neatlaisvinti.
00047  *
00048  * @see studentas
00049  */
00050 class zmogus {
00051 protected:
00052     std::string vardas_; ///< Žmogaus vardas
00053     std::string pavarde_; ///< Žmogaus pavardė
00054
00055     /**
00056      * @brief Numatytasis konstruktorius.
00057      *
00058      * Sukuria objektą su tuščiu vardu ir pavarde.
00059      * `protected` -- kviečiamas tik iš išvestinių klasių.
```

```

00060     */
00061     zmogus();
00062
00063     /**
00064     * @brief Pilnas konstruktorius su validacija.
00065     *
00066     * @param vardas Žmogaus vardas (negali būti tuščias)
00067     * @param pavarde Žmogaus pavardė (negali būti tuščia)
00068     *
00069     * @throws std::invalid_argument jei vardas arba pavardė tuščia
00070     */
00071     zmogus(const std::string& vardas, const std::string& pavarde);
00072
00073     /**
00074     * @brief Kopijavimo konstruktorius.
00075     *
00076     * Sukuria gilia kopiją -- `std::string` laukai kopijuoja
00077     * savo vidinius buferius.
00078     *
00079     * @param other Kopijuojamas objektas
00080     */
00081     zmogus(const zmogus& other);
00082
00083     /**
00084     * @brief Perkėlimo (move) konstruktorius.
00085     *
00086     * Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas
00087     * galiojančioje, bet tuščioje būsenoje.
00088     *
00089     * @param other Objektas, iš kurio perkeliami duomenys
00090     *
00091     * @note `noexcept` būtinas: be jo `std::vector` persikirstymo metu
00092     *        naudotų kopijavimo konstruktorių (`std::move_if_noexcept`).
00093     */
00094     zmogus(zmogus&& other) noexcept;
00095
00096     /**
00097     * @brief Kopijavimo priskyrimo operatorius.
00098     *
00099     * @param other Priskiriamas objektas
00100     * @return Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)
00101     *
00102     * @note Turi apsaugą nuo priskyrimo sau (`this == &other`).
00103     */
00104     zmogus& operator=(const zmogus& other);
00105
00106     /**
00107     * @brief Perkėlimo (move) priskyrimo operatorius.
00108     *
00109     * @param other Objektas, iš kurio perkeliami duomenys
00110     * @return Nuoroda į šį objektą
00111     */
00112     zmogus& operator=(zmogus&& other) noexcept;
00113
00114 public:
00115     /**
00116     * @brief Gyvuojančių objektų skaitiklis.
00117     *
00118     * Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje.
00119     * Naudojamas testuose objektų nutekėjimui aptikti.
00120     */
00121     static int gyvuZmoniu;
00122
00123     /**
00124     * @brief Virtualus destruktorius.
00125     *
00126     * @warning `virtual` kvalifikatorius kritiškai svarbus. Be jo:
00127     * @code
00128     * zmogus* p = new studentas(...);
00129     * delete p;      // kviečiamas TIK ~zmogus()
00130     *               // studentas laukai NEATLAISVINAMI
00131     * @endcode
00132     *
00133     * Su `virtual` naikinimo tvarka teisinga: `~studentas()` + `~zmogus()`.
00134     */
00135     virtual ~zmogus();
00136
00137     /**
00138     * @brief Gražina žmogaus vardą.
00139     * @return Konstanti nuoroda į vardą (be kopijavimo)
00140     */
00141     inline const std::string& vardas() const { return vardas_; }
00142
00143     /**
00144     * @brief Gražina žmogaus pavardę.
00145     * @return Konstanti nuoroda į pavardę (be kopijavimo)
00146     */

```



```

00147     inline const std::string& pavarde() const { return pavarde_; }
00148
00149     /**
00150      * @brief Gražina pilną vardą.
00151      * @return Vardas ir pavardė, atskirti tarpu
00152      */
00153     inline std::string pilnasVardas() const { return vardas_ + " " + pavarde_; }
00154
00155     /**
00156      * @brief Nustato vardą su validacija.
00157      * @param v Naujas vardas
00158      * @throws std::invalid_argument jei vardas tuščias
00159      */
00160     void setVardas(const std::string& v);
00161
00162     /**
00163      * @brief Nustato pavardę su validacija.
00164      * @param p Nauja pavardė
00165      * @throws std::invalid_argument jei pavardė tuščia
00166      */
00167     void setPavarde(const std::string& p);
00168
00169     /**
00170      * @brief Gražina galutinį įvertinimą.
00171      *
00172      * Grynai virtualus -- kiekvienas žmogaus tipas skaičiuoja jį savaip.
00173      * Studentui tai galutinis balas pagal namų darbų vidurkį ir egzaminą.
00174      *
00175      * @return Galutinis įvertinimas
00176      */
00177     virtual double galutinis() const = 0;
00178
00179     /**
00180      * @brief Išveda objektą į srautą.
00181      *
00182      * Grynai virtualus. Realizuoja polimorfinį `operator«`.
00183      *
00184      * @param os Išvesties srautas
00185      */
00186     virtual void print(std::ostream& os) const = 0;
00187
00188     /**
00189      * @brief Nuskaito objektą iš srauto.
00190      *
00191      * Grynai virtualus. Realizuoja polimorfinį `operator»`.
00192      *
00193      * @param is Įvesties srautas
00194      * @return Nuoroda į tą patį srautą (grandininiam skaitymui)
00195      */
00196     virtual std::istream& read(std::istream& is) = 0;
00197
00198     /**
00199      * @brief Gražina žmogaus tipo pavadinimą.
00200      *
00201      * Grynai virtualus. Naudojamas diagnostikai ir logams.
00202      *
00203      * @return Tipo pavadinimas, pvz. "Studentas"
00204      */
00205     virtual std::string tipas() const = 0;
00206 };
00207
00208 /**
00209  * @brief Polimorfinis išvesties operatorius.
00210  *
00211  * Priima `zmogus&` nuorodą, bet kviečia virtualų print() metodą.
00212  * Todėl veikia su bet kuria išvestine klase -- dabartine ir būsimomis.
00213  *
00214  * @param os Išvesties srautas
00215  * @param z Išvedamas objektas
00216  * @return Nuoroda į srautą
00217  *
00218  * @par Pavyzdys
00219  * @code
00220  * studentas s("Jonas", "Jonaitis", {5, 6}, 7);
00221  * s.calculateGalutinis();
00222  *
00223  * std::cout << s << "\n";           // į ekraną
00224  *
00225  * std::ofstream f("rezultatai.txt");
00226  * f << s << "\n";                   // į failą
00227  *
00228  * zmogus& z = s;
00229  * std::cout << z << "\n";           // per bazinės klasės nuorodą
00230  * @endcode
00231  */
00232     std::ostream& operator<(std::ostream& os, const zmogus& z);
00233

```

```
00234 /**
00235  * @brief Polimorfinis įvesties operatorius.
00236  *
00237  * Priima `zmogus` nuorodą, bet kviečia virtualų read() metodą.
00238  *
00239  * @param is Įvesties srautas
00240  * @param z Objektas, į kurį nuskaitomi duomenys
00241  * @return Nuoroda į srautą
00242  *
00243  * @throws std::runtime_error jei duomenys netinkami
00244  */
00245 std::istream& operator>(std::istream& is, zmogus& z);
00246
00247 #endif // ZMOGUS_H_DEFINED
```

8.47 README.md Failo Nuoroda